claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67\agent-a3a4bc3405adb215b.jsonl`
- SHA-256: `7606758931a1351496fa65fadd2361778b0476a1059bcf4d45e2294810db4788`
- Source modified: `2026-06-18T01:36:14+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:33:12.223Z)

```
Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read
READ-ONLY in this worktree (origin/master), DO NOT modify:
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch
what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE
trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the
trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list
is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py,
backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.

CLASS: numeric / statistical / off-by-one.
Across eval + pipeline windowing: div-by-zero, n=0/1 means, NaN/inf, fps/frame<->time rounding,
interval boundary inclusivity, IoU/tolerance match edges, bootstrap/sign-test/CI math, cap/trim
boundary conditions.
```

2. assistant (2026-06-18T01:33:14.555Z)

```
I'll start by exploring the eval and pipeline windowing code to hunt for
numeric/statistical/off-by-one bugs. Let me pull the repo first and map the relevant files.
```

3. user (2026-06-18T01:33:16.729Z)

```
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
```

4. user (2026-06-18T01:33:16.961Z)

```
.claude\worktrees\hunt-base\backend\eval\ablation.py
.claude\worktrees\hunt-base\backend\eval\__init__.py
.claude\worktrees\hunt-base\backend\eval\annotations.py
.claude\worktrees\hunt-base\backend\eval\audio\e1_probe.py
.claude\worktrees\hunt-base\backend\eval\audio\__init__.py
.claude\worktrees\hunt-base\backend\eval\batch.py
.claude\worktrees\hunt-base\backend\eval\calibrate_local.py
.claude\worktrees\hunt-base\backend\eval\calibrate_wasb.py
.claude\worktrees\hunt-base\backend\eval\calibration.py
.claude\worktrees\hunt-base\backend\eval\classifier.py
.claude\worktrees\hunt-base\backend\eval\distill_local.py
.claude\worktrees\hunt-base\backend\eval\eval_partial_labels.py
.claude\worktrees\hunt-base\backend\eval\eval_stats.py
.claude\worktrees\hunt-base\backend\eval\experiment.py
.claude\worktrees\hunt-base\backend\eval\export_wasb_segments.py
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py
.claude\worktrees\hunt-base\backend\eval\fusion_golden.py
```

.claude\worktrees\hunt-base\backend\eval\gemini_refine.py
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py
.claude\worktrees\hunt-base\backend\eval\gt_loader.py
.claude\worktrees\hunt-base\backend\eval\harness.py
.claude\worktrees\hunt-base\backend\eval\metrics.py
.claude\worktrees\hunt-base\backend\eval\per_user_eval.py
.claude\worktrees\hunt-base\backend\eval\per_user_gate.py
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py
.claude\worktrees\hunt-base\backend\eval\promotion.py
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py
.claude\worktrees\hunt-base\backend\eval\rally_seq_proto.py
.claude\worktrees\hunt-base\backend\eval\regression.py
.claude\worktrees\hunt-base\backend\eval\score_calibration.py
.claude\worktrees\hunt-base\backend\eval\segmentation_metrics.py
.claude\worktrees\hunt-base\backend\eval\served_gate_a.py
.claude\worktrees\hunt-base\backend\eval\serve_contrast.py
.claude\worktrees\hunt-base\backend\eval\splits.py
.claude\worktrees\hunt-base\backend\eval\tolerance_metrics.py
.claude\worktrees\hunt-base\backend\eval\training.py
.claude\worktrees\hunt-base\backend\eval\windowing.py

5. assistant (2026-06-18T01:33:22.653Z)

Let me focus on the core numeric/statistical files. I'll read the key metrics, tolerance, windowing, and stats files in parallel.

6. user (2026-06-18T01:33:23.468Z)

```
1       """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3       The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4       golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5       over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6       boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7       number (a single ? misleads; see the design doc).
8
9       Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10      asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11      rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13      Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14      else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15      eligibility graphs that rally intervals produce, so scores are environment-independent.
16      """
17      from __future__ import annotations
18
19      import statistics
20      from typing import Any, Dict, List, Optional, Tuple
21
22      Interval = Tuple[float, float]
23      #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
24      Match = Tuple[int, int, float, float]
25
26      #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
```

```
    #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
    TAU_START: float = 2.0
    TAU_END: float = 1.5
    SWEEP_TAUS: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)


    def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
        return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end


    def _greedy_match_1to1(preds: List[Interval], gts: List[Interval],
                           tau_start: float, tau_end: float) -> List[Match]:
        """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
        assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
        graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
        pairs: List[Tuple[float, float, int, int]] = []
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _eligible(p, g, tau_start, tau_end):
                    # sort key: combined error, then (i, j) for a fully deterministic
tiebreak
                    pairs.append((abs(p[0] - g[0]) + abs(p[1] - g[1]), float(i * 1e-6 + j *
1e-9), i, j))
        pairs.sort()
        used_p: set = set()
        used_g: set = set()
        matches: List[Match] = []
        for _, _, i, j in pairs:
            if i in used_p or j in used_g:
                continue
            used_p.add(i)
            used_g.add(j)
            matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
        matches.sort(key=lambda mm: mm[1])
        return matches


    def _hungarian_match_1to1(preds: List[Interval], gts: List[Interval],
                              tau_start: float, tau_end: float) -> Optional[List[Match]]:
        """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
        numpy/scipy are unavailable so the caller falls back to greedy."""
        try:
            import numpy as np
            from scipy.optimize import linear_sum_assignment
        except Exception:
            return None
        n, m = len(preds), len(gts)
        size = max(n, m)
        big = 1.0e6
        cost = np.full((size, size), big, dtype=float)
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _eligible(p, g, tau_start, tau_end):
                    cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
        rows, cols = linear_sum_assignment(cost)
        matches: List[Match] = []
        for i, j in zip(rows.tolist(), cols.tolist()):
            if i < n and j < m and cost[i, j] < big:  # drop forced ineligible assignments
                matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
        matches.sort(key=lambda mm: mm[1])
        return matches

```

```
87
88      def match_1to1(preds: List[Interval], gts: List[Interval],
89                     tau_start: float = TAU_START, tau_end: float = TAU_END
90                     ) -> Tuple[List[Match], float, float, float]:
91          """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
92          (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
93          present; deterministic greedy otherwise."""
94          n, m = len(preds), len(gts)
95          if n == 0 or m == 0:
96              return [], 0.0, 0.0, 0.0
97          matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
98          if matches is None:
99              matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
100         k = len(matches)
101         precision = k / n
102         recall = k / m
103         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else
0.0
104         return matches, precision, recall, f1
105
106
107     def _percentile(xs: List[float], p: float) -> float:
108         """Linear-interpolated percentile (pure stdlib)."""
109         if not xs:
110             return 0.0
111         s = sorted(xs)
112         k = (len(s) - 1) * p / 100.0
113         lo = int(k)
114         hi = min(lo + 1, len(s) - 1)
115         return s[lo] + (s[hi] - s[lo]) * (k - lo)
116
117
118     def _overlap(a: Interval, b: Interval) -> float:
119         return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
120
121
122     def boundary_error_report(preds: List[Interval], gts: List[Interval]) -> Dict[str,
Dict[str, float]]:
123         """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
124         end open" diagnostic that aims R4.
125
126         Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
127         positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
128         ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
129         this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
130         to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""
131         def dist(vals: List[float]) -> Dict[str, float]:
132             av = [abs(v) for v in vals]
133             return {
134                 "signed_median": statistics.median(vals) if vals else 0.0,
135                 "abs_median": statistics.median(av) if av else 0.0,
136                 "abs_p25": _percentile(av, 25.0),
137                 "abs_p75": _percentile(av, 75.0),
138                 "n": float(len(vals)),
139             }
140         ds: List[float] = []
141         de: List[float] = []
```

```
        for g in gts:
            best = max(preds, key=lambda p: _overlap(p, g), default=None)
            if best is not None and _overlap(best, g) > 0.0:
                ds.append(best[0] - g[0])
                de.append(best[1] - g[1])
        return {"dstart": dist(ds), "dend": dist(de)}


    def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
        """The over-segmentation GUARD (per-video no-regression in promotion):
``split_count`` (?2
        preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
bipartite
        overlap-graph in ``segmentation_metrics.merge_split_report``."""
        from backend.eval.segmentation_metrics import merge_split_report
        r = merge_split_report(preds, gts)
        return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}


    # ---------------------------------------------------------------------- #
    # R1 ? net over-segmentation promotion guard
    # ---------------------------------------------------------------------- #
    #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
one
    #: predicted window ? the user can't reach the hidden one ? recall loss at rally
granularity); a
    #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
strictly
    #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
R1 evidence;
    #: the asymmetry is the principled default, not a number tuned to force a verdict.)
    WEIGHT_MERGE: float = 2.0
    WEIGHT_SPLIT: float = 1.0


    def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
                       weight_merge: float = WEIGHT_MERGE, weight_split: float =
WEIGHT_SPLIT,
                       use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
        """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
video rule).

        ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
already
        emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
(default) ?
        ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
(else by
        position). Returns the promote-or-block verdict for promoting ``cand`` over
``base``.

        **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
``cand`` raises
        ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
cue (same
        windowing structure, an increase = a real regression) but mis-fires on a
*structural* change
        (mega-windows ? bounded): splitting a single mega-window necessarily lifts
``split_count`` from
        ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
rallies is
        ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
into pieces
        that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
rallies hidden).
```

```
        So the strict count rule blocks a config that strictly *reduces* the fraction of rallies lost to
        over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts mean
        ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ? RALLY_QUALITY_RESEARCH §6.)

        **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost per video and
        require BOTH:
          1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
          2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window that hides MORE
             of a video's rallies is the one over-merge regression that is never acceptable, the honest
             "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity), not the count.
        With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction of GT rallies
        affected, ? [0,1] ? comparable across a structural change); ``use_rates=False`` scores raw counts.

        **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to the original
        strict per-video count rule. Both verdicts are always reported, so the choice is auditable.
        """
        def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
            if base and cand and all("name" in b for b in base) and all("name" in c for c in cand):
                cmap = {c["name"]: c for c in cand}
                return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
            n = min(len(base), len(cand))
            return [(base[i], cand[i]) for i in range(n)]

        def _cost(row: Dict[str, Any]) -> float:
            if use_rates:
                return weight_merge * float(row["merge_rate"]) + weight_split * float(row["split_rate"])
            return weight_merge * float(row["merge_count"]) + weight_split * float(row["split_count"])

        pairs = _aligned()
        n = len(pairs)
        base_costs = [_cost(b) for b, _ in pairs]
        cand_costs = [_cost(c) for _, c in pairs]
        base_net = (sum(base_costs) / n) if n else 0.0
        cand_net = (sum(cand_costs) / n) if n else 0.0
        net_delta = cand_net - base_net

        # Per-video regressions, reported for transparency.
        new_merges: List[Dict[str, Any]] = []           # raw COUNT increases (what strict trips on)
        new_splits: List[Dict[str, Any]] = []           # raw COUNT increases (what strict trips on)
        merge_rate_regress: List[Dict[str, Any]] = []    # RATE increases (the real over-merge regression)
        for b, c in pairs:
            if float(c["merge_count"]) > float(b["merge_count"]) + eps:
                new_merges.append({"name": b.get("name", ""),
                                   "base": float(b["merge_count"]), "cand": float(c["merge_count"])})
            if float(c["split_count"]) > float(b["split_count"]) + eps:
                new_splits.append({"name": b.get("name", ""),
                                   "base": float(b["split_count"]), "cand":
```

```
float(c["split_count"])})
235             if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
236                 merge_rate_regress.append({"name": b.get("name", ""),
237                                            "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])})
238
239         strict_passes = (not new_merges) and (not new_splits)
240         net_cost_ok = net_delta <= eps
241         no_merge_rate_regress = not merge_rate_regress
242         passes = bool(net_cost_ok and no_merge_rate_regress)
243         return {
244             "passes": passes,
245             "strict_passes": strict_passes,
246             "net_cost_ok": net_cost_ok,
247             "no_merge_rate_regress": no_merge_rate_regress,
248             "base_net": base_net,
249             "cand_net": cand_net,
250             "net_delta": net_delta,
251             "n": n,
252             "use_rates": use_rates,
253             "weight_merge": weight_merge,
254             "weight_split": weight_split,
255             "new_merges": new_merges,
256             "new_splits": new_splits,
257             "merge_rate_regress": merge_rate_regress,
258         }
259
260
261     def tolerance_report(preds: List[Interval], gts: List[Interval],
262                          tau_start: float = TAU_START, tau_end: float = TAU_END) ->
Dict[str, object]:
263         """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
264         matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
265         return {
266             "tau_start": tau_start, "tau_end": tau_end,
267             "n_pred": len(preds), "n_gt": len(gts), "matched": len(matches),
268             "precision": precision, "recall": recall, "f1": f1,
269             "boundary_error": boundary_error_report(preds, gts),
270             "over_seg": over_seg_report(preds, gts),
271         }
272
273
274     def tolerance_sweep(preds: List[Interval], gts: List[Interval],
275                         taus: Tuple[float, ...] = SWEEP_TAUS) -> Dict[float, float]:
276         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
277         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
278
```

7. user (2026-06-18T01:33:24.425Z)

```
1       """Pure temporal interval-matching metrics for rally evaluation.
2
3       No I/O, no DB, no global state ? every function here is deterministic and unit
4       tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5       start < end. We score a list of *predicted* rally intervals against a list of
6       *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7       """
8
9       from dataclasses import dataclass, field
10      from typing import List, Tuple, Optional
11
12      Interval = Tuple[float, float]
13
```

```

    def interval_iou(a: Interval, b: Interval) -> float:
        """Temporal intersection-over-union of two intervals.

        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
        overlap / union, both measured as durations on the timeline.
        """
        a_start, a_end = a
        b_start, b_end = b
        if a_end <= a_start or b_end <= b_start:
            return 0.0
        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
        if inter <= 0.0:
            return 0.0
        union = (a_end - a_start) + (b_end - b_start) - inter
        return inter / union if union > 0 else 0.0


    @dataclass
    class Match:
        """One matched predicted?ground-truth pair."""
        pred_index: int
        gt_index: int
        iou: float


    @dataclass
    class MatchResult:
        """Outcome of matching predictions to ground truth at a given IoU threshold."""
        matches: List[Match] = field(default_factory=list)
        unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
        unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)


    def match_intervals(preds: List[Interval], gts: List[Interval],
                        iou_threshold: float = 0.5) -> MatchResult:
        """Greedy one-to-one matching of predictions to ground truth by descending IoU.

        Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
        are claimed highest-IoU first, and each prediction/GT can be used once. This
        keeps a single prediction from being credited for two ground-truth rallies
        (and vice versa).
        """
        candidates = []
        for pi, p in enumerate(preds):
            for gi, g in enumerate(gts):
                iou = interval_iou(p, g)
                # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
                # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
                if iou > 0.0 and iou >= iou_threshold:
                    candidates.append((iou, pi, gi))
        # Highest IoU first; ties broken by indices for determinism.
        candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

        used_pred, used_gt = set(), set()
        matches: List[Match] = []
        for iou, pi, gi in candidates:
            if pi in used_pred or gi in used_gt:
                continue
            used_pred.add(pi)
            used_gt.add(gi)
            matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

```

```
        unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
        unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
        return MatchResult(matches=matches,
                           unmatched_pred_indices=unmatched_pred,
                           unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
        true_positives: int
        false_positives: int
        false_negatives: int
        precision: float
        recall: float
        f1: float
        mean_iou: float            # over matched pairs (0.0 if none)
        mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
        mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds

        def as_dict(self) -> dict:
            return self.__dict__.copy()


    def score(preds: List[Interval], gts: List[Interval],
              iou_threshold: float = 0.5,
              match_result: Optional[MatchResult] = None) -> Score:
        """Compute precision/recall/F1, mean matched IoU, and boundary errors.

        Pass a precomputed `match_result` to avoid re-matching (e.g. when also
        extracting the failure lists); otherwise matching is done here.
        """
        mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)

        tp = len(mr.matches)
        fp = len(mr.unmatched_pred_indices)
        fn = len(mr.unmatched_gt_indices)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0

        if mr.matches:
            mean_iou = sum(m.iou for m in mr.matches) / tp
            mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
            mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
        else:
            mean_iou = mean_start_err = mean_end_err = 0.0

        return Score(
            iou_threshold=iou_threshold,
            num_pred=len(preds), num_gt=len(gts),
            true_positives=tp, false_positives=fp, false_negatives=fn,
            precision=precision, recall=recall, f1=f1,
            mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
        )


```

```
138     def pr_curve(preds: List[Interval], gts: List[Interval],
139                  thresholds: Optional[List[float]] = None) -> List[Score]:
140         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142         Lets you see strict-vs-lenient behaviour rather than a single operating point.
143         """
144         if thresholds is None:
145             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147
```

8. user (2026-06-18T01:33:24.872Z)

```
1       """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3       **The bug this closes.** The offline eval substrate (``distill_local.build_candidates``
?
4       ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5       on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
6       deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7       (``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02,
merge_gap=2.0,
8       min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured on the proposal set
is
9       measured on candidates **production never serves**, so a "win" need not transfer. This
is
10      exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
11      (``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).
12
13      **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14      SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15      (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16      build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17      *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18      operating point a given deployment truly serves.
19
20      The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21      windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22      ``per_user_gate``). This module is the **what windowing**; that one is the **promotion
gate**.
23
24      Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25      caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26      """
27      from __future__ import annotations
28
29      from dataclasses import dataclass
30      from typing import Any, Dict, List, Mapping, Tuple
31
32      from backend.config.models import IndexingConfig
33
34      Interval = Tuple[float, float]
35
36      __all__ = [
37          "WindowingPreset",
38          "SERVED",
```

```
39          "PROPOSAL",
40          "PRESETS",
41          "served_windowing_from_config",
42          "build_windows",
43          "windows_to_intervals",
44      ]
45
46
47      @dataclass(frozen=True)
48      class WindowingPreset:
49          """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51          ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52          "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53          ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54          knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55          """
56
57          name: str
58          velocity_thresh: float
59          merge_gap: float
60          min_window_duration: float
61          #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62          #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63          max_window_duration: float = 0.0
64          #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65          #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66          hard_cap: bool = False
67          #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68          #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69          inactivity_end: float = 0.0
70          #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71          inactivity_rally_thresh: float = 0.08
72          inactivity_min_density: float = 0.34
73          #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74          #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75          #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76          #: factor keeps it surgical (a normal long rally is left alone).
77          blob_fallback: bool = False
78          blob_factor: float = 4.0
79
80          def as_tuple(self) -> Tuple[float, float, float]:
81              """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82              ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83              bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84              return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
```

```
        def to_dict(self) -> Dict[str, Any]:
            return {
                "name": self.name,
                "velocity_thresh": self.velocity_thresh,
                "merge_gap": self.merge_gap,
                "min_window_duration": self.min_window_duration,
                "max_window_duration": self.max_window_duration,
                "hard_cap": self.hard_cap,
                "inactivity_end": self.inactivity_end,
                "inactivity_rally_thresh": self.inactivity_rally_thresh,
                "inactivity_min_density": self.inactivity_min_density,
                "blob_fallback": self.blob_fallback,
                "blob_factor": self.blob_factor,
            }


    def _served_preset_from_defaults() -> WindowingPreset:
        """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.

        ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
          - velocity:   ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
          - merge_gap:  ``idx_cfg.dead_time_merge_gap`` (default 2.0)
          - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)

        Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
        point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
        so the two can never silently diverge again.
        """
        idx = IndexingConfig()  # construct with defaults ? the served operating point
        return WindowingPreset(
            name="served",
            velocity_thresh=float(idx.wasb.velocity_threshold),  # == fusion/tracknet
default 0.02
            merge_gap=float(idx.dead_time_merge_gap),
            min_window_duration=float(idx.min_action_window_duration),
            max_window_duration=float(idx.max_window_duration),  # W1 bounded-windowing
(default 0=OFF)
            hard_cap=bool(idx.window_hard_cap),                   # W1 hard-cap fallback
(default OFF)
            inactivity_end=float(idx.window_inactivity_end),      # R4 rally-end trim
(default 0=OFF)
            inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
            inactivity_min_density=float(idx.inactivity_min_density),
            blob_fallback=bool(idx.window_blob_fallback),         # R5 blob-fallback
(default OFF)
            blob_factor=float(idx.window_blob_factor),
        )


    #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
    #: only promotable if it holds up HERE ? the operating point real users get.
    SERVED: WindowingPreset = _served_preset_from_defaults()

    #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
    #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
    #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
    #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
```

```
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(
158             name=f"served:{family}",
159             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
160             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166             inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167             blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168             blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169         )
170
171
172     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
174         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175         if obj is None:
176             return {}
177         if isinstance(obj, Mapping):
178             sub = obj.get(key, {})
179         elif hasattr(obj, "get"):
180             sub = obj.get(key, {})
181         else:
182             sub = getattr(obj, key, {})
183         return sub if isinstance(sub, Mapping) else {}
184
185
186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                       preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
189
```

```
190         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211
```

9. user (2026-06-18T01:33:25.755Z)

```
1       """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3       The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4       single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5       and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6       ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7       turns those into an honest summary:
8
9       - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10        interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11        many *videos* it wins on (the distribution-free test the repo already leans on:
12        "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13      - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14        producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15        match, never a held-out window from the same video).
16
17      See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18      deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19      ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20      """
21      from __future__ import annotations
22
23      import math
24      import random
25      import statistics
26      from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28      Interval = Tuple[float, float]
29      FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
```

```
    def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
                     n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
        """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.

        ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
        resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
        """
        vals = [float(v) for v in values]
        n = len(vals)
        if n == 0:
            return (0.0, 0.0)
        if n == 1:
            return (vals[0], vals[0])
        rng = random.Random(seed)
        means: List[float] = []
        for _ in range(max(1, n_boot)):
            means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
        means.sort()
        alpha = (1.0 - confidence) / 2.0
        last = len(means) - 1
        lo = means[max(0, int(math.floor(alpha * last)))]
        hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
        return (lo, hi)


    def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
                     n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
        """``{mean, std, n, ci_low, ci_high, confidence}`` ? report this, never a bare mean
(C1)."""
        vals = [float(v) for v in values]
        n = len(vals)
        mean = statistics.mean(vals) if vals else 0.0
        std = statistics.pstdev(vals) if n > 1 else 0.0
        lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
        return {"mean": mean, "std": std, "n": float(n),
                "ci_low": lo, "ci_high": hi, "confidence": confidence}


    def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
        """Two-sided exact sign test over per-fold deltas (B ? A).

        ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
        excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
        distribution-free "did B win on enough *videos*?" check.
        """
        wins = sum(1 for d in deltas if d > eps)
        losses = sum(1 for d in deltas if d < -eps)
        ties = len(deltas) - wins - losses
        n_eff = wins + losses
        if n_eff == 0:
            p = 1.0
        else:
            k = min(wins, losses)
            tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
            p = min(1.0, 2.0 * tail)
        return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
                "n_effective": float(n_eff), "p_value": p}


    def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
                        confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
                        eps: float = 0.0) -> Dict[str, Any]:
        """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
```

```
94
95          ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
96          fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
sign
97          test agrees ? not when a single mean ticked up.
98          """
99          n = min(len(a_values), len(b_values))
100         deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
101         summary = mean_with_ci(deltas, confidence, n_boot, seed)
102         return {
103             "mean_delta": summary["mean"],
104             "ci_low": summary["ci_low"],
105             "ci_high": summary["ci_high"],
106             "n": float(n),
107             "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
108             "sign_test": paired_sign_test(deltas, eps),
109         }
110
111
112     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.
114
115         The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116         first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117         """
118         seen: List[Any] = []
119         for g in groups:
120             if g not in seen:
121                 seen.append(g)
122         n = len(groups)
123         folds: List[Tuple[List[int], List[int]]] = []
124         for g in seen:
125             test = [i for i in range(n) if groups[i] == g]
126             train = [i for i in range(n) if groups[i] != g]
127             if test and train:
128                 folds.append((train, test))
129         return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
137         scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
```

150

10. assistant (2026-06-18T01:33:28.436Z)

Let me look at the windowing implementation in the detector runner, and the segmentation_metrics and other statistical files.

11. user (2026-06-18T01:33:29.398Z)

```
1       """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3       Temporal IoU (``metrics.score``) is nearly blind to *fragmentation*: a single rally
4       chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5       still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6       literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7       thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8       sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9       tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10      structure invisibly. See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C4.
11
12      This is **additive**: new functions over the existing ``metrics`` matcher; nothing here
13      changes ``score``/``compare``/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
?
14      so it runs in the GPU-free / numpy-free lanes too.
15
16      Conventions match ``metrics``: an ``Interval`` is a ``(start, end)`` tuple of seconds.
17      A ``ScoredInterval`` is ``(interval, confidence)`` ? the confidence is whatever the
scorer
18      emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.
19
20      Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
the
21      ordered sequence of segment *labels* and is degenerate for a single action class ?
rally/
22      background segments strictly alternate, so the edit distance collapses to (twice) the
23      segment-count difference, adding nothing over ``segment_count_ratio``. So we
intentionally
24      omit it and instead make over-segmentation explicit with ``merge_split_report`` (a
bipartite
25      overlap-graph breakdown into merge/split/matched/missed/spurious ? see below), alongside
26      ``f1_at_overlaps`` (F1@k) + ``mean_average_precision`` (mAP@tIoU). Those, not a single-
class
27      edit score, are the over-segmentation-sensitive metrics for this binary rally setting.
28      """
29      from __future__ import annotations
30
31      from typing import Dict, List, Sequence, Tuple
32
33      from backend.eval.metrics import Interval, interval_iou, score
34
35      #: A predicted interval plus the confidence used to rank it for average precision.
36      ScoredInterval = Tuple[Interval, float]
37
38      #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
39      DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
40      #: Default tIoU sweep for mean average precision ? the action-detection convention.
41      DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)
42
43
44      def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
45                         overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
46          """Segmental F1 at each overlap threshold (``{overlap: f1}``) ? the TAS F1@k.
47
48          Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
49          for 1 rally) scores extra false positives and a merged prediction misses a rally ?
```

```
        both drag F1 down where temporal-IoU alone would not flinch.
        """
        return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}


    def average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
                          iou_threshold: float = 0.5) -> float:
        """Average precision at a single tIoU (all-points / VOC-2010 interpolation).

        Predictions are ranked by descending confidence; each is a true positive iff it
        claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
        available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
        there are no ground-truth rallies or no predictions reach the threshold.
        """
        n_gt = len(gts)
        if n_gt == 0 or not scored_preds:
            return 0.0

        order = sorted(range(len(scored_preds)),
                       key=lambda i: (-scored_preds[i][1], i))   # conf desc, stable by
index
        matched_gt = set()
        precisions: List[float] = []
        recalls: List[float] = []
        cum_tp = 0
        cum_fp = 0
        for i in order:
            interval, _conf = scored_preds[i]
            best_iou, best_gi = 0.0, -1
            for gi, g in enumerate(gts):
                if gi in matched_gt:
                    continue
                iou = interval_iou(interval, g)
                if iou >= iou_threshold and iou > best_iou:
                    best_iou, best_gi = iou, gi
            if best_gi >= 0:
                matched_gt.add(best_gi)
                cum_tp += 1
            else:
                cum_fp += 1
            precisions.append(cum_tp / (cum_tp + cum_fp))
            recalls.append(cum_tp / n_gt)

        # Monotone precision envelope (take the max precision to the right), then integrate
        # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
        for i in range(len(precisions) - 2, -1, -1):
            precisions[i] = max(precisions[i], precisions[i + 1])
        ap = 0.0
        prev_recall = 0.0
        for p, r in zip(precisions, recalls):
            if r > prev_recall:
                ap += p * (r - prev_recall)
                prev_recall = r
        return ap


    def mean_average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
                               iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
        """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
        tious = list(iou_thresholds)
        if not tious:
            return 0.0
        return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)


```

```
    def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
        """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.

        ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
        merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
``gts``
        is empty (use F1@k / mAP for the graded picture).
        """
        if not gts:
            return 0.0
        return len(preds) / len(gts)


    def _overlaps(a: Interval, b: Interval) -> bool:
        """True iff two intervals share any positive temporal extent."""
        return min(a[1], b[1]) - max(a[0], b[0]) > 0.0


    def merge_split_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, float]:
        """Explicit over-/under-segmentation breakdown via the bipartite overlap graph.

        Where ``segment_count_ratio`` only compares *counts* and ``f1_at_overlaps`` blends
merge
        and split into one F1, this names the two failure modes directly ? the diagnostic
that
        surfaced our local-segmenter over-MERGE (one giant window swallowing many rallies).
Build
        the bipartite graph (edge = any positive temporal overlap between a pred and a gt),
take its
        connected components, and classify each by ``(#preds, #gts)``:

          - ``(1,1)`` matched   ``(1,0)`` spurious (pred, no gt)   ``(0,1)`` missed (gt, no
pred)
          - ``(1,>1)`` **merge** ? one predicted window covers ?2 rallies (over-MERGE; our
failure)
          - ``(>1,1)`` **split** ? one rally cut across ?2 predictions (over-segmentation)
          - ``(>1,>1)`` complex (tangled many-to-many)

        Returns counts plus ``merge_rate`` (fraction of GT rallies swallowed inside a merge
group)
        and ``split_rate`` (fraction of GT rallies cut by ?2 predictions) ? the two headline
        over-segmentation rates. Pure, deterministic, stdlib-only.
        """
        n, m = len(preds), len(gts)
        p_adj: List[List[int]] = [[] for _ in range(n)]
        g_adj: List[List[int]] = [[] for _ in range(m)]
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _overlaps(p, g):
                    p_adj[i].append(j)
                    g_adj[j].append(i)

        seen_p: set = set()
        seen_g: set = set()
        comps: List[Tuple[set, set]] = []
        for start in range(n):
            if start in seen_p:
                continue
            ps: set = set()
            gs: set = set()
            stack: List[Tuple[str, int]] = [("p", start)]
            while stack:
                side, k = stack.pop()
                if side == "p":
                    if k in ps:
```

```
                        continue
                    ps.add(k)
                    seen_p.add(k)
                    stack.extend(("g", j) for j in p_adj[k] if j not in gs)
                else:
                    if k in gs:
                        continue
                    gs.add(k)
                    seen_g.add(k)
                    stack.extend(("p", i) for i in g_adj[k] if i not in ps)
            comps.append((ps, gs))
        comps.extend((set(), {j}) for j in range(m) if j not in seen_g)  # gts with no
overlapping pred

        counts = {"matched": 0, "merges": 0, "splits": 0, "spurious": 0, "missed": 0,
"complex": 0}
        gts_merged = gts_split = 0
        for ps, gs in comps:
            a, b = len(ps), len(gs)
            if (a, b) == (1, 1):
                counts["matched"] += 1
            elif (a, b) == (1, 0):
                counts["spurious"] += 1
            elif (a, b) == (0, 1):
                counts["missed"] += 1
            elif a == 1 and b > 1:
                counts["merges"] += 1
                gts_merged += b
            elif b == 1 and a > 1:
                counts["splits"] += 1
                gts_split += 1
            else:
                counts["complex"] += 1
        out: Dict[str, float] = {k: float(v) for k, v in counts.items()}
        out["merge_rate"] = (gts_merged / m) if m else 0.0
        out["split_rate"] = (gts_split / m) if m else 0.0
        return out


    def segmentation_report(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
                            overlaps: Sequence[float] = DEFAULT_OVERLAPS,
                            iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
object]:
        """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio + merge/split
breakdown.

        ``scored_preds`` carry confidences for AP; F1@k + the merge/split breakdown use the
bare
        intervals. Returns a plain dict (JSON-friendly, string keys) so it can ride next to
a
        ``compare`` result or land in the experiment leaderboard.
        """
        preds = [iv for iv, _conf in scored_preds]
        return {
            "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
overlaps).items()},
            "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
iou_thresholds},
            "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
            "segment_count_ratio": segment_count_ratio(preds, gts),
            "merge_split": merge_split_report(preds, gts),
            "num_pred": len(preds),
            "num_gt": len(gts),
        }

```

12. user (2026-06-18T01:33:30.623Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                       # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)
44
45          @classmethod
46          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47              tn = (idx_cfg or {}).get("tracknet", {}) or {}
48              return cls(
49                  distro=tn.get("wsl_distro", "Ubuntu"),
50                  repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51                  conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52                  conda_env=tn.get("conda_env", "TrackNetV4"),
53                  weights_path=tn.get("weights_path", ""),
54                  queue_length=int(tn.get("queue_length", 5)),
55                  stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
```

```
                wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(tn.get("timeout_sec", 1800)),
                keep_frames=bool(tn.get("keep_frames", False)),
                min_free_gb=float(tn.get("min_free_gb", 0.0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(WslCommandMixin, DetectorRunner):
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference
```

```
119
120         def run_predict(self, video_win_path: str, output_win_dir: str,
121                         video_id: Optional[str] = None) -> Optional[str]:
122             """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
123
124             ``output_win_dir`` is a Windows directory (under the repo's output/) that
125             WSL writes into via its /mnt view, so the Windows process can read the CSV
126             back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
127             therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
128             a filename cannot collide on the output CSV.
129             """
130             if not self.cfg.weights_path:
131                 logger.error(
132                     "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
133                     "in config.json to the WSL path of your trained TrackNetV4 weights."
134                 )
135                 return None
136
137             # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138             # relative paths through unchanged, so WSL would resolve them against the
139             # predict.py cwd instead of the repo (same bug class as wasb_runner).
140             output_win_dir = os.path.abspath(output_win_dir)
141             os.makedirs(output_win_dir, exist_ok=True)
142             # Normalize separators so basename/splitext work when tests (or collector)
143             # pass Windows-style paths on a Linux host.
144             video_win_path = str(video_win_path).replace("\\", "/")
145             base = os.path.basename(video_win_path)
146             stem, ext = os.path.splitext(base)
147
148             # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149             if ext.lower() not in (".mp4", ".avi"):
150                 logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151                 return None
152
153             wsl_out = to_wsl_mnt_path(output_win_dir)
154             # Namespace by video_id so two same-filename videos don't collide on the CSV.
155             # predict.py derives its output name from the (staged) video stem, so staging
156             # under the namespaced name propagates into "<key>_predict.csv".
157             key = f"{stem}__{video_id[:12]}" if video_id else stem
158             expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160             # Resolve the video path WSL will read.
161             if self.cfg.stage_in_wsl:
162                 staged = self._stage_video(video_win_path, f"{key}{ext}")
163                 if staged is None:
164                     return None
165                 wsl_video = staged
166             else:
167                 # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
168                 # We cannot rename its output, so fall back to the un-namespaced name.
169                 wsl_video = to_wsl_mnt_path(video_win_path)
170                 expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
171
172             if self.cfg.min_free_gb > 0:
173                 free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
174                 if free is not None and free < self.cfg.min_free_gb:
175                     logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                                    "consider running tools/cleanup_caches.py.",
177                                    free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179             cmd = (
180                 f"conda activate {self.cfg.conda_env} && "
181                 f"cd {self.cfg.repo_dir}/src && "
```

```
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--output_dir {wsl_tilde_quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------ CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
```

```
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
            max_window_duration: float = 0.0,
            min_split_gap: float = 0.5,
            window_hard_cap: bool = False,
            window_inactivity_end: float = 0.0,
            inactivity_min_density: float = 0.34,
            inactivity_rally_thresh: float = 0.08,
            window_blob_fallback: bool = False,
            window_blob_factor: float = 4.0,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.

            ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
            the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
            and a single window swallows several rallies + the dead time between them. When
set, any
            window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
            longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
            that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
            ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
            drop, and it is config-gated default-OFF until measured on the golden set.

            ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
            over-long window has NO internal inactivity gap to split on (a continuously-
active
            trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
            blob where the gap-splitter alone left a single 174 s window), it is recursively
```

```
296             hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
297             (never merges), so recall cannot drop; gated default-OFF until measured.
298
299             ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
300             ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
301             over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
302             force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` ×
303             ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
304             their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
305             alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
306             each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
307             rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
308             can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
309             """
310             if fps <= 0:
311                 fps = 30.0
312             if frame_width <= 0:
313                 frame_width = 1280.0
314
315             # Compute per-frame normalised velocity from the last *visible* point.
316             active = []  # (frame_idx, velocity)
317             prev: Optional[TrajectoryPoint] = None
318             for p in points:
319                 if not p.visible:
320                     prev = None  # break the trajectory; absent shuttle = not in flight
321                     continue
322                 if prev is not None:
323                     dframes = p.frame - prev.frame
324                     if dframes > 0:
325                         dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                         velocity = (dist / frame_width) * (fps / dframes)
327                         if velocity > velocity_thresh:
328                             active.append((p.frame, velocity))
329                 prev = p
330
331             if not active:
332                 return []
333
334             max_gap_frames = int(merge_gap * fps)
335
336             def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337                 vels = [v for _, v in group]
338                 return {
339                     "start": group[0][0] / fps + chunk_start,
340                     "end": group[-1][0] / fps + chunk_start,
341                     "active_frame_count": len(group),
342                     "peak_velocity": max(vels),
343                     "mean_velocity": sum(vels) / len(vels),
344                     "track_id_count": 1,  # single shuttle; kept for window-shape parity
345                     "chunk_index": chunk_index,
346                 }
347
348             groups: List[List[Tuple[int, float]]] = []
349             group = [active[0]]
```

```
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    groups.append(group)
                    group = [af]
            groups.append(group)

            if max_window_duration > 0:
                groups = TrackNetRunner._split_overlong_groups(
                    groups, int(max_window_duration * fps), int(min_split_gap * fps),
                    hard_cap=window_hard_cap)

            # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
            # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
            # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
            # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
            # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
            if window_inactivity_end > 0:
                tw = int(window_inactivity_end * fps)
                groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
                                                             inactivity_rally_thresh) for g
in groups]

            # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
            # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
            # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
            # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
            forced_flags = [False] * len(groups)
            if window_blob_fallback and max_window_duration > 0:
                groups, forced_flags = TrackNetRunner._blob_fallback_chop(
                    groups, int(max_window_duration * fps), window_blob_factor)

            windows = []
            for g, forced in zip(groups, forced_flags):
                w = _finalize(g)
                if forced:
                    w["forced_cut"] = True
                windows.append(w)
            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

        @staticmethod
        def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
                                min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
            """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
            (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
            over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
            (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
            ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
            LAST such frame ? everything after is sustained slow drift (rally over). Only
```

```
cuts (recall
            can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
            exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
            is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
            if trail_frames <= 0 or len(group) < 2:
                return group
            frames = [f for f, _ in group]
            fast = [1 if v > rally_thresh else 0 for _, v in group]
            lo = 0
            fast_sum = 0
            last_dense = -1
            for i in range(len(frames)):
                fast_sum += fast[i]
                while frames[lo] <= frames[i] - trail_frames:
                    fast_sum -= fast[lo]
                    lo += 1
                win_count = i - lo + 1
                if win_count > 0 and fast_sum / win_count >= min_density:
                    last_dense = i
            return group[: last_dense + 1] if last_dense >= 0 else group

        @staticmethod
        def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
                                   min_split_frames: int,
                                   hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
            """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
            internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
            boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
            invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
            rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
            temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
            bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
            if max_win_frames <= 0:
                return groups
            out: List[List[Tuple[int, float]]] = []
            work = list(groups)
            while work:
                g = work.pop()
                if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
                    out.append(g)
                    continue
                best_i, best_gap = -1, -1
                for i in range(1, len(g)):
                    gap = g[i][0] - g[i - 1][0]
                    if gap > best_gap:
                        best_gap, best_i = gap, i
                if best_i < 0 or best_gap < min_split_frames:
                    # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
                    # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
                    if hard_cap:
                        mid = (g[0][0] + g[-1][0]) / 2.0
                        split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
                        work.append(g[:split_i])
```

```
                        work.append(g[split_i:])
                    else:
                        out.append(g)
                    continue
                work.append(g[:best_i])
                work.append(g[best_i:])
            out.sort(key=lambda grp: grp[0][0])
            return out

        @staticmethod
        def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
                                blob_factor: float = 4.0
                                ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
            """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
            inactivity trim have each had their chance, a group still longer than
``blob_factor`` ×
            ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
            signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
            11× a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2× the cap)
            is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
            midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
            degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
            rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
            can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
            that piece came from a forced chop. Iterative worklist to bound stack depth."""
            if max_win_frames <= 0:
                return groups, [False] * len(groups)
            blob_frames = max_win_frames * max(1.0, blob_factor)
            out: List[List[Tuple[int, float]]] = []
            flags: List[bool] = []
            forced = 0
            for g in groups:
                if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
                    out.append(g)
                    flags.append(False)
                    continue
                pieces: List[List[Tuple[int, float]]] = []
                work = [g]
                while work:
                    h = work.pop()
                    if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
                        pieces.append(h)
                        continue
                    mid = (h[0][0] + h[-1][0]) / 2.0
                    split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid))  #
noqa: B023
                    work.append(h[:split_i])
                    work.append(h[split_i:])
                pieces.sort(key=lambda grp: grp[0][0])
                out.extend(pieces)
                flags.extend([True] * len(pieces))
                forced += len(pieces) - 1
            if forced:
                logger.warning(
                    "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
                    "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
```

```
two-sided "
505                     "motion), not a bigger cap", forced)
506             return out, flags
507
```

13. assistant (2026-06-18T01:33:33.912Z)

Let me examine the splits, calibration, classifier, and other stats-heavy files for boundary conditions.

14. user (2026-06-18T01:33:34.878Z)

```
1       """Train/eval split guard (pure-logic core, in-container).
2
3       Spec from BACKLOG.md "Train/eval separation guardrails" / DEFERRED #13
4       and plan M0:
5       - Split unit = match (never clip / file MD5).
6       - load_split(manifest) -> (train_ids, eval_ids) keyed by match id.
7       - assert_disjoint(train_ids, eval_ids) raises on any overlap.
8       - This would have caught train/eval leakage in training/eval paths.
9       - Cross-repo collector manifest `split` field wiring + wiring into
10        distill_local / run_regression deferred to later PR.
11
12      Usage (once wired):
13          train, eval_ = load_split(manifest)
14          assert_disjoint(train, eval_)
15      """
16      from __future__ import annotations
17
18      import json
19      import os
20      from typing import Any, Dict, Iterable, List, Set, Tuple, Union, cast
21
22
23      def load_split(manifest: Union[str, Iterable[Dict[str, Any]], os.PathLike[str]]) ->
Tuple[Set[str], Set[str]]:
24          """Load (train_ids, eval_ids) sets keyed by match id.
25
26          manifest: path to JSON (str/bytes/PathLike) or iterable of dicts. Each entry should
have 'match_id'
27          (or 'id' fallback; must be non-empty str for real match key; name/video_id
deliberately not used as they are weak/MD5).
28
29          Raises ValueError on:
30          - non-list manifest after load
31          - no real match-level key (empty/falsy after explicit check)
32          - split label not exactly "train" or "eval"
33
34          The collector 'split' field and cross-repo wiring is deferred; this supports
35          synthetic or pre-split manifests for the pure-logic guard and tests.
36          """
37          if isinstance(manifest, (str, bytes, os.PathLike)):
38              with open(manifest, "r", encoding="utf-8-sig") as f:
39                  raw = json.load(f)
40                  if not isinstance(raw, list):
41                      raise ValueError(f"manifest must be a list of entries, got {type(raw)}")
42                  data = cast(List[Dict[str, Any]], raw)
43          else:
44              if isinstance(manifest, dict):
45                  raise ValueError(f"manifest must be a list of entries, got
{type(manifest)}")
46              data = list(manifest)
47              if not isinstance(data, list):
48                  raise ValueError(f"manifest must be a list of entries, got {type(data)}")
49
```

```
50          train_ids: Set[str] = set()
51          eval_ids: Set[str] = set()
52
53          for entry in data:
54              if not isinstance(entry, dict):
55                  continue
56              # Explicit not-None + non-empty for real match key; no silent fallback to
name/video_id (weak/MD5)
57              mid = entry.get("match_id")
58              if mid is None or str(mid).strip() == "":
59                  mid = entry.get("id")
60              if mid is None or str(mid).strip() == "":
61                  raise ValueError(f"no real match-level key (match_id or id) in entry:
{entry}")
62              mid = str(mid).strip()
63              sp = str(entry.get("split", "")).strip().lower()
64              if sp not in ("train", "eval"):
65                  raise ValueError(f"split label must be exactly 'train' or 'eval', got {sp!r}
in entry: {entry}")
66              if sp == "eval":
67                  eval_ids.add(mid)
68              else:
69                  train_ids.add(mid)
70
71          return train_ids, eval_ids
72
73
74      def assert_disjoint(train_ids: Iterable[str], eval_ids: Iterable[str]) -> None:
75          """Raise if any match-id overlap between train and eval sets.
76
77          This would have caught train/eval leakage (e.g. a video used for both
78          training a segmenter and for regression/eval baselines).
79          """
80          t = set(str(x) for x in train_ids)
81          e = set(str(x) for x in eval_ids)
82          overlap = t & e
83          if overlap:
84              raise ValueError(
85                  f"Train/eval leakage detected on match ids: {sorted(overlap)}. "
86                  "This would have caught train/eval leakage."
87              )
88
```

15. user (2026-06-18T01:33:35.501Z)

```
1       """Quantify rally-segmentation improvement ? and guard against overfitting.
2
3       This is the measurement layer for per-video / per-corpus calibration. It answers
4       two questions honestly:
5
6       1. **"Did precision improve, by how much?"** ? `improvement_report` scores a
7          baseline config vs a tuned config on the same ground truth and reports the
8          before/after metrics and the % delta.
9
10      2. **"Is that gain real or overfit?"** ? calibration must be measured on data the
11         tuning never saw, otherwise you're just memorising. Two protocols:
12         - `cross_validate` ? k-fold over one video's rallies: pick the config on the
13           train folds, score on the held-out fold. The **generalization_gap**
14           (train ? test) is the overfit alarm.
15         - `leave_one_video_out` ? the protocol that matters as more videos arrive:
16           pick the config on all *other* videos, score on the held-out video. Reports
17           per-video held-out scores + mean±std. This is the number to trust.
18
19      Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
20      List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
```

```
    harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
    """
    from __future__ import annotations

    import statistics as _st
    from typing import Callable, Dict, List, Sequence, Tuple, Any

    from backend.eval.metrics import score, Score, Interval

    Config = Any                                # opaque to this module (e.g. a thresholds
dict)
    PredictFn = Callable[[Config], List[Interval]]


    def pct_improvement(before: float, after: float) -> float:
        """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
        if before <= 0:
            return float("inf") if after > 0 else 0.0
        return (after - before) / before * 100.0


    def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
Score:
        """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
        return score(preds, gts, iou_threshold)


    def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
        """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
        if n <= 0:
            return []
        k = max(2, min(k, n))                   # need >=2 folds; cap at n
        folds = [list(range(i, n, k)) for i in range(k)]   # round-robin ? balanced,
deterministic
        splits = []
        for f in range(k):
            test = folds[f]
            train = [i for i in range(n) if i not in set(test)]
            if test and train:
                splits.append((train, test))
        return splits


    def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
                    metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
        """Pick the config whose predictions maximise `metric` against `gts`."""
        best_cfg, best_val = None, -1.0
        for cfg in configs:
            val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
            if val > best_val:
                best_cfg, best_val = cfg, val
        return best_cfg, best_val


    def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],
                       k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
        """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).

        Per fold: choose the best config on the train rallies, score it on the held-out
        rallies. A large `generalization_gap` (train >> test) means the thresholds are
        overfitting that video's quirks.

        Default metric is **recall** on purpose: predictions are global (whole-video
```

```
        windows), so scoring them against a *fold subset* of GT would unfairly count the
        other rallies' windows as false positives ? precision/F1 are not well-defined
        per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")
        is the honest within-video generalization signal. For precision/F1, use
        `leave_one_video_out` (each video scored whole) or `improvement_report` on a
        held-out video.
        """
        splits = kfold_indices(len(gts), k)
        if not splits:
            return {"error": "not enough rallies for cross-validation", "n": len(gts)}
        train_scores, test_scores, picked = [], [], []
        for train_idx, test_idx in splits:
            gts_tr = [gts[i] for i in train_idx]
            gts_te = [gts[i] for i in test_idx]
            cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
            te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
            train_scores.append(tr)
            test_scores.append(te)
            picked.append(cfg)
        mean_test = _st.mean(test_scores)
        mean_train = _st.mean(train_scores)
        return {
            "metric": metric,
            "folds": len(splits),
            "mean_test": mean_test,
            "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
            "mean_train": mean_train,
            "generalization_gap": mean_train - mean_test,   # >~0.1 = likely overfitting
            "picked_configs": picked,
        }


    def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
                            metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
Any]:
        """Leave-one-video-out CV ? the honest metric as the corpus grows.

        `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
        For each held-out video: pick the config that's best *pooled across the other
        videos*, then score it on the held-out one. The mean held-out score is the
        "precision on unseen videos" number; per-video spread shows stability.
        """
        if len(videos) < 2:
            return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
        per_video = []
        for i, held in enumerate(videos):
            others = [v for j, v in enumerate(videos) if j != i]

            def pooled_score(cfg: Config, others: List[dict] = others) -> float:
                vals = [getattr(score(v["predict_fn"](cfg), v["gts"], iou_threshold),
metric)
                        for v in others]
                return _st.mean(vals)

            best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
t: t[1])
            held_score = getattr(score(held["predict_fn"](best_cfg), held["gts"],
iou_threshold), metric)
            per_video.append({"video": held.get("name", f"video_{i}"),
                              "held_out": held_score, "train_pooled": best_train, "config":
best_cfg})
        held_scores = [p["held_out"] for p in per_video]
        train_scores = [p["train_pooled"] for p in per_video]
        return {
            "metric": metric,
```

```
141             "mean_held_out": _st.mean(held_scores),
142             "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
143             "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),
144             "per_video": per_video,
145         }
146
147
148     def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
149                            gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
150         """The "+X% precision" headline: baseline vs tuned config on the same GT.
151
152         NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
153         or read the cross-validated `mean_test` from `cross_validate` instead.
154         """
155         b = score(predict_fn(baseline_config), gts, iou_threshold)
156         t = score(predict_fn(tuned_config), gts, iou_threshold)
157         return {
158             "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
159             "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
t.mean_iou},
160             "precision_delta_pts": t.precision - b.precision,
161             "precision_delta_pct": pct_improvement(b.precision, t.precision),
162             "f1_delta_pct": pct_improvement(b.f1, t.f1),
163         }
164
```

16. assistant (2026-06-18T01:33:42.631Z)

Let me examine the R4 inactivity trim more closely ? there's a subtle sliding-window bug candidate there. Let me also look at the classifier and tolerance_metrics percentile handling, plus eval_partial_labels.

Looking at `_trim_inactive_tail` (lines 391-417): the sliding window condition is `while frames[lo] <= frames[i] - trail_frames`. This means the window contains frames `f` where `f > frames[i] - trail_frames`. Let me check whether `frames[lo]` could advance past `i`. When `lo == i`, `frames[i] <= frames[i] - trail_frames` is `0 <= -trail_frames` which is false (trail_frames > 0), so `lo` never passes `i`. Good. `win_count = i - lo + 1 >= 1`. That's fine.

Let me look at more files ? the classifier, eval_partial_labels, and per_user_eval which often have statistical edge cases.

17. user (2026-06-18T01:33:43.994Z)

```
1       """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3       The offline learned segmenter (ADR-004) must run locally with no external API and
4       no heavy ML stack ? the project already depends on numpy, and sklearn is not
5       installed. This implements just enough: standardized features, balanced-class
6       gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7       model is a small, human-readable, version-controllable file.
8
9       It deliberately stays simple: the rally-window features are few and roughly
10      linearly separable (high player motion + sustained activity => real rally), so a
11      well-regularized linear model is a sensible, inspectable baseline. Swap in a
12      richer model later behind the same fit/predict_proba/save/load interface.
13      """
14
15      import json
16      from typing import List, Optional
17
18      import numpy as np
19
```

```

    class LogisticRegression:
        def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
            self.lr = lr
            self.epochs = epochs
            self.l2 = l2
            self.w: Optional[np.ndarray] = None
            self.b: float = 0.0
            self.mean: Optional[np.ndarray] = None
            self.std: Optional[np.ndarray] = None
            self.feature_names: Optional[List[str]] = None

        @staticmethod
        def _sigmoid(z: np.ndarray) -> np.ndarray:
            # Clip for numerical stability (avoid overflow in exp).
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

        def _standardize(self, X: np.ndarray) -> np.ndarray:
            return (X - self.mean) / self.std

        def fit(self, X, y, feature_names: Optional[List[str]] = None,
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

            self.w = np.zeros(d)
            self.b = 0.0
            for _ in range(self.epochs):
                p = self._sigmoid(Xs @ self.w + self.b)
                err = (p - y) * sw
                grad_w = Xs.T @ err / n + self.l2 * self.w
                grad_b = float(err.mean())
                self.w -= self.lr * grad_w
                self.b -= self.lr * grad_b
            return self

        def predict_proba(self, X) -> np.ndarray:
            if self.w is None:
                raise RuntimeError("model is not fitted")
            Xs = self._standardize(np.asarray(X, dtype=float))
            return self._sigmoid(Xs @ self.w + self.b)

        def predict(self, X, threshold: float = 0.5) -> np.ndarray:
            return (self.predict_proba(X) >= threshold).astype(int)

        # --- persistence (small JSON, no pickle/joblib) ---------------------------
```

```
85
86          def to_dict(self) -> dict:
87              return {
88                  "model": "logreg",
89                  "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90                  "w": self.w.tolist() if self.w is not None else None,
91                  "b": self.b,
92                  "mean": self.mean.tolist() if self.mean is not None else None,
93                  "std": self.std.tolist() if self.std is not None else None,
94                  "feature_names": self.feature_names,
95              }
96
97          @classmethod
98          def from_dict(cls, d: dict) -> "LogisticRegression":
99              m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100             m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101             m.b = float(d.get("b", 0.0))
102             m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103             m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104             m.feature_names = d.get("feature_names")
105             return m
106
107         def save(self, path: str) -> None:
108             with open(path, "w") as f:
109                 json.dump(self.to_dict(), f, indent=2)
110
111         @classmethod
112         def load(cls, path: str) -> "LogisticRegression":
113             with open(path) as f:
114                 return cls.from_dict(json.load(f))
115
```

18. user (2026-06-18T01:33:44.262Z)

```
1       """Evaluate the rally pipeline against PARTIAL human golden labels.
2
3       The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4       ending_reason,sport). While a video is only partially tagged, the labels cover a
5       PREFIX of the video ? every rally up to some point is marked, and everything after is
6       simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7       the detector's *correct* rallies in the un-labeled tail would be counted as false
8       positives and crush precision.
9
10      This harness fixes that by restricting the evaluation to the LABELED WINDOW
11      [window_start, window_end] (window_end defaults to the last labeled rally's end) and
12      dropping predicted segments that fall outside it. Within that window the human labels
13      ARE complete, so precision/recall/F1 are honest.
14
15      It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16        - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17        - pipeline.detectors.rally_gate.apply_gate   (net-crossing "gate A")
18        - eval.metrics.match_intervals / score
19        - export_wasb_segments.build_comparison / write_comparison_csv  (per-rally detail CSV)
20
21      Run:
22          python -m backend.eval.eval_partial_labels \
23              --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
24              --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
Avi.rallies.csv" \
25              --fps 59.94 --frame-width 1920
26      """
27      from __future__ import annotations
28
29      import argparse
```

```
    import os.path as osp
    from typing import List, Optional, Tuple

    from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
    from backend.eval.metrics import score
    from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.pipeline.detectors import rally_gate

    Interval = Tuple[float, float]
    Config = Tuple[float, float, float]

    # Operating points to compare. The tuned config wins on its fit-video but over-segments
    # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
    TUNED: Config = (0.05, 1.0, 1.0)


    def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
        """Keep intervals that overlap the labeled window [w0, w1].

        A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
        in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
        they are neither rewarded nor penalized. GTs are assumed already inside the window.
        """
        return [(s, e) for (s, e) in intervals if e > w0 and s < w1]


    def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
                    window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
        """Score one (config, gate) operating point inside the labeled window."""
        preds = predict_fn(cfg)
        if gate_crossings > 0 and points:
            preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
        preds = restrict_to_window(preds, *window)
        sc = score(preds, gts, iou)
        return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
"preds": preds}


    def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
            window_start: float = 0.0, window_end: Optional[float] = None,
            gate_crossings: int = 2, iou: float = 0.5,
            comparison_out: Optional[str] = None) -> dict:
        gts_all = load_golden_gts(labels_csv)
        if not gts_all:
            raise SystemExit(f"no usable human rallies in {labels_csv}")
        if window_end is None:
            window_end = max(e for _, e in gts_all)
        window = (window_start, window_end)
        gts = restrict_to_window(gts_all, *window)

        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        traj_end = (max(p.frame for p in points) / fps) if points else 0.0

        print("=== Pipeline vs PARTIAL human labels ===")
        print(f"labels: {labels_csv}")
        print(f"  {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
{window_end:.1f}s] "
              f"-> {len(gts)} rallies in-window")
        print(f"trajectory covers ~0-{traj_end:.1f}s "
              f"({'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter
than window'})")
        print(f"IoU>={iou}; net-crossing gate K={gate_crossings}\n")
```

```
91
92          # Compare the standard operating points: baseline / baseline+gate / tuned /
tuned+gate.
93          points_for_gate = points
94          candidates = [
95              ("baseline", BASELINE, 0),
96              (f"baseline+gate{gate_crossings}", BASELINE, gate_crossings),
97              ("tuned", TUNED, 0),
98              (f"tuned+gate{gate_crossings}", TUNED, gate_crossings),
99          ]
100         results = []
101         header = f"{'operating point':<18} {'pred':>4} {'TP':>3} {'FN':>3} {'FP':>3} " \
102                  f"{'prec':>6} {'recall':>6} {'F1':>6} {'mIoU':>6} {'sErr':>6} {'eErr':>6}"
103         print(header)
104         print("-" * len(header))
105         for name, cfg, gate in candidates:
106             r = _eval_point(predict_fn, cfg, points_for_gate, fps, gts, window, gate, iou)
107             sc = r["score"]
108             r["name"] = name
109             results.append(r)
110             print(f"{name:<18} {r['n_pred']:>4} {sc.true_positives:>3}
{sc.false_negatives:>3} "
111                   f"{sc.false_positives:>3} {sc.precision:>6.3f} {sc.recall:>6.3f}
{sc.f1:>6.3f} "
112                   f"{sc.mean_iou:>6.3f} {sc.mean_start_error:>6.2f}
{sc.mean_end_error:>6.2f}")
113
114         best = max(results, key=lambda r: r["score"].f1)
115         print(f"\nbest operating point by F1: {best['name']} (F1 {best['score'].f1:.3f})")
116
117         # Per-rally detail for the best point: which human rallies matched/missed + extra
fires.
118         comp = build_comparison(best["preds"], gts, iou)
119         comp_out = comparison_out or osp.join("output", "adarsh_avi_vs_human.csv")
120         write_comparison_csv(comp_out, comp)
121         print(f"per-rally detail ({best['name']}) -> {comp_out}")
122
123         return {
124             "window": window, "n_human_in_window": len(gts),
125             "results": [{"name": r["name"], "cfg": r["cfg"], "gate": r["gate"],
126                          "n_pred": r["n_pred"], **r["score"].as_dict()} for r in results],
127             "best": best["name"], "comparison_csv": comp_out,
128         }
129
130
131     # Grid centered on the two levers the human-GT failure analysis flagged:
132     # min_window_duration DOWN (recover short 3-5s rallies) and merge_gap UP (heal
133     # fragmented long rallies). Gate is swept on/off because it can over-prune.
134     DEFAULT_GRID = [
135         (vt, mg, mwd)
136         for vt in (0.01, 0.02, 0.03, 0.05)
137         for mg in (1.0, 2.0, 3.0, 4.0)
138         for mwd in (0.5, 1.0, 1.5, 2.0)
139     ]
140
141
142     def sweep(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
143               window_start: float = 0.0, window_end: Optional[float] = None,
144               gate_options: Tuple[int, ...] = (0, 2), iou: float = 0.5,
145               grid: Optional[List[Config]] = None, top: int = 12) -> dict:
146         """Grid-search (velocity, merge_gap, min_dur) x gate vs human labels, in-window.
147
148         WARNING: this is fit-on-self on ONE video -> optimistic. It shows the achievable
149         ceiling + which levers matter, NOT a transferable config. The honest operating
150         point comes from leave-one-video-out once >=3 videos are labeled.
```

```
        """
        grid = grid or DEFAULT_GRID
        gts_all = load_golden_gts(labels_csv)
        if not gts_all:
            raise SystemExit(f"no usable human rallies in {labels_csv}")
        if window_end is None:
            window_end = max(e for _, e in gts_all)
        window = (window_start, window_end)
        gts = restrict_to_window(gts_all, *window)
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        rows: List[dict] = []
        for cfg in grid:
            for gate in gate_options:
                r = _eval_point(predict_fn, cfg, points, fps, gts, window, gate, iou)
                sc = r["score"]
                rows.append({"cfg": cfg, "gate": gate, "n_pred": r["n_pred"], "f1": sc.f1,
                             "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
                             "tp": sc.true_positives, "fp": sc.false_positives, "fn":
sc.false_negatives})
        rows.sort(key=lambda x: (x["f1"], x["mean_iou"]), reverse=True)

        print(f"=== Config sweep vs human labels ({len(gts)} rallies in "
              f"[{window_start:.0f},{window_end:.0f}]s; {len(grid)}x{len(gate_options)}
configs) ===")
        print("NOTE: fit-on-self on ONE video = optimistic; real test is leave-one-video-
out.\n")
        hdr = f"{'#':>3} {'veloc':>6} {'merge':>6} {'minDur':>6} {'gate':>4} " \
              f"{'F1':>6} {'prec':>6} {'recall':>6} {'mIoU':>6} {'TP':>3} {'FP':>3}
{'FN':>3}"
        print(hdr)
        print("-" * len(hdr))
        for i, r in enumerate(rows[:top], 1):
            vt, mg, mwd = r["cfg"]
            print(f"{i:>3} {vt:>6.3f} {mg:>6.1f} {mwd:>6.1f} {r['gate']:>4} "
                  f"{r['f1']:>6.3f} {r['precision']:>6.3f} {r['recall']:>6.3f}
{r['mean_iou']:>6.3f} "
                  f"{r['tp']:>3} {r['fp']:>3} {r['fn']:>3}")
        b = rows[0]
        print(f"\nbest: velocity={b['cfg'][0]} merge_gap={b['cfg'][1]} min_dur={b['cfg'][2]}
"
              f"gate={b['gate']} -> F1 {b['f1']:.3f} (P {b['precision']:.3f} / R
{b['recall']:.3f})")
        return {"window": window, "n_gt": len(gts), "ranked": rows, "best": b}


    def main() -> None:
        ap = argparse.ArgumentParser(description=__doc__)
        ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
        ap.add_argument("--labels", required=True, help="human golden CSV
(rally_number,start_time,end_time,...)")
        ap.add_argument("--fps", type=float, default=59.94)
        ap.add_argument("--frame-width", type=float, default=1920.0)
        ap.add_argument("--window-start", type=float, default=0.0)
        ap.add_argument("--window-end", type=float, default=None,
                        help="end of the labeled region (default: last labeled rally end)")
        ap.add_argument("--gate", type=int, default=2, help="net-crossing gate K (0=off, 2
recommended)")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--comparison-out", default=None)
        ap.add_argument("--sweep", action="store_true",
                        help="grid-search configs vs labels (fit-on-self; finds the ceiling
```

```
+ best levers)")
205         ap.add_argument("--top", type=int, default=12, help="how many sweep rows to print")
206         args = ap.parse_args()
207         if args.sweep:
208             sweep(args.trajectory, args.labels, args.fps, args.frame_width,
209                   window_start=args.window_start, window_end=args.window_end,
210                   iou=args.iou, top=args.top)
211         else:
212             run(args.trajectory, args.labels, args.fps, args.frame_width,
213                 window_start=args.window_start, window_end=args.window_end,
214                 gate_crossings=args.gate, iou=args.iou, comparison_out=args.comparison_out)
215
216
217     if __name__ == "__main__":
218         main()
219
```

19. assistant (2026-06-18T01:33:47.218Z)

Now let me look at the rally_gate (net-crossing detection - frame/time windowing), per_user_eval, promotion, and harness files. Let me also check the bootstrap_ci percentile indexing more carefully against eval_stats.

20. user (2026-06-18T01:33:48.656Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21
22      from dataclasses import dataclass
23      from typing import List, Optional, Sequence, Tuple
24
25      Interval = Tuple[float, float]
26
27
28      @dataclass
29      class GatedWindow:
30          start: float
31          end: float
32          crossings: int
33          visible_points: int
34          kept: bool
35
36
37      def _spread(vals: Sequence[float]) -> float:
38          """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39          if len(vals) < 2:
40              return 0.0
```

```
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06,
                     estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
        """Annotate each (start,end) window with its shuttle net-crossing count and a
        keep decision (crossings >= min_crossings). Deadband is a fraction of the play
        axis span. Net/axis are estimated once over the whole trajectory (stable).

        `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
        `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
        per window ? same result, avoids the per-window re-estimation."""
        axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
        deadband = deadband_frac * span
        out: List[GatedWindow] = []
```

```
        for (s, e) in windows:
            wp = _points_in(points, fps, s, e)
            c = count_net_crossings(wp, axis, net, deadband)
            vis = sum(1 for p in wp if p.visible)
            out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                                   kept=c >= min_crossings))
        return out


    def apply_gate(points, windows: List[Interval], fps: float,
                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
        """Convenience: return only the windows that pass the crossing gate."""
        return [(g.start, g.end) for g in
                gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]

```

21. user (2026-06-18T01:33:48.958Z)

```
      """Held-out-user LEARNING-CURVE eval ? "<K videos to superior quality" as a release
gate.

      The personalization feature's promise is operational, not aspirational: *after the user
      contributes K of their own videos, the personalized (candidate) recipe should beat the
      generic (control) one on the user's STILL-UNSEEN footage.* This module turns that
promise
      into a falsifiable criterion by sweeping the corpus size ``k`` and asking, at each
``k``,
      **does the candidate beat control on a held-out split, honestly?**

      It is the per-user analogue of `fusion_compare`: pure-CPU, no new scoring/stats math.
Every
      fold reuses the shipped experiment primitives ?

        - `experiment.predict`  ? variant ? held-out intervals (static variants need no fit);
        - `experiment._train`   ? fit a learned variant on the corpus split (the same honest
          train-on-others step `evaluate_variant`'s LOVO uses, here train-on-the-k-corpus);
        - `metrics.score`       ? per-video held-out F1;
        - `eval_stats.paired_delta_ci` ? the honest paired ?F1 (bootstrap CI + exact sign
test, C1).

      **Split semantics (held-out USER split, NOT leave-one-out).** At each ``k`` the first
``k``
      videos are the *personalization corpus* (fit/operating-point source) and the remaining
      ``len(videos) - k`` are *held-out* (scored, never seen during fit). This mirrors the
product
      reality ? the user has uploaded ``k`` videos and we want quality on their next one ? and
keeps
      ``k`` and the evaluation set cleanly disjoint, so a measured lift can't be in-sample.

      The headline ``smallest_k_with_lift`` is the smallest ``k`` at which the candidate beats
      control with the **CI clearing 0 AND the sign test agreeing** (B wins on more held-out
videos
      than it loses) ? the same promotion-grade bar `compare()`'s honest block uses, never a
bare
      mean. ``None`` means the criterion was never met on this corpus (the honest negative).

      Pure, offline, deterministic given ``seed``; no GPU/torch/cv2; no file I/O (callers pass
      `VideoCandidates`). See `docs/EXPERIMENT_HARNESS.md` +
`docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
      """
      from __future__ import annotations

      from typing import Any, Dict, List, Optional, Sequence

      from backend.eval import eval_stats as _stats
```

```
    from backend.eval import experiment
    from backend.eval.classifier import LogisticRegression
    from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
    from backend.eval.metrics import score


    def _fit_model(variant: Variant, corpus: Sequence[VideoCandidates],
                   iou: float) -> Optional[LogisticRegression]:
        """The fitted model a variant needs to predict held-out videos, or None if it needs
none.

        - **static variant** (no learned filter): None ? `predict` scores directly, no
leakage.
        - **pinned model** (``classifier_model`` set): the shipped model, loaded as-is (no
per-fold
          training ? reports "how the shipped artifact does on this user's held-out
footage").
        - **learned recipe** (``feature_names``, no pinned model): fit on the ``corpus``
split via
          the harness's own honest training step. The corpus and held-out sets are disjoint,
so the
          held-out scores are genuinely out-of-sample.
        """
        if variant.classifier_model is not None:
            return LogisticRegression.load(variant.classifier_model)
        if variant.feature_names:
            return experiment._train(variant, corpus, iou)
        return None


    def _heldout_f1s(variant: Variant, corpus: Sequence[VideoCandidates],
                     heldout: Sequence[VideoCandidates], iou: float) -> List[float]:
        """Per-held-out-video F1 for ``variant``, fit on ``corpus`` (if learned), scored on
        ``heldout``. Held-out videos are video-aligned with the returned list (caller pairs
B?A)."""
        model = _fit_model(variant, corpus, iou)
        f1s: List[float] = []
        for vc in heldout:
            assert vc.gts is not None  # guarded in learning_curve before any scoring
            preds = experiment.predict(variant, vc, model=model)
            f1s.append(score(preds, vc.gts, iou).f1)
        return f1s


    def learning_curve(videos: Sequence[VideoCandidates],
                       control_variant: Variant,
                       candidate_variant: Variant,
                       *,
                       iou: float = 0.5,
                       min_k: int = 2,
                       seed: int = 0) -> Dict[str, Any]:
        """Sweep corpus size ``k`` and report where the candidate first honestly beats
control.

        For one user's ``videos``, for each ``k`` in ``min_k .. len(videos) - 1``: fit/score
both
        variants on the first ``k`` videos (the personalization corpus) and evaluate on the
remaining
        held-out videos. Per-held-out-video F1s are paired B?A and summarised with
        ``eval_stats.paired_delta_ci`` (mean ?F1 + bootstrap CI + exact sign test).

        Returns ``{n_videos, min_k, iou, per_k, smallest_k_with_lift}`` where each ``per_k``
record is::

            {k, n_heldout, control_f1, candidate_f1,
```

```
91               mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test}
92
93          ``control_f1`` / ``candidate_f1`` are the mean held-out F1 of each variant at that
``k``.
94          ``smallest_k_with_lift`` is the smallest ``k`` whose record shows a real lift ?
candidate
95          above control, the ?F1 CI clearing 0, AND the sign test agreeing (B wins on strictly
more
96          held-out videos than it loses) ? else ``None`` (the honest "never wins" answer).
97
98          Pure + deterministic given ``seed`` (the only randomness is the CI bootstrap).
Raises
99          `ExperimentError` on unlabeled videos or a too-small corpus (no valid ``k`` to
sweep).
100         """
101         n = len(videos)
102         for vc in videos:
103             if vc.gts is None:
104                 raise ExperimentError(
105                     f"{vc.name} has no golden labels; learning_curve needs labeled held-out
videos")
106         if min_k < 1:
107             raise ExperimentError(f"min_k must be >= 1 (got {min_k})")
108         if n <= min_k:
109             # Every k in [min_k, n-1] needs k >= min_k corpus videos AND >= 1 held-out
video, so
110             # n must exceed min_k. Too-few videos ? no sweepable k; report an empty, honest
curve.
111             return {
112                 "n_videos": n,
113                 "min_k": min_k,
114                 "iou": iou,
115                 "per_k": [],
116                 "smallest_k_with_lift": None,
117             }
118
119         per_k: List[Dict[str, Any]] = []
120         smallest_k_with_lift: Optional[int] = None
121         for k in range(min_k, n):
122             corpus = videos[:k]
123             heldout = videos[k:]
124             control_f1s = _heldout_f1s(control_variant, corpus, heldout, iou)
125             candidate_f1s = _heldout_f1s(candidate_variant, corpus, heldout, iou)
126
127             paired = _stats.paired_delta_ci(control_f1s, candidate_f1s, seed=seed)
128             sign_test = paired["sign_test"]
129             # Honest lift = the candidate is above control AND the CI clears 0 AND the sign
test
130             # agrees (B wins on strictly more held-out videos than it loses) ? the promotion
bar.
131             control_mean = sum(control_f1s) / len(control_f1s) if control_f1s else 0.0
132             candidate_mean = sum(candidate_f1s) / len(candidate_f1s) if candidate_f1s else
0.0
133             has_lift = (
134                 candidate_mean > control_mean
135                 and bool(paired["ci_excludes_zero"])
136                 and paired["mean_delta"] > 0.0
137                 and sign_test["wins"] > sign_test["losses"]
138             )
139             per_k.append({
140                 "k": k,
141                 "n_heldout": len(heldout),
142                 "control_f1": control_mean,
143                 "candidate_f1": candidate_mean,
144                 "mean_delta": paired["mean_delta"],
```

```
                "ci_low": paired["ci_low"],
                "ci_high": paired["ci_high"],
                "ci_excludes_zero": bool(paired["ci_excludes_zero"]),
                "sign_test": sign_test,
            })
            if has_lift and smallest_k_with_lift is None:
                smallest_k_with_lift = k

        return {
            "n_videos": n,
            "min_k": min_k,
            "iou": iou,
            "per_k": per_k,
            "smallest_k_with_lift": smallest_k_with_lift,
        }

```

22. assistant (2026-06-18T01:33:54.368Z)

Now let me look carefully at the `count_net_crossings` function and `_points_in`. There's a potential issue: `_points_in` uses `start * fps <= p.frame <= end * fps` ? this is frame index comparison. That seems consistent. Let me examine the bootstrap_ci percentile bounds more carefully, and look at promotion.py, harness.py, and the calibrate files.

Let me cross-cut on a specific concern in `bootstrap_ci`: the percentile index computation. With `confidence=0.95`, `alpha=0.025`, `last=n_boot-1`. `lo = means[floor(0.025*last)]`, `hi = means[ceil(0.975*last)]`. This looks reasonable.

Let me check the harness, promotion, and gt_loader files.

23. user (2026-06-18T01:33:55.767Z)

```
      """Score the pipeline's detected rallies against ground-truth annotations.

      Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
      so scoring is fast, deterministic, and free. Two prediction "stages" can be
      scored:

        - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
          the shuttle/person detector found, before AI confirmation.
        - ``final``      ? confirmed play segments (``play_segments`` table). After the
          AI handoff trimmed/validated boundaries.

      Comparing the two tells you whether the weak link is *detection* (candidates
      already miss rallies) or *segmentation* (candidates catch them but final is
      wrong) ? see docs/EVALUATION.md.
      """

      import logging
      from dataclasses import dataclass, field
      from typing import Any, List, Dict, Optional

      from backend.infrastructure.database import Database
      from backend.eval import metrics
      from backend.eval.metrics import Interval, Score, MatchResult

      logger = logging.getLogger(__name__)

      STAGES = ("candidates", "final")


      @dataclass
      class StageReport:
          stage: str
          pred_intervals: List[Interval]
```

```
        score: Score
        match_result: MatchResult
        pr_curve: List[Score] = field(default_factory=list)


    @dataclass
    class EvalReport:
        video_id: str
        iou_threshold: float
        gt_intervals: List[Interval]
        stages: Dict[str, StageReport] = field(default_factory=dict)


    def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
        """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
        ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
        ivs.sort(key=lambda iv: iv[0])
        return ivs


    def get_predictions(db: Database, video_id: str, stage: str,
                        detector: Optional[str] = None) -> List[Interval]:
        """Fetch predicted rally intervals for one stage from the DB."""
        if stage == "candidates":
            rows = db.query_candidate_segments(video_id, detector=detector)
        elif stage == "final":
            rows = db.query_play_segments(video_id, {})
        else:
            raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
        return _rows_to_intervals(rows)


    def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
                 stages: List[str], iou_threshold: float = 0.5,
                 detector: Optional[str] = None,
                 with_pr_curve: bool = False) -> EvalReport:
        """Score the requested stages for one video against ground truth."""
        report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
                            gt_intervals=gt_intervals)
        for stage in stages:
            preds = get_predictions(db, video_id, stage, detector=detector)
            mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
            s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
            curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
            report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
                                               score=s, match_result=mr, pr_curve=curve)
        return report


    # ---------------------------------------------------------------------- reporting

    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    def report_to_dict(report: EvalReport) -> dict:
        """JSON-serialisable view of an EvalReport."""
        out: Dict[str, Any] = {
            "video_id": report.video_id,
            "iou_threshold": report.iou_threshold,
            "num_ground_truth": len(report.gt_intervals),
            "stages": {},
```

```
99          }
100         for stage, sr in report.stages.items():
101             out["stages"][stage] = {
102                 "score": sr.score.as_dict(),
103                 "pr_curve": [s.as_dict() for s in sr.pr_curve],
104             }
105         return out
106
107
108     def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109         """Human-readable console report for one video."""
110         lines = [
111             "=" * 64,
112             f" Rally evaluation ? video '{report.video_id}'",
113             f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
114             "=" * 64,
115             f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116             f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117             "-" * 64,
118         ]
119         for stage in report.stages:
120             s = report.stages[stage].score
121             lines.append(
122                 f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
123                 f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124                 f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125             )
126         lines.append("=" * 64)
127
128         if show_failures:
129             for stage, sr in report.stages.items():
130                 misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
131                 fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132                 lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
133                 for iv in misses:
134                     lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135                 lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136                 for iv in fps:
137                     lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139         return "\n".join(lines)
140
```

24. user (2026-06-18T01:33:56.155Z)

```
1       """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not
regress
2       at the SERVED windowing.
3
4       **Why this exists.** ``per_user_gate.should_promote_for_user`` already grades a B?A lift
5       honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever
scores
6       it is handed ? and the foundational bug was that those scores came from the *recall-
oriented
7       PROPOSAL* windowing (``backend.eval.windowing.PROPOSAL``), which production never
serves. So a
8       cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **+0.074**)
while
9       regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on
the
```

```
    served windowing** is.

    This module composes ? it adds no new statistics. It layers ONE rule on top of
    ``should_promote_for_user``:

        A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES
NOT
        REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served
CI is
        estimable ? that CI's lower bound does not sit clearly in regression territory).

    A proposal-windowing win is still *reported* (it's useful signal for where to look
next), but it
    can never carry a promotion on its own. The decision returns BOTH the served and the
proposal
    verdicts so the eval?serve contrast is explicit and auditable.

    Pure, deterministic, stdlib-only over ``eval_stats``/``per_user_gate``. Additive:
existing eval
    paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
    """
    from __future__ import annotations

    from dataclasses import dataclass, field
    from typing import Any, Dict, Optional, Sequence

    from backend.eval.eval_stats import paired_delta_ci
    from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user

    __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]


    def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
                         eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str,
Any]:
        """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
core).

        ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
windowing**
        (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
test) plus
        a single ``regresses`` boolean:

          - ``regresses=True``  when the served mean ?F1 is below ``-eps`` (a real served
drop), OR
            the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative
band).
          - ``regresses=False`` when the cue holds the line at the served operating point
(mean ?
            ``-eps`` and the CI is not a confident net-negative).

        ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served
mean". A tiny
        positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at
n?6).
        """
        delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
        mean_delta = float(delta["mean_delta"])
        ci_low = float(delta["ci_low"])
        ci_high = float(delta["ci_high"])
        # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-
negative.
        mean_regresses = mean_delta < -eps
```

```
        ci_confidently_negative = ci_high < -eps
        regresses = bool(mean_regresses or ci_confidently_negative)
        return {
            "regresses": regresses,
            "mean_delta": mean_delta,
            "ci_low": ci_low,
            "ci_high": ci_high,
            "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
            "sign_test": delta["sign_test"],
            "n": int(delta["n"]),
            "eps": float(eps),
        }


    @dataclass
    class ServedGateResult:
        """A promotion decision that BOTH measures the cue on the served windowing AND
reports the
        proposal-windowing view, so the eval?serve contrast is never hidden.

        - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would
promote
          *and* the served check did not veto (the cue does not regress at the served
windowing).
        - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not
regress served).
        - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted
but the
          served regression check overruled it (the exact Gate-A failure mode this module
prevents).
        - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on
the
          *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
        - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs
+ sign
          tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
        - ``reason`` ? short human-readable explanation of the final verdict.
        """

        promote: bool
        served_safe: bool
        vetoed_by_served: bool
        reason: str
        base_decision: PromotionDecision
        served: Dict[str, Any]
        proposal: Optional[Dict[str, Any]] = field(default=None)

        def as_dict(self) -> Dict[str, Any]:
            return {
                "promote": self.promote,
                "served_safe": self.served_safe,
                "vetoed_by_served": self.vetoed_by_served,
                "reason": self.reason,
                "base_decision": self.base_decision.as_dict(),
                "served": self.served,
                "proposal": self.proposal,
            }


    def promote_if_served_safe(
        served_a: Sequence[float],
        served_b: Sequence[float],
        *,
        proposal_a: Optional[Sequence[float]] = None,
        proposal_b: Optional[Sequence[float]] = None,
```

```
        structural: bool = False,
        min_n: int = 5,
        p_threshold: float = 0.05,
        served_regress_eps: float = 0.0,
        n_boot: int = 2000,
        seed: int = 0,
    ) -> ServedGateResult:
        """Promote a cue ONLY if it does not regress at the SERVED windowing (the
foundational rule).

        The honest bar (``should_promote_for_user``: min-N floor + ?F1 CI excludes 0 + sign
test) is
        applied to the **served** scores ? the operating point production runs ? so a
proposal-only
        win can never carry a promotion. The optional ``proposal_*`` scores are scored too
and
        reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
their
        own).

        Args:
          served_a/served_b: per-video held-out scores for variant A (control) and B (the
cue),
            measured at the **SERVED** windowing. These drive the decision.
          proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
Reported for
            contrast; the served regression guard still has the final say.
          structural: forwarded to ``should_promote_for_user`` ? a structural guard (e.g.
Gate-A vs
            held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
            the served default if it regresses served. So for ``structural=True`` we report
            ``keep_on`` honestly but ``promote`` reflects the served-safety check.
          served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").

        Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
        """
        # 1) The honest gate, applied to the SERVED scores (never the proposal ones).
        base = should_promote_for_user(
            served_a, served_b,
            structural=structural, min_n=min_n, p_threshold=p_threshold,
            n_boot=n_boot, seed=seed,
        )

        # 2) The eval?serve guard: does B regress A at the served windowing?
        served = served_regresses(served_a, served_b, eps=served_regress_eps,
                                  n_boot=n_boot, seed=seed)
        served_safe = not served["regresses"]

        # 3) Optional proposal-windowing view (reported, never promoting on its own).
        proposal: Optional[Dict[str, Any]] = None
        if proposal_a is not None and proposal_b is not None:
            proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
                                        n_boot=n_boot, seed=seed)

        # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
add one.
        if structural:
            # A structural guard ships ON regardless of measured lift (base.keep_on stays
True), but
            # it is only auto-promoted to the SERVED default when it is served-safe. If it
regresses
            # served, promote=False (don't bake an eval-only win into production) while
```

```
keep_on holds.
167             promote = served_safe
168             vetoed = False
169             if served_safe:
170                 reason = ("structural guard kept ON; served-safe (does not regress at the
served "
171                           f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for
served default")
172             else:
173                 reason = ("structural guard kept ON, but NOT promoted to the served default:
it "
174                           f"REGRESSES at the served windowing (?F1
{served['mean_delta']:+.4f}); "
175                           "an eval-only win must not be served")
176         else:
177             would_promote = base.promote
178             if would_promote and not served_safe:
179                 promote = False
180                 vetoed = True
181                 reason = ("VETOED by the served-windowing guard: the cue clears the honest
bar but "
182                           f"REGRESSES at the served windowing (served ?F1
{served['mean_delta']:+.4f}"
183                           f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win
on a "
184                           "windowing production doesn't serve cannot promote.")
185             elif would_promote:
186                 promote = True
187                 vetoed = False
188                 reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
189                           f"{served['mean_delta']:+.4f}). " + base.reason)
190             else:
191                 promote = False
192                 vetoed = False
193                 reason = base.reason
194
195         return ServedGateResult(
196             promote=promote,
197             served_safe=served_safe,
198             vetoed_by_served=vetoed,
199             reason=reason,
200             base_decision=base,
201             served=served,
202             proposal=proposal,
203         )
204
```

25. assistant (2026-06-18T01:33:59.700Z)

Let me look at the per_user_gate, gt_loader, and the rally_seg_eval files, plus the calibrate_wasb to understand gt loading and the windowing for time/frame rounding.

26. user (2026-06-18T01:34:01.135Z)

```
1       """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3       The owner adopted personalization as a **flag-gated** feature on a **generalization-
first**
4       baseline: a cue/variant B may be flipped ON *for one user* only when that user's own
5       held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6       fall back to the shipped baseline (variant A). This module is the decision logic for
that
7       flip; it is pure (no I/O) and **reuses** ``backend.eval.eval_stats`` for every statistic
8       (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.
```

```
9
10      The counter-case this guards against (and its mitigations, implemented here):
11
12      1. **Minimum-N floor.** Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13         (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from
14         too few held-out videos are noise, not signal.
15      2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI
16         **excludes 0** *and* the paired sign test agrees (majority of videos win, p <=
threshold).
17         This is the global promotion bar applied per user.
18      3. **Structural guards are never disabled on "no lift".** A *structural* guard (e.g.
Gate-A
19         net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20         modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21         that simply lacks those failure cases must NEVER turn the guard OFF. For
``structural=True``
22         the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24      Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25      default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26      threaded through). Nothing here changes existing eval behaviour; callers opt in.
27      """
28      from __future__ import annotations
29
30      from dataclasses import dataclass, field
31      from typing import Any, Dict, Sequence
32
33      from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35      __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38      @dataclass
39      class PromotionDecision:
40          """Structured result of a per-user promotion decision.
41
42          - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
43            cue that cleared the full bar).
44          - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
45            always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
46            ``promote``.
47          - ``reason`` ? short human-readable explanation of the decision.
48          - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
49            so a caller can log/audit exactly why a user was or wasn't personalized.
50          """
51
52          promote: bool
53          keep_on: bool
54          reason: str
55          n: int
56          mean_delta: float
57          ci_low: float
58          ci_high: float
59          ci_excludes_zero: bool
60          sign_test: Dict[str, float] = field(default_factory=dict)
```

```
61          floor_met: bool = False
62          structural_protected: bool = False
63
64          def as_dict(self) -> Dict[str, Any]:
65              """Plain-dict view (for JSON logging / telemetry)."""
66              return {
67                  "promote": self.promote,
68                  "keep_on": self.keep_on,
69                  "reason": self.reason,
70                  "n": self.n,
71                  "mean_delta": self.mean_delta,
72                  "ci_low": self.ci_low,
73                  "ci_high": self.ci_high,
74                  "ci_excludes_zero": self.ci_excludes_zero,
75                  "sign_test": self.sign_test,
76                  "floor_met": self.floor_met,
77                  "structural_protected": self.structural_protected,
78              }
79
80
81      def should_promote_for_user(
82          a_scores: Sequence[float],
83          b_scores: Sequence[float],
84          *,
85          structural: bool = False,
86          min_n: int = 5,
87          p_threshold: float = 0.05,
88          confidence: float = 0.95,
89          n_boot: int = 2000,
90          seed: int = 0,
91          eps: float = 0.0,
92      ) -> PromotionDecision:
93          """Decide whether to flip variant B ON for ONE user from that user's held-out
scores.
94
95          ``a_scores`` / ``b_scores`` are the user's **per-video** held-out F1 (or any score)
for
96          variant A (control / shipped baseline) and variant B (with the cue), aligned by
video
97          (same order). The decision rules (faithful to the module docstring):
98
99          1. ``structural=True`` ? the guard is **never disabled on "no lift"**: return
100             ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
value is
101             in failure modes the user's corpus may not yet contain.
102         2. Otherwise promote **iff ALL** hold ? the minimum-N floor (``n >= min_n``), the
paired
103             ?F1 bootstrap CI **excludes 0**, and the paired sign test agrees (majority of
videos
104             win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
105
106         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
``paired_sign_test``);
107         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
108         """
109         n = min(len(a_scores), len(b_scores))
110         floor_met = n >= min_n
111
112         # --- Evidence (reuse eval_stats; never reinvent the statistics)
----------------------
113         if n == 0:
114             # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
115             sign_test: Dict[str, float] = paired_sign_test([], eps)
116             mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
117         else:
```

```
            delta = paired_delta_ci(
                a_scores, b_scores,
                confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
            )
            mean_delta = float(delta["mean_delta"])
            ci_low = float(delta["ci_low"])
            ci_high = float(delta["ci_high"])
            ci_excludes_zero = bool(delta["ci_excludes_zero"])
            sign_test = delta["sign_test"]

        sign_p = float(sign_test.get("p_value", 1.0))
        sign_wins = float(sign_test.get("wins", 0.0))
        sign_losses = float(sign_test.get("losses", 0.0))
        # The sign test "agrees" with a promotion only if it is significant AND the majority
        # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
        # never satisfy the promotion bar.
        sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)

        # --- Rule 1: structural guards are never disabled on "no lift"
----------------------
        if structural:
            return PromotionDecision(
                promote=False,           # nothing to "promote" ? it ships ON by design
                keep_on=True,            # ALWAYS kept ON, regardless of measured per-user
lift
                reason=(
                    "structural guard kept ON regardless of measured per-user lift "
                    "(value is in failure modes this user's corpus may not yet contain)"
                ),
                n=n,
                mean_delta=mean_delta,
                ci_low=ci_low,
                ci_high=ci_high,
                ci_excludes_zero=ci_excludes_zero,
                sign_test=sign_test,
                floor_met=floor_met,
                structural_protected=True,
            )

        # --- Rule 2: minimum-N floor (refuse before even looking at lift)
--------------------
        if not floor_met:
            return PromotionDecision(
                promote=False,
                keep_on=False,           # fall back to the shipped baseline (variant A)
                reason=(
                    f"below minimum-N floor (n={n} < min_n={min_n}); "
                    "falling back to shipped baseline"
                ),
                n=n,
                mean_delta=mean_delta,
                ci_low=ci_low,
                ci_high=ci_high,
                ci_excludes_zero=ci_excludes_zero,
                sign_test=sign_test,
                floor_met=False,
                structural_protected=False,
            )

        # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
----------------
        if ci_excludes_zero and sign_agrees:
            promote = True
            reason = (
```

```
                f"promoted: n={n} >= {min_n}, ?F1 95% CI "
                f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
                f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
            )
        else:
            promote = False
            misses = []
            if not ci_excludes_zero:
                misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
            if not sign_agrees:
                misses.append(
                    f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}, "
                    f"p={sign_p:.4f})"
                )
            reason = (
                "insufficient evidence (" + "; ".join(misses) + "); "
                "falling back to shipped baseline"
            )

        return PromotionDecision(
            promote=promote,
            keep_on=promote,             # non-structural: keep_on tracks promote exactly
            reason=reason,
            n=n,
            mean_delta=mean_delta,
            ci_low=ci_low,
            ci_high=ci_high,
            ci_excludes_zero=ci_excludes_zero,
            sign_test=sign_test,
            floor_met=True,
            structural_protected=False,
        )

```

27. user (2026-06-18T01:34:01.355Z)

```
      """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).

      The repo historically carried two incompatible loaders:
      - ``annotations.load_annotations``: positional ``start,end`` (the collector's
        ``curate export`` format), raises on bad rows;
      - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
        (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.

      A file in one convention fed to the other loader yielded zero/garbage intervals with no
      error. This module accepts BOTH conventions (header-mapped, else positional), so every
      quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
      flows through one reader. Both legacy entry points now delegate here.

      Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
      - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
        columns ? rally_number, ending_reason, sport, ? ? are ignored);
      - no header: columns 0,1 positionally.
      Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
(``annotations.parse_timestamp``).
      """
      import csv
      import logging
      from typing import List, Optional, Tuple

      from backend.eval.annotations import parse_timestamp

      logger = logging.getLogger(__name__)
```

```

    Interval = Tuple[float, float]

    _START_NAMES = ("start", "start_time")
    _END_NAMES = ("end", "end_time")


    def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
        """If `row` is a header naming start/end columns, return their indices, else
None."""
        cells = [(c or "").strip().lower() for c in row]
        start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
        end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
        if start_idx is not None and end_idx is not None:
            return start_idx, end_idx
        return None


    def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
        """Load GT rally intervals from either golden-CSV convention.

        strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
        (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
        strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
        skipped ? a silently shrinking GT set flatters precision.

        Returns intervals sorted by start; overlapping GT intervals are logged as a warning
        (they break one-to-one matching and usually indicate a labeling error).
        """
        intervals: List[Interval] = []
        skipped = 0
        cols: Optional[Tuple[int, int]] = None   # (start_idx, end_idx); None until detected
        saw_header = False

        with open(csv_path, newline="", encoding="utf-8-sig") as f:
            for lineno, row in enumerate(csv.reader(f), start=1):
                if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
                    continue
                if cols is None and not saw_header:
                    header = _header_columns(row)
                    if header is not None:
                        cols = header
                        saw_header = True
                        continue            # consume the header row
                    cols = (0, 1)           # headerless: positional start,end
                assert cols is not None     # every branch above assigns it
                si, ei = cols
                try:
                    if len(row) <= max(si, ei):
                        raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
                    start = parse_timestamp(row[si])
                    end = parse_timestamp(row[ei])
                    if start < 0:
                        raise ValueError(f"negative start ({start})")
                    if start >= end:
                        raise ValueError(f"start ({start}) >= end ({end})")
                except ValueError as e:
                    if strict:
                        raise ValueError(f"{csv_path}:{lineno}: {e}") from e
                    skipped += 1
                    continue
                intervals.append((start, end))

        if skipped:
```

```
90              logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92          intervals.sort(key=lambda iv: iv[0])
93          for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94              if s2 < e1:
95                  logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                                 "labeling error? Overlaps break one-to-one matching.",
97                                 csv_path, s1, e1, s2, e2)
98          return intervals
99
```

28. assistant (2026-06-18T01:34:06.023Z)

Now let me examine `parse_timestamp` in annotations.py (timestamp parsing ? common off-by-one / rounding source), and rally_seg_eval.py which aggregates per-video metrics. Let me also look at experiment.py for the variant prediction logic.

29. user (2026-06-18T01:34:07.434Z)

```
1       """Load hand-annotated ground-truth rally timestamps from CSV.
2
3       Annotation file format (one file per video):
4
5           start,end
6           00:01:12,00:01:39
7           102.5,131.0
8
9       - A header row `start,end` is optional (auto-detected and skipped).
10      - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11        `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12      - Blank lines and `#` comment lines are ignored.
13      """
14
15      import csv
16      import logging
17      from typing import List, Tuple
18
19      logger = logging.getLogger(__name__)
20
21      Interval = Tuple[float, float]
22
23
24      def parse_timestamp(value: str) -> float:
25          """Parse a timestamp string to float seconds.
26
27          Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28          Raises ValueError on anything unparseable so bad annotations fail loudly.
29          """
30          value = value.strip()
31          if not value:
32              raise ValueError("empty timestamp")
33          if ":" in value:
34              parts = value.split(":")
35              if len(parts) > 3:
36                  raise ValueError(f"too many ':' segments in timestamp '{value}'")
37              total = 0.0
38              for part in parts:                # most-significant first (H, M, S)
39                  total = total * 60 + float(part)
40              return total
41          return float(value)
42
43
44      def load_annotations(csv_path: str) -> List[Interval]:
45          """Load and validate ground-truth rally intervals from a CSV file.
```

```
46
47          Delegates to the canonical loader (gt_loader.py, ADR-008 prereq), which accepts BOTH
48          golden conventions: positional/headered ``start,end`` (the collector export) and the
49          rally-annotator ``rally_number,start_time,end_time,...`` schema. Strict: raises
50          ValueError on malformed rows (bad timestamp, start >= end) so labeling mistakes
51          surface immediately rather than silently skewing scores. Returns intervals sorted by
52          start time.
53          """
54          from backend.eval.gt_loader import load_gt_intervals
55          intervals = load_gt_intervals(csv_path, strict=True)
56          logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
57          return intervals
58
59
60      def write_scaffold(csv_path: str) -> None:
61          """Write an empty annotation file with just the header, for hand-filling."""
62          with open(csv_path, "w", newline="") as f:
63              writer = csv.writer(f)
64              writer.writerow(["start", "end"])
65
```

30. user (2026-06-18T01:34:07.838Z)

```
1       """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2       over-segmentation-sensitive metric set, honestly.
3
4       This is the harness the rally-quality project (``docs/RALLY_QUALITY_RESEARCH.md`` §4.5 /
§7)
5       measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6       IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k,
the
7       segment-count ratio, and the explicit ``merge_split`` breakdown
(``segmentation_metrics``).
8
9       For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10      windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11      operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12      the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13      diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15      GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16      trajectory path + fps + frame_width; the GT rally intervals come from
17      ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
files.
18
19      Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20      """
21      from __future__ import annotations
22
23      import argparse
24      import json
25      import os
26      from typing import Any, Dict, List, Optional
27
28      from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
29      from backend.eval.metrics import Interval
30      from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
31
```

```
    DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
    DEFAULT_FEATURES_DIR = "output"
    DEFAULT_IOU = 0.5


    # ---------------------------------------------------------------------- #
    # Pure scoring core (no I/O ? unit-testable)
    # ---------------------------------------------------------------------- #
    def score_intervals(preds: List[Interval], gts: List[Interval],
                        iou: float = DEFAULT_IOU,
                        tau_start: Optional[float] = None,
                        tau_end: Optional[float] = None) -> Dict[str, Any]:
        """The full per-video metric row for predicted vs ground-truth rally intervals.

        Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
        over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
        merged mega-window is *visible* even when IoU-F1 isn't moved.

        Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
        tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
        tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
        start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
        """
        from backend.eval import tolerance_metrics as tm
        ts = tm.TAU_START if tau_start is None else tau_start
        te = tm.TAU_END if tau_end is None else tau_end
        sc = metrics.score(preds, gts, iou_threshold=iou)
        f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
        ms = segmentation_metrics.merge_split_report(preds, gts)
        tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
        be = tm.boundary_error_report(preds, gts)  # unconditional best-overlap (the
R4-aiming diagnostic)
        osr = tm.over_seg_report(preds, gts)
        return {
            "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
            "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
            "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
            "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
            "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
            "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
            "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
            # --- R2 (tolerance-match) ? the task-faithful headline block ---
            "tau_start": ts, "tau_end": te,
            "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
            "tol_matched": float(len(tol_matches)),
            "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
            "split_count": osr["split_count"], "merge_count": osr["merge_count"],
        }


    _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
                 "precision", "recall",
                 "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")


    def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
```

```
87          """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
88
89          Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
90          leave-one-video-out discipline (windows within a video are correlated)."""
91          out: Dict[str, Any] = {"n": len(rows)}
92          for k in _AGG_KEYS:
93              vals = [r[k] for r in rows]
94              out[k] = eval_stats.mean_with_ci(vals) if vals else None
95          return out
96
97
98      # ---------------------------------------------------------------------- #
99      # Golden corpus I/O
100     # ---------------------------------------------------------------------- #
101     def load_golden(manifest_path: str = DEFAULT_MANIFEST,
102                     features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
103         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
104         intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
105         with open(manifest_path) as fh:
106             manifest = json.load(fh)
107         out: List[Dict[str, Any]] = []
108         for v in manifest:
109             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
110             gts: List[Interval] = []
111             if os.path.exists(feat):
112                 with open(feat) as fh:
113                     gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
114             out.append({"name": v["name"], "trajectory": v["trajectory"],
115                         "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
"gts": gts})
116         return out
117
118
119     def windows_for(video: Dict[str, Any], preset: WindowingPreset,
120                     min_crossings: int = 0) -> List[Interval]:
121         """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
122
123         ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
124         ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
125         times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
126         from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
127         windowing (W1) leaves behind.
128
129         NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
130         drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
131         strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
132         """
133         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
134         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
135         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
136         ivs = windowing.windows_to_intervals(wins)
137         if min_crossings > 0 and ivs:
138             from backend.pipeline.detectors.rally_gate import gate_windows
```

```
139             gated = [(g.start, g.end)
140                      for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
141                      if g.kept]
142             # Gating emptied a non-empty video ? keep the ungated windows.
143             ivs = gated if gated else ivs
144         return ivs
145
146
147     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
148                  iou: float = DEFAULT_IOU, min_crossings: int = 0,
149                  tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
150         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
151         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
152         rows: List[Dict[str, Any]] = []
153         for v in golden:
154             if not v["gts"] or not os.path.exists(v["trajectory"]):
155                 continue
156             preds = windows_for(v, preset, min_crossings)
157             row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
158             row["name"] = v["name"]
159             rows.append(row)
160         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161                 "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
166                 cand_min_crossings: int = 0,
167                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168                 guard_weight_merge: Optional[float] = None,
169                 guard_weight_split: Optional[float] = None,
170                 guard_use_rates: bool = True) -> Dict[str, Any]:
171         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
172         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
173         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
174         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
175         alongside so the refinement is auditable."""
176         from backend.eval import tolerance_metrics as tm
177         b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end)
178         c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end)
179         by_name_b = {r["name"]: r for r in b["rows"]}
180         paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
181         delta: Dict[str, Any] = {}
182         for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
183             delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
184                                                   [rc[k] for _, rc in paired])
185         gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
186         if guard_weight_merge is not None:
187             gkw["weight_merge"] = guard_weight_merge
188         if guard_weight_split is not None:
189             gkw["weight_split"] = guard_weight_split
190         guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
191         return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}
192
193
```

```
    # ---------------------------------------------------------------------- #
    # Reporting
    # ---------------------------------------------------------------------- #
    def _ci(m: Optional[Dict[str, Any]]) -> str:
        return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"


    def _format_guard(g: Dict[str, Any]) -> str:
        """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
        basis = "rate" if g["use_rates"] else "count"
        lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
                 f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
                 f"  NET verdict: {'PASS' if g['passes'] else 'BLOCK'}  "
                 f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
{g['net_delta']:+.3f}; "
                 f"no_merge_rate_regress={g['no_merge_rate_regress']})",
                 f"  strict per-video rule (R2 §2.3.3, for contrast): "
                 f"{'pass' if g['strict_passes'] else 'BLOCK'}  "
                 f"(count increases ? merges:{len(g['new_merges'])}
splits:{len(g['new_splits'])})"]
        if g["merge_rate_regress"]:
            names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
                              for r in g["merge_rate_regress"])
            lines.append(f"  ? merge_rate REGRESSED on: {names}")
        return "\n".join(lines)


    def format_report(result: Dict[str, Any]) -> str:
        p = result["preset"]
        bounded = []
        if p.get("max_window_duration"):
            bounded.append(f"cap={p['max_window_duration']}" + ("(hard)" if
p.get("hard_cap") else ""))
        if p.get("inactivity_end"):

bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}"
                           f"/md={p['inactivity_min_density']}")
        if p.get("blob_fallback"):
            bounded.append("blob_fallback")
        extra = (", " + ", ".join(bounded)) if bounded else ""
        lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
                 f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
                 f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}", ""]
        lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
        lines.append("|---|---|---|---|---|---|---|---|")
        for r in result["rows"]:
            lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | "
                         f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
                         f"{r['num_pred']}/{r['num_gt']} |")
        a = result["aggregate"]
        lines += ["", f"**Aggregate (n={a['n']}, mean [95% CI]):** "
                  f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5
{_ci(a['f1@0.5'])} · "
                  f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
{_ci(a['segment_count_ratio'])}"]
        # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
        rows = result["rows"]
        if rows and "tol_f1" in rows[0]:
            ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
            lines += ["", f"## R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict
```

```
1:1)", "",
                      "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
                      "|---|---|---|---|---|---|---|---|---|"]
            for r in rows:
                lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| "
                             f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
                             f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
{int(r['merge_count'])} |")
            lines += ["", f"**R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} · "
                      f"tolP {_ci(a['tol_precision'])} · tolR {_ci(a['tol_recall'])} · "
                      f"|?start| {_ci(a['dstart_abs_median'])} · |?end|
{_ci(a['dend_abs_median'])} "
                      f"(start?solved / end=the gap)"]
        return "\n".join(lines)


    def _resolve_preset(name: str) -> WindowingPreset:
        if name in PRESETS:
            return PRESETS[name]
        raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")


    def main() -> None:
        ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
(offline).")
        ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
        ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
        ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
served)")
        ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
        ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
        ap.add_argument("--min-crossings", type=int, default=0,
                        help="net-crossings rally-state gate on --preset (0=off; W2
Gate-A)")
        ap.add_argument("--vs-min-crossings", type=int, default=0,
                        help="net-crossings gate on --vs (default: same as --min-
crossings)")
        ap.add_argument("--max-window-duration", type=float, default=None,
                        help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
12)")
        ap.add_argument("--vs-max-window-duration", type=float, default=None,
                        help="cap (s) on --vs ONLY (default: same as --max-window-duration);
lets a "
                             "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
one shot")
        ap.add_argument("--hard-cap", action="store_true",
                        help="enforce the cap as a HARD cap on --preset (chop continuously-
active "
                             "windows at the cap when no inactivity gap exists; W1 hard-cap
fallback)")
        ap.add_argument("--vs-hard-cap", action="store_true",
                        help="hard-cap fallback on --vs (default: same as --hard-cap)")
        ap.add_argument("--blob-fallback", action="store_true",
                        help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
                             "cap at its midpoint + flag/log it; the continuously-active
blob last resort)")
        ap.add_argument("--vs-blob-fallback", action="store_true",
                        help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
        ap.add_argument("--blob-factor", type=float, default=None,
                        help="R5 magnitude gate (both presets): chop only groups > this ×
cap (default 4.0)")
        ap.add_argument("--inactivity-end", type=float, default=None,
                        help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
```

```
post-rally drift tail)")
297         ap.add_argument("--vs-inactivity-end", type=float, default=None,
298                         help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
299         ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
300                         help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
301         ap.add_argument("--inactivity-min-density", type=float, default=None,
302                         help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
303         ap.add_argument("--tau-start", type=float, default=None,
304                         help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
305         ap.add_argument("--tau-end", type=float, default=None,
306                         help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
307         ap.add_argument("--guard-weight-merge", type=float, default=None,
308                         help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
309         ap.add_argument("--guard-weight-split", type=float, default=None,
310                         help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
311         ap.add_argument("--guard-counts", action="store_true",
312                         help="R1 over-seg guard scores raw counts instead of normalized
rates "
313                              "(rates are the default ? comparable across a structural
mega?bounded change)")
314         ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
315         args = ap.parse_args()
316
317         from dataclasses import replace
318         golden = load_golden(args.manifest, args.features_dir)
319         def _apply_inactivity(preset, end_val):
320             if end_val is not None:
321                 preset = replace(preset, inactivity_end=end_val)
322             if args.inactivity_rally_thresh is not None:
323                 preset = replace(preset,
inactivity_rally_thresh=args.inactivity_rally_thresh)
324             if args.inactivity_min_density is not None:
325                 preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
326             return preset
327
328         base = _resolve_preset(args.preset)
329         if args.max_window_duration is not None:
330             base = replace(base, max_window_duration=args.max_window_duration)
331         if args.hard_cap:
332             base = replace(base, hard_cap=True)
333         if args.blob_fallback:
334             base = replace(base, blob_fallback=True)
335         if args.blob_factor is not None:
336             base = replace(base, blob_factor=args.blob_factor)
337         base = _apply_inactivity(base, args.inactivity_end)
338         if args.vs:
339             vs_mc = args.vs_min_crossings or args.min_crossings
340             cand = _resolve_preset(args.vs)
341             cand_cap = (args.vs_max_window_duration if args.vs_max_window_duration is not
None
342                         else args.max_window_duration)
343             if cand_cap is not None:
344                 cand = replace(cand, max_window_duration=cand_cap)
345             if args.vs_hard_cap or args.hard_cap:
346                 cand = replace(cand, hard_cap=True)
347             if args.vs_blob_fallback or args.blob_fallback:
348                 cand = replace(cand, blob_fallback=True)
```

```
349             if args.blob_factor is not None:
350                 cand = replace(cand, blob_factor=args.blob_factor)
351             cand = _apply_inactivity(cand, args.vs_inactivity_end if args.vs_inactivity_end
is not None
352                                      else args.inactivity_end)
353             result = compare(golden, base, cand, args.iou,
354                              base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
355                              tau_start=args.tau_start, tau_end=args.tau_end,
356                              guard_weight_merge=args.guard_weight_merge,
357                              guard_weight_split=args.guard_weight_split,
358                              guard_use_rates=not args.guard_counts)
359             print(format_report(result["base"]))
360             print("\n" + format_report(result["cand"]))
361             d = result["delta"]
362             print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
363                   f"n_paired={result['n_paired']}")
364             for k, dd in d.items():
365                 st = dd["sign_test"]
366                 print(f"  ?{k}: {dd['mean_delta']:+.3f}
[{dd['ci_low']:+.3f},{dd['ci_high']:+.3f}] "
367                       f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}")
368             print(_format_guard(result["over_seg_guard"]))
369         else:
370             result = evaluate(golden, base, args.iou, args.min_crossings,
371                               tau_start=args.tau_start, tau_end=args.tau_end)
372             print(format_report(result))
373         if args.json_out:
374             with open(args.json_out, "w", encoding="utf-8") as fh:
375                 json.dump(result, fh, indent=2)
376             print(f"\n[written {args.json_out}]")
377
378
379     if __name__ == "__main__":
380         main()
381
```

31. assistant (2026-06-18T01:34:14.115Z)

Now let me look carefully at a few things I've flagged. First, let me re-examine the `_points_in` function in rally_gate.py. The window selection is `start * fps <= p.frame <= end * fps`. But in `trajectory_to_action_windows`, the window start/end are computed as `group[0][0] / fps + chunk_start`. So when `chunk_start=0`, `start = frame/fps`, and `start * fps = frame`. That round-trips fine.

Let me look at the `_percentile` in tolerance_metrics and `_spread` in rally_gate for off-by-one edge cases, and also examine the experiment.py and calibrate_wasb. Let me also check the `bootstrap_ci` boundary when `confidence` makes `int(ceil(0.975*last))` potentially equal to last.

Let me look at experiment.py and the remaining calibration/calibrate_wasb files.

32. user (2026-06-18T01:34:15.436Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8
9         - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
```

```
11        - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
12        - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
13        - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
14        - `backend.eval.calibration`    ? `pct_improvement`
15
16      The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17      (per-frame signals ? run once on the GPU/WSL box, persisted into
18      `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19      signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20      given persisted candidate features it produces `List[Interval]` deterministically,
21      with no GPU and zero production risk. So most experiments are pure offline
22      re-scoring of stored data and never touch the served pipeline.
23
24      Three modes (`EXPERIMENT_HARNESS.md` §5):
25        - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
26          aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27          held-out loop, but variant-parameterised.
28        - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29          variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30          human spot-checks (and what two highlight reels would show side by side).
31        - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32          (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34      Pure + offline + unit-tested. The only DB-touching helper is
35      `load_video_candidates`, which reads persisted candidates via
36      `Database.query_candidate_segments`.
37      """
38      from __future__ import annotations
39
40      import json
41      import logging
42      import os
43      from dataclasses import dataclass, field
44      from datetime import datetime, timezone
45      from hashlib import sha1
46      from typing import Any, Callable, Dict, List, Optional, Sequence
47
48      from backend.eval.calibration import pct_improvement
49      from backend.eval.classifier import LogisticRegression
50      from backend.eval.gemini_refine import merge_overlaps
51      from backend.eval.metrics import Interval, match_intervals, score
52      from backend.eval.regression import gt_hash
53      from backend.eval.training import label_candidates
54      from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
55      from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
56      from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores
57
58      logger = logging.getLogger(__name__)
59
60      # Where a comparison run appends one reproducible record. Tracked location (NOT under
61      # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62      # regression.DEFAULT_BASELINE_PATH.
63      DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64      _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67      class ExperimentError(ValueError):
68          """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69          score an unlabeled video with no pinned model, or a gate referencing an absent
70          feature)."""
71
72
73      # ---------------------------------------------------------------------- Variant
```

```
74
75
76      @dataclass
77      class Variant:
78          """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80          A variant turns one video's persisted candidate windows + features into rally
81          intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82          filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83          to control ? the core no-regression guarantee.
84
85          Fields:
86            - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87              for the leaderboard and for selecting the served path (the existing
88              `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89              New cues are recorded here default-OFF.
90            - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91              ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92              `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93              windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94            - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95              whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96              cue persists that feature).
97            - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98              `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99              required for `compare_unlabeled` on a learned variant.
100           - ``feature_names`` ? the persisted features the learned filter consumes. When set
101             WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102             (honest) ? the recipe, not a pinned artifact.
103           - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104             overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105         """
106
107         name: str
108         segmenter: str = "wasb_hybrid"
109         config_overrides: Dict[str, Any] = field(default_factory=dict)
110         cue_flags: Dict[str, bool] = field(default_factory=dict)
111         min_crossings: int = 0
112         min_players: int = 0
113         classifier_model: Optional[str] = None
114         feature_names: Optional[List[str]] = None
115         threshold: float = 0.5
116         merge_gap: float = 1.0
117         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119         # failure where a fixed 0.5 zeroed out a small-candidate video.
120         tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121         calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123         def is_learned(self) -> bool:
124             """True if this variant applies a learned classifier (pinned model or
recipe)."""
125             return self.classifier_model is not None or bool(self.feature_names)
126
127         def to_dict(self) -> dict:
128             return {
129                 "name": self.name,
130                 "segmenter": self.segmenter,
131                 "config_overrides": self.config_overrides,
132                 "cue_flags": self.cue_flags,
133                 "min_crossings": self.min_crossings,
```

```
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
```

```
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
```

```
257                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
258             if variant.calibrate == "platt":
259                 calibrator = fit_platt(raw, y)
260             elif variant.calibrate == "isotonic":
261                 calibrator = fit_isotonic(raw, y)
262             else:
263                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
264         threshold: Optional[float] = None
265         if variant.tune_threshold:
266             best_t, best_f1 = variant.threshold, -1.0
267             for k in range(1, 20):                      # grid 0.05..0.95 on the train
fold's rally F1
268                 t = round(0.05 * k, 2)
269                 f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                              vc.gts or [], iou).f1 for vc in train_videos]
271                 mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272                 if mean_f1 > best_f1:
273                     best_f1, best_t = mean_f1, t
274             threshold = best_t
275         return calibrator, threshold
276
277
278     # ------------------------------------------------------------------- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                          iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302           - **static variant** (no learned filter): predict each video directly ? no
training,
303             so no leakage; per-video scoring is already honest.
304           - **learned, pinned model** (``classifier_model`` set): score every video with the
305             SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306             report "how the shipped artifact does here", not for the promotion decision.
307           - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
308             train on the OTHER videos, predict the held-out one. The number to trust.
309           - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
```

```
                raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")

        per_video: List[Dict[str, Any]] = []
        predictions: Dict[str, List[Interval]] = {}

        recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
        pinned_model = (LogisticRegression.load(variant.classifier_model)
                        if variant.classifier_model is not None else None)
        all_model = None
        if recipe_lovo and not honest:
            all_model = _train(variant, videos, iou)

        for i, vc in enumerate(videos):
            cal: Optional[Callable[[float], float]] = None
            thr: Optional[float] = None
            if recipe_lovo and honest:
                others = [v for j, v in enumerate(videos) if j != i]
                if not others:
                    raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
                model: Optional[LogisticRegression] = _train(variant, others, iou)
                if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
                    assert model is not None                       # _train never returns
None
                    cal, thr = _calibrate_and_tune(variant, model, others, iou)
            elif recipe_lovo:                       # honest=False ? one model over all
videos
                model = all_model
            else:                                   # static variant or pinned model
                model = pinned_model
            preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
            assert vc.gts is not None               # guarded at function top
            sc = score(preds, vc.gts, iou)
            per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
            predictions[vc.name] = preds

        aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
                     for m in _METRICS}
        return {
            "variant": variant.name,
            "spec_hash": variant.spec_hash(),
            "is_learned": variant.is_learned(),
            "honest_lovo": recipe_lovo and honest,
            "per_video": per_video,
            "aggregate": aggregate,
            "predictions": predictions,
        }


    def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
        """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
```

```
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out


    def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
        """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).

        ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
        so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
        the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
        """
        a_f1 = [p["f1"] for p in eval_a["per_video"]]
        b_f1 = [p["f1"] for p in eval_b["per_video"]]
        return {
            "variant_a_ci": _ci_block(eval_a),
            "variant_b_ci": _ci_block(eval_b),
            "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
            "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                             "variant_b": _segmentation_block(videos, eval_b)},
        }


    def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
        """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
        golden corpus and reports the **B ? A** deltas, per video and in aggregate.

        The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
        learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
        `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.

        ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
        paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
        existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
        shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
        bare LOVO mean at n?6 is not trustworthy on its own.
        """
        if len(videos) < 1:
```

```
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422 
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425 
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431 
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435 
436         result: Dict[str, Any] = {
437             "iou": iou,
438             "label_set_hash": gt_hash(all_gts),
439             "num_videos": len(videos),
440             "variant_a": eval_a,
441             "variant_b": eval_b,
442             "delta": delta,
443             "delta_pct": delta_pct,
444             "per_video_delta": per_video_delta,
445         }
446         if honest_stats:
447             result["honest"] = honest_summary(videos, eval_a, eval_b)
448         return result
449 
450 
451     # ------------------------------------------------------------------ SxS on unlabeled video
452 
453 
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                           agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457 
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461 
462           - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
463           - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
464           - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
465             the human / a later golden label adjudicates).
466 
467         Both variants must be predictable without labels: a learned variant therefore needs
a
468         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469         `metrics.match_intervals` ? no new matching logic.
470         """
471         preds_a = predict(variant_a, video)
472         preds_b = predict(variant_b, video)
473         mr = match_intervals(preds_a, preds_b, agree_iou)
474 
475         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
476                   for m in mr.matches]
477         agree_ious = [m.iou for m in mr.matches]
478         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
479         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
```

```
480
481         def _dur(ivs: List[Interval]) -> float:
482             return sum(e - s for s, e in ivs)
483
484         return {
485             "video": video.name,
486             "agree_iou": agree_iou,
487             "variant_a": variant_a.name,
488             "variant_b": variant_b.name,
489             "num_a": len(preds_a),
490             "num_b": len(preds_b),
491             "num_agreed": len(agreed),
492             "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
493             "duration_a": _dur(preds_a),
494             "duration_b": _dur(preds_b),
495             "agreed": agreed,
496             "a_only": a_only,
497             "b_only": b_only,
498         }
499
500
501     # ---------------------------------------------------------------------- leaderboard / DB
502
503
504     def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                           timestamp: Optional[str] = None) -> Dict[str, Any]:
506         """Append a compact, reproducible record of a `compare()` result to the JSON
507         leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508         `regression_baseline.json`; deliberately the size of a baseline file, not a service
509         (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510         """
511         row: Dict[str, Any] = {
512             "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513             "iou": result.get("iou"),
514             "label_set_hash": result.get("label_set_hash"),
515             "num_videos": result.get("num_videos"),
516             "variant_a": {"name": result["variant_a"]["variant"],
517                           "spec_hash": result["variant_a"]["spec_hash"],
518                           "aggregate": result["variant_a"]["aggregate"]},
519             "variant_b": {"name": result["variant_b"]["variant"],
520                           "spec_hash": result["variant_b"]["spec_hash"],
521                           "aggregate": result["variant_b"]["aggregate"]},
522             "delta": result.get("delta"),
523         }
524         # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
525         # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
526         honest = result.get("honest") or {}
527         if honest.get("paired_delta_f1"):
528             row["honest_delta_f1"] = honest["paired_delta_f1"]
529         entries: List[Dict[str, Any]] = []
530         if os.path.isfile(path):
531             with open(path, encoding="utf-8") as f:
532                 entries = json.load(f)
533         entries.append(row)
534         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535         with open(path, "w", encoding="utf-8") as f:
536             json.dump(entries, f, indent=2)
537         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                     row["variant_a"]["name"], row["variant_b"]["name"],
539                     (result.get("delta") or {}).get("f1", 0.0), path)
540         return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
```

```
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.
547
548         ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         """
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

33. assistant (2026-06-18T01:34:18.913Z)

Now let me focus on examining the bootstrap_ci CI bounds math carefully against the known issue in `paired_delta_ci`, and look at the `merge_overlaps`, `label_candidates`, and `gemini_refine` functions for boundary conditions. Let me also check `score_calibration.py` (Platt/isotonic) which is statistical.

34. user (2026-06-18T01:34:20.336Z)

```
1       """Per-cue score calibration (course-correction C2) ? make ``confidence`` mean
something.
2
3       A CNN softmax, an audio onset score, and an integer net-crossing count live on wildly
4       different scales. Fed raw into the learned arbiter (the ADR-004 logistic regression),
the
5       most over-confident cue dominates and starves complementary cues. Calibrating each
6       producer's score to a probability *before* it becomes a feature column makes the
7       ``(value, confidence)`` semantics real and the arbiter's weights stable. See
8       ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C2.
9
10      Two calibrators, both standard:
11      - **Platt scaling** ? fit ``p = sigmoid(a*s + b)`` (parametric; good for monotone,
12        roughly-sigmoidal cues; uses Platt's target smoothing to avoid 0/1 overfitting).
13      - **Isotonic regression** (pool-adjacent-violators) ? non-parametric monotone fit; more
14        flexible, needs a bit more data.
15
16      Plus ``brier_score`` and ``expected_calibration_error`` to *measure* calibration. All
17      **additive**, pure (stdlib only), deterministic ? fits run in the numpy-free lanes too.
18      Calibrators are tiny dataclasses (their params are JSON-friendly), so a fitted
calibrator
19      can be pinned alongside a variant the way the classifier is.
20      """
21      from __future__ import annotations
22
23      import bisect
24      import math
25      from dataclasses import dataclass
```

```
    from typing import List, Sequence, Tuple

    _EPS = 1e-12


    def _sigmoid(z: float) -> float:
        if z >= 0:
            return 1.0 / (1.0 + math.exp(-z))
        e = math.exp(z)
        return e / (1.0 + e)


    def _clip01(p: float) -> float:
        return min(1.0 - _EPS, max(_EPS, p))


    @dataclass(frozen=True)
    class PlattCalibrator:
        """``p = sigmoid(a*score + b)``. Call it on a raw cue score to get a probability."""
        a: float
        b: float

        def __call__(self, s: float) -> float:
            return _clip01(_sigmoid(self.a * s + self.b))


    @dataclass(frozen=True)
    class IsotonicCalibrator:
        """Piecewise-constant monotone map from raw score to probability (PAVA fit).

        ``xs`` are sorted training scores, ``ys`` the non-decreasing fitted probabilities;
        a query returns the fitted value of the largest ``x <= score`` (clipped at the
ends).
        """
        xs: List[float]
        ys: List[float]

        def __call__(self, s: float) -> float:
            if not self.xs:
                return 0.5
            idx = bisect.bisect_right(self.xs, s) - 1
            if idx < 0:
                idx = 0
            return self.ys[idx]


    def fit_platt(scores: Sequence[float], labels: Sequence[int],
                  iters: int = 200, lr: float = 0.5) -> PlattCalibrator:
        """Fit Platt scaling by gradient descent on (smoothed) log-loss.

        Scores are standardised internally for a stable fit, then folded back so the
returned
        ``a``/``b`` apply to the **raw** score. Platt target smoothing
        (``t+ = (N+ + 1)/(N+ + 2)``, ``t- = 1/(N- + 2)``) keeps the fit off the 0/1 rails.
        Degenerate inputs (empty, zero-variance, single class) fall back to a sane constant
map.
        """
        n = len(scores)
        if n == 0 or n != len(labels):
            return PlattCalibrator(a=0.0, b=0.0)            # ? p = 0.5 everywhere
        mean = sum(scores) / n
        var = sum((s - mean) ** 2 for s in scores) / n
        sd = math.sqrt(var)
        n_pos = sum(1 for y in labels if y >= 0.5)
        n_neg = n - n_pos
```

```
        if sd <= _EPS or n_pos == 0 or n_neg == 0:
            # No usable score spread or one class: best constant probability = base rate.
            base = _clip01(n_pos / n)
            return PlattCalibrator(a=0.0, b=math.log(base / (1.0 - base)))
        t_pos = (n_pos + 1.0) / (n_pos + 2.0)
        t_neg = 1.0 / (n_neg + 2.0)
        z = [(s - mean) / sd for s in scores]
        targets = [t_pos if y >= 0.5 else t_neg for y in labels]

        a, b = 1.0, 0.0                                  # in standardised space
        for _ in range(max(1, iters)):
            ga = gb = 0.0
            for zi, t in zip(z, targets):
                p = _sigmoid(a * zi + b)
                d = p - t
                ga += d * zi
                gb += d
            a -= lr * ga / n
            b -= lr * gb / n
        # Fold standardisation back to raw score: z = (s-mean)/sd ? a*z+b = (a/sd)*s + (b -
a*mean/sd).
        return PlattCalibrator(a=a / sd, b=b - a * mean / sd)


    def _pava(ys: Sequence[float]) -> List[float]:
        """Pool-adjacent-violators: nearest non-decreasing fit to ``ys`` (in given
order)."""
        blocks: List[List[float]] = []                   # each [value, weight]
        for y in ys:
            blocks.append([float(y), 1.0])
            while len(blocks) >= 2 and blocks[-2][0] > blocks[-1][0]:
                v2, w2 = blocks.pop()
                v1, w1 = blocks.pop()
                blocks.append([(v1 * w1 + v2 * w2) / (w1 + w2), w1 + w2])
        fitted: List[float] = []
        for value, weight in blocks:
            fitted.extend([value] * int(round(weight)))
        return fitted


    def fit_isotonic(scores: Sequence[float], labels: Sequence[int]) -> IsotonicCalibrator:
        """Fit isotonic (monotone) calibration via PAVA on score-sorted labels."""
        n = len(scores)
        if n == 0 or n != len(labels):
            return IsotonicCalibrator(xs=[], ys=[])
        order = sorted(range(n), key=lambda i: (scores[i], i))
        xs = [float(scores[i]) for i in order]
        ys = [_clip01(v) for v in _pava([float(labels[i]) for i in order])]
        return IsotonicCalibrator(xs=xs, ys=ys)


    def brier_score(probs: Sequence[float], labels: Sequence[int]) -> float:
        """Mean squared error of probabilities vs 0/1 labels (lower = better calibrated)."""
        n = len(probs)
        if n == 0 or n != len(labels):
            return 0.0
        return sum((p - y) ** 2 for p, y in zip(probs, labels)) / n


    def expected_calibration_error(probs: Sequence[float], labels: Sequence[int],
                                   n_bins: int = 10) -> float:
        """Expected calibration error: |accuracy ? confidence| per equal-width bin,
weighted."""
        n = len(probs)
        if n == 0 or n != len(labels):
```

```
            return 0.0
        bins = max(1, n_bins)
        bin_sum_p = [0.0] * bins
        bin_sum_y = [0.0] * bins
        bin_cnt = [0] * bins
        for p, y in zip(probs, labels):
            b = min(bins - 1, int(p * bins))
            bin_sum_p[b] += p
            bin_sum_y[b] += y
            bin_cnt[b] += 1
        ece = 0.0
        for i in range(bins):
            if bin_cnt[i]:
                conf = bin_sum_p[i] / bin_cnt[i]
                acc = bin_sum_y[i] / bin_cnt[i]
                ece += (bin_cnt[i] / n) * abs(acc - conf)
        return ece


    def calibration_report(probs: Sequence[float], labels: Sequence[int],
                           n_bins: int = 10) -> Tuple[float, float]:
        """Convenience: ``(brier, ece)`` for a set of probabilities vs labels."""
        return brier_score(probs, labels), expected_calibration_error(probs, labels, n_bins)

```

35. user (2026-06-18T01:34:20.789Z)

```
      """Build a training set for the offline, API-free learned rally segmenter.

      Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
      a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
      produces candidate rally windows with features; the collector's ground-truth
      rally intervals tell us which windows are real. A lightweight classifier learns
      that mapping, replacing the paid Gemini refinement with a local model.

      This module is the *data layer* ? pure and unit-tested:

        - `label_candidates`  ? generic: label windows positive/negative by whether they
          match a ground-truth rally (same IoU matching the evaluator uses, so training
          labels are consistent with how we score).
        - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
          persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
        - `build_training_set` ? glue: rows + GT -> (X, y, intervals).

      No model training or I/O here; that lives in the trainer/segmenter once the
      substrate is chosen from the Phase 3 candidate-recall result.
      """

      from typing import Callable, Dict, List, Tuple

      from backend.eval.metrics import Interval, match_intervals

      # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
      # (see HybridYoloSegmenter). Duration is derived from the window bounds.
      YOLO_FEATURE_NAMES = [
          "duration", "peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count",
      ]


      def extract_yolo_features(row: Dict) -> List[float]:
          """Feature vector for one YOLO candidate row (dict from query_candidate_segments).

          `stats` is already JSON-deserialized by the DB layer. Missing fields default
          to 0.0 so a sparse/old row still yields a well-formed vector.
```

```
        """
        stats = row.get("stats") or {}
        duration = float(row["end_time"]) - float(row["start_time"])
        return [
            duration,
            float(stats.get("peak_velocity", 0.0)),
            float(stats.get("mean_velocity", 0.0)),
            float(stats.get("active_frame_count", 0)),
            float(stats.get("track_id_count", 0)),
        ]


    def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
                         iou_threshold: float = 0.5) -> List[int]:
        """Label each candidate window 1 (real rally) or 0 (spurious).

        A window is positive iff it is matched one-to-one to a ground-truth rally at
        `iou_threshold` ? the *same* matching the evaluator uses for true positives,
        so the classifier is trained on exactly what we score. Substrate-agnostic:
        works for YOLO or WASB-derived windows alike.
        """
        mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
        positive = {m.pred_index for m in mr.matches}
        return [1 if i in positive else 0 for i in range(len(candidate_intervals))]


    def build_training_set(
        candidate_rows: List[Dict],
        gt_intervals: List[Interval],
        iou_threshold: float = 0.5,
        feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
    ) -> Tuple[List[List[float]], List[int], List[Interval]]:
        """Turn one video's candidate rows + ground truth into (X, y, intervals).

        `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
        features without changing the labeling logic. Returns the feature matrix `X`,
        binary labels `y`, and the candidate intervals (handy for downstream merging).
        """
        intervals: List[Interval] = [
            (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
        ]
        y = label_candidates(intervals, gt_intervals, iou_threshold)
        X = [feature_fn(r) for r in candidate_rows]
        return X, y, intervals

```

36. assistant (2026-06-18T01:34:23.939Z)

Let me examine `merge_overlaps` in gemini_refine.py and the regression.py `gt_hash`, plus `eval_partial_labels` flow already reviewed. Let me also check calibrate_wasb.py `load_golden_gts`.

37. user (2026-06-18T01:34:25.315Z)

```
    """Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).

    WASB (shuttle-only) over-fires on between-point service feeds. This drives the
    EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
    each candidate window is cut to a small padded clip and handed to Gemini, which ?
    seeing the actual footage ? returns the true rally inside it (tight boundaries) or
    nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
    not GB-scale chunks; high-precision because Gemini judges each candidate visually.

    Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
    sports.badminton prompt (sport_registry), utils.video.extract_video_chunk,
```

```
    calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.

    This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
    trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.

    Run (GEMINI_API_KEY in env):
        python -m backend.eval.gemini_refine --video Adarsh.MP4 --trajectory adarsh_traj.csv
\
            --fps 59.94 --frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
    """
    from __future__ import annotations

    import os
    import os.path as osp
    import time
    import argparse
    import concurrent.futures
    from typing import List, Tuple, Optional

    from backend.eval.calibrate_wasb import make_predict_fn, BASELINE
    from backend.eval.export_wasb_segments import TUNED, write_segments_csv, seconds_to_mmss
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry

    Interval = Tuple[float, float]


    def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
        """Union of overlapping intervals, optionally stitching across a small gap.

        gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal
brief
        shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally
gaps
        (several seconds) as separate rallies. (Replaces the old envelope-per-candidate
logic,
        which over-merged multiple real rallies WASB had bundled into one candidate.)"""
        merged: List[Interval] = []
        for s, e in sorted(intervals):
            if merged and s <= merged[-1][1] + gap_tol:
                merged[-1] = (merged[-1][0], max(merged[-1][1], e))
            else:
                merged.append((s, e))
        return merged


    def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
        """Map a Gemini clip's returned segments back to full-video seconds, clamped to
clip."""
        out: List[Interval] = []
        for sg in raw or []:
            try:
                st = float(sg["start_time"]) + clip_start
                en = float(sg["end_time"]) + clip_start
            except (TypeError, ValueError, KeyError):
                continue
            st = max(clip_start, st)
            en = min(clip_end, en)
            if en > st:
                out.append((round(st, 3), round(en, 3)))
        return out


    def refine_window(api_key: str, model: str, sport_profile, video: str, window: Interval,
```

```
                      pad: float, duration: float, tmpdir: str, idx: int,
                      clip_scale_width: Optional[int] = 1280) -> Tuple[int, Interval,
List[Interval]]:
        # Own provider/client per call: the genai client isn't reliably thread-safe, and a
        # shared one across workers throws socket errors. A fresh lazy client is cheap.
        provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
        s, e = window
        cs = max(0.0, s - pad)
        ce = min(duration, e + pad) if duration > 0 else e + pad
        clip = osp.join(tmpdir, f"cand_{idx:04d}.mp4")
        # Downscale the uploaded clip (Gemini analyses at low res) -> small, fast, under the
        # 500MB upload cap even for 4K/5K sources.
        if not extract_video_chunk(video, cs, ce - cs, clip, reencode=True,
scale_width=clip_scale_width):
            return idx, window, []
        try:
            prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
            raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs,
label=f"cand{idx}")
            mapped = _offset_segments(raw, cs, ce)
        finally:
            try:
                os.remove(clip)
            except OSError:
                pass
        # Keep Gemini's segments separate ? a WASB candidate can bundle several real rallies
        # (merge_gap bridged the between-point gaps), and collapsing them to one envelope
swept
        # dead time into a "rally" (precision loss). Tiny invisibility splits are healed
later
        # by the gap-tolerant global merge in run().
        return idx, window, mapped


    def run(video: str, trajectory: str, fps: float, frame_width: float,
            cfg=BASELINE, pad: float = 3.0, model: str = "gemini-flash-latest",
            max_workers: int = 4, out_csv: str = "output/gemini_refined.csv",
            min_dur: float = 2.0, limit: int = 0, clip_scale_width: Optional[int] = 1280) ->
dict:
        api_key = os.environ.get("GEMINI_API_KEY")
        if not api_key:
            raise SystemExit("GEMINI_API_KEY not set in env")
        sport_profile = sport_registry.get("badminton")
        duration = get_video_duration(video)

        candidates = make_predict_fn(trajectory, fps, frame_width)(cfg)
        if limit:
            candidates = candidates[:limit]
        tmpdir = osp.join("output", "gemini_refine_clips")
        os.makedirs(tmpdir, exist_ok=True)

        print(f"[gemini_refine] {len(candidates)} WASB candidates (cfg {cfg}) -> Gemini
{model}, "
              f"pad {pad}s, {max_workers} workers", flush=True)
        t0 = time.time()
        per_candidate = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
            futs = [ex.submit(refine_window, api_key, model, sport_profile, video, w, pad,
duration,
                              tmpdir, i, clip_scale_width)
                    for i, w in enumerate(candidates)]
            for fut in concurrent.futures.as_completed(futs):
                idx, window, mapped = fut.result()
                per_candidate.append((idx, window, mapped))
                tag = "REJECT (no rally)" if not mapped else \
```

```
                      " ".join(f"{seconds_to_mmss(a)}-{seconds_to_mmss(b)}" for a, b in
mapped)
                print(f"  cand {idx:>2}
{seconds_to_mmss(window[0])}-{seconds_to_mmss(window[1])} -> {tag}", flush=True)

        raw_intervals = [iv for _, _, m in per_candidate for iv in m]
        refined = [(s, e) for s, e in merge_overlaps(raw_intervals, gap_tol=1.0) if e - s >=
min_dur]
        write_segments_csv(out_csv, refined)
        try:
            os.rmdir(tmpdir)
        except OSError:
            pass

        kept_cands = sum(1 for _, _, m in per_candidate if m)
        play = sum(e - s for s, e in refined)
        print(f"\n[gemini_refine] {kept_cands}/{len(candidates)} candidates had a rally; "
              f"{len(refined)} merged rallies, {play:.1f}s play -> {out_csv} "
              f"({time.time() - t0:.0f}s)", flush=True)
        return {"refined": refined, "num_candidates": len(candidates),
                "candidates_with_rally": kept_cands, "out_csv": out_csv}


    def full_video_rallies(video: str, model: str = "gemini-flash-latest",
                           out_csv: str = "output/gemini_fullvideo.csv", clip_scale_width:
Optional[int] = 1280,
                           min_dur: float = 2.0, chunk_sec: float = 120.0, overlap: float =
10.0,
                           max_workers: int = 3) -> List[Interval]:
        """Full-context pass: hand Gemini the WHOLE clip (downscaled, chunked to stay under
the
        upload cap), so it finds every rally itself ? not bounded by WASB's candidate
recall.
        Independent of the trajectory; complements refine()."""
        api_key = os.environ.get("GEMINI_API_KEY")
        if not api_key:
            raise SystemExit("GEMINI_API_KEY not set in env")
        sport_profile = sport_registry.get("badminton")
        duration = get_video_duration(video)
        starts, t = [], 0.0
        while t < duration:
            starts.append(t)
            t += max(1.0, chunk_sec - overlap)
        tmpdir = osp.join("output", "gemini_full_clips")
        os.makedirs(tmpdir, exist_ok=True)

        def do_chunk(ci: int, cstart: float) -> List[Interval]:
            cend = min(cstart + chunk_sec, duration)
            clip = osp.join(tmpdir, f"chunk_{ci:03d}.mp4")
            if not extract_video_chunk(video, cstart, cend - cstart, clip, reencode=True,
scale_width=clip_scale_width):
                return []
            try:
                provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
                prompt = sport_profile.get_prompt(chunk_duration=cend - cstart)
                raw, _ = provider.analyze_video(clip, prompt, chunk_duration=cend - cstart,
label=f"chunk{ci}")
                return _offset_segments(raw, cstart, cend)
            finally:
                try:
                    os.remove(clip)
                except OSError:
                    pass

```

```
184         print(f"[gemini_full] {len(starts)} chunk(s) of {video} (<= {chunk_sec}s, width
{clip_scale_width}) "
185               f"-> Gemini {model}", flush=True)
186         with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
187             chunks = [f.result() for f in [ex.submit(do_chunk, i, s) for i, s in
enumerate(starts)]]
188         rallies = [(s, e) for s, e in merge_overlaps([iv for c in chunks for iv in c],
gap_tol=1.0) if e - s >= min_dur]
189         write_segments_csv(out_csv, rallies)
190         try:
191             os.rmdir(tmpdir)
192         except OSError:
193             pass
194         play = sum(e - s for s, e in rallies)
195         print(f"[gemini_full] {len(rallies)} rallies, {play:.1f}s play -> {out_csv}",
flush=True)
196         return rallies
197
198
199     def main() -> None:
200         ap = argparse.ArgumentParser(description="Refine WASB candidates with Gemini.")
201         ap.add_argument("--video", required=True)
202         ap.add_argument("--trajectory", default=None, help="WASB trajectory CSV (required
unless --full-video)")
203         ap.add_argument("--fps", type=float, default=59.94)
204         ap.add_argument("--frame-width", type=float, default=1920.0)
205         ap.add_argument("--config", choices=["baseline", "tuned"], default="baseline",
206                         help="WASB candidate config to feed Gemini (baseline = whole
rallies, good recall)")
207         ap.add_argument("--pad", type=float, default=3.0)
208         ap.add_argument("--model", default="gemini-flash-latest")
209         ap.add_argument("--max-workers", type=int, default=4)
210         ap.add_argument("--min-dur", type=float, default=2.0)
211         ap.add_argument("--out", default="output/gemini_refined.csv")
212         ap.add_argument("--limit", type=int, default=0, help="only the first N candidates
(smoke test)")
213         ap.add_argument("--clip-scale-width", type=int, default=1280,
214                         help="downscale uploaded clips to this width (0=keep source res)")
215         ap.add_argument("--full-video", action="store_true",
216                         help="full-context pass: send the whole clip (chunked/downscaled) to
Gemini "
217                              "instead of refining WASB candidates (needs no --trajectory)")
218         args = ap.parse_args()
219         scale = args.clip_scale_width or None
220         if args.full_video:
221             full_video_rallies(args.video, model=args.model, out_csv=args.out,
222                                clip_scale_width=scale, min_dur=args.min_dur,
max_workers=args.max_workers)
223             return
224         if not args.trajectory:
225             raise SystemExit("--trajectory is required unless --full-video")
226         cfg = BASELINE if args.config == "baseline" else TUNED
227         run(args.video, args.trajectory, args.fps, args.frame_width, cfg=cfg, pad=args.pad,
228             model=args.model, max_workers=args.max_workers, out_csv=args.out,
229             min_dur=args.min_dur, limit=args.limit, clip_scale_width=scale)
230
231
232     if __name__ == "__main__":
233         main()
234
```

38. user (2026-06-18T01:34:25.825Z)

```
1       """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
```

```
2
3       Wires the already-tested pieces together:
4         WASB trajectory CSV (Frame,Visibility,X,Y)
5           -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
6           -> backend.eval.calibration (improvement_report / cross_validate)
7
8       Run (after a full-video WASB pass exists):
9           python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10              --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an **optimistic** upper bound. The trustworthy number is the
14      cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17      from __future__ import annotations
18
19      import argparse
20      from typing import List, Tuple, Callable
21
22      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
23      from backend.eval.calibration import (
24          select_best, improvement_report, cross_validate, evaluate,
25      )
26
27      Interval = Tuple[float, float]
28      Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap,
min_window_duration)
29
30      BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults
31
32
33      def load_golden_gts(csv_path: str) -> List[Interval]:
34          """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35
36          Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37          start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38          driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39          """
40          from backend.eval.gt_loader import load_gt_intervals
41          return load_gt_intervals(csv_path, strict=False)
42
43
44      def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45          """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
47
48          def predict_fn(cfg: Config) -> List[Interval]:
49              vt, mg, mwd = cfg
50              wins = TrackNetRunner.trajectory_to_action_windows(
51                  points, fps=fps, frame_width=frame_width,
52                  velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53              )
54              return [(w["start"], w["end"]) for w in wins]
55
56          return predict_fn
57
58
59      def default_grid() -> List[Config]:
60          return [(vt, mg, mwd)
61                  for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62                  for mg in (1.0, 2.0, 3.0)
63                  for mwd in (1.0, 2.0, 3.0)]
```

```
64
65
66      def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
67              iou: float = 0.5) -> dict:
68          gts = load_golden_gts(labels_csv)
69          if not gts:
70              raise SystemExit(f"no usable golden rallies in {labels_csv}")
71          predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
72          grid = default_grid()
73
74          base = evaluate(predict_fn(BASELINE), gts, iou)
75          best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
76          fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)   # OPTIMISTIC
(fit on full GT)
77          cv = cross_validate(predict_fn, grid, gts, k=5)                      # HONEST held-
out recall
78
79          print("=== WASB rally-segmentation calibration ===")
80          print(f"golden rallies: {len(gts)}  | IoU thresh: {iou}")
81          print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
82                f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
83          print(f"tuned cfg    {best_cfg}: f1={best_f1:.3f}")
84          print(f"  precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
85                f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
86          print(f"  f1        {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
87                f"({fit['f1_delta_pct']:+.1f}%)")
88          print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
89                f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
90                f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
91          print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
92          return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
93
94
95      def main():
96          ap = argparse.ArgumentParser()
97          ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
98          ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
99          ap.add_argument("--fps", type=float, default=59.94)
100         ap.add_argument("--frame-width", type=float, default=1920.0)
101         ap.add_argument("--iou", type=float, default=0.5)
102         args = ap.parse_args()
103         run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
104
105
106     if __name__ == "__main__":
107         main()
108
```

39. assistant (2026-06-18T01:34:39.446Z)

I've now reviewed the core numeric/statistical paths. Let me re-examine a couple of suspicious spots more carefully before concluding.

First, the `_offset_segments` in gemini_refine (lines 55-68): it computes `en = float(sg["end_time"]) + clip_start` ? note it adds `clip_start` to end_time, not `clip_end`. That's correct since Gemini returns relative-to-clip times and the clip starts at `clip_start`. Fine.

Now let me scrutinize the R4 `_trim_inactive_tail` sliding window more carefully ? this is the

kind of windowing logic where an off-by-one matters. And let me re-examine the `over_seg_guard` `_aligned` function: when names aren't present in all rows, it falls back to positional alignment, but `compare` always passes rows that have `name`. Let me verify the `merge_split_report` connected-components with `complex` classification edge: `gts_split` only counts `+1` per split component regardless of how many GTs ? but split is `(>1, 1)` so exactly 1 GT, fine. `split_rate = gts_split / m`. OK.

Let me look at the `_split_overlong_groups` hard_cap branch more carefully for the midpoint split. When `hard_cap` and a group can't be split by gap, it computes `mid` and `split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0]-mid))`. If `split_i` could be `len(g)` (the last index), then `g[split_i:]` would be empty and `g[:split_i]` would be the whole group ? infinite loop. Can `split_i == len(g)`? `range(1, len(g))` goes up to `len(g)-1`, so `split_i <= len(g)-1`. So `g[split_i:]` has at least 1 element and `g[:split_i]` has at least 1. Both non-empty. But could one piece still be the full length causing non-termination? If `split_i=1`, then `g[:1]` (length 1) and `g[1:]` (length len-1). The length-1 piece terminates; the other is strictly smaller. Progress guaranteed. Good.

Now let me check the blob_fallback similar logic ? same structure, `split_i` in `range(1, len(h))`, fine.

Let me look at the `bootstrap_ci` more carefully for a real edge case, and check the regression.py and batch.py files.

40. user (2026-06-18T01:34:40.922Z)

```
1       """Regression runner ? verify the production tool against golden-set ground truth.
2
3       Implements the explicit ask: *"given a video, get a golden set created and verify
4       its precision and recall against the production tool."* This is deliberately a thin
5       wrapper over machinery that already exists:
6
7           ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8           predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
9           comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
10          baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12      A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13      baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14      later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15      independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16      """
17      import os
18      import json
19      import hashlib
20      import logging
21      from dataclasses import dataclass, asdict
22      from typing import Dict, List, Optional
23
24      from backend.infrastructure.database import Database
25      from backend.eval import metrics
26      from backend.eval.harness import get_predictions
27      from backend.annotations import annotation_registry
28
29      logger = logging.getLogger(__name__)
30
31      # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32      # actually load a committed baseline to compare against.
33      DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34      DEFAULT_TOLERANCE = 0.05
35
36
37      def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38          """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39          norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
```

```
        return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]


    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]
        # True only when a baseline existed to compare against. Distinguishes a genuine PASS
        # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
        has_baseline: bool = False
        gt_hash: Optional[str] = None

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store

    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH,
                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
                      gt_fingerprint: Optional[str] = None) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
        later run computed under different settings (or against different labels) is
detected
        as incommensurable rather than silently compared.
        """
        baselines = load_baselines(path)
        baselines[_baseline_key(video_id, stage)] = {
            "video_id": video_id, "stage": stage,
            "precision": precision, "recall": recall, "f1": f1,
            "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
        }
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)


```

```
    # ------------------------------------------------------------------ the runner

    def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
                stage: str = "final", iou_threshold: float = 0.5,
                detector: Optional[str] = None,
                tolerance: float = DEFAULT_TOLERANCE,
                baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
        """Score stored predictions for one video against ground truth and compare to
baseline.

        `ground_truth` is a list of (start, end) intervals ? typically obtained from an
        AnnotationProvider (see `regress_with_provider`). A regression is flagged when
        precision OR recall falls more than `tolerance` below the snapshotted baseline.
        If no baseline exists yet, `regressed` is False (nothing to compare to) and the
        caller can choose to snapshot the current numbers via `save_baseline`.
        """
        preds = get_predictions(db, video_id, stage, detector=detector)
        s = metrics.score(preds, ground_truth, iou_threshold)
        fp = gt_hash(ground_truth)

        baselines = load_baselines(baseline_path)
        b = baselines.get(_baseline_key(video_id, stage))

        reasons: List[str] = []
        regressed = False
        has_baseline = b is not None
        base_p = base_r = None
        if b is not None:
            base_p, base_r = b["precision"], b["recall"]
            # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
            # against different labels is not a valid reference ? flag it instead of
trusting it.
            b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
            if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
                regressed = True
                reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
            if b_det is not None and b_det != detector:
                regressed = True
                reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
            if b_hash is not None and b_hash != fp:
                regressed = True
                reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
            if s.precision < base_p - tolerance:
                regressed = True
                reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
            if s.recall < base_r - tolerance:
                regressed = True
                reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
        else:
            reasons.append("no baseline recorded for this video/stage ? nothing to compare")

        return RegressionResult(
            video_id=video_id, stage=stage, iou_threshold=iou_threshold,
            precision=s.precision, recall=s.recall, f1=s.f1,
            baseline_precision=base_p, baseline_recall=base_r,
            tolerance=tolerance, regressed=regressed, reasons=reasons,
            has_baseline=has_baseline, gt_hash=fp,
        )
```

```
158
159
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                               tier: str = "file", stage: str = "final",
162                               iou_threshold: float = 0.5,
163                               detector: Optional[str] = None,
164                               tolerance: float = DEFAULT_TOLERANCE,
165                               baseline_path: str = DEFAULT_BASELINE_PATH,
166                               guideline: Optional[str] = None,
167                               sport: str = "badminton",
168                               provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                        detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "".join(f"\n   - {why}" for why in r.reasons)
189
```

41. user (2026-06-18T01:34:41.671Z)

```
1       """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3       `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4       replay code path** used by both the freeze tool and the characterization gate, and it
5       deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6       variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7       across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8       only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9       merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10      LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11      `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12      path simply never calls into it.)
13
14      The replay reuses the already-shipped eval stack VERBATIM::
15
16          parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17              ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18              ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20      Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
21
22      * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
```

```
23        provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24      * Every fixture is tagged ``kind="synthetic"``, ``minors_free=true``,
``license="synthetic"``.
25      * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26        person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27      * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28        passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29        named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30        silently churn fixtures.
31      * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32        the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33        takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34        has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35        ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36        tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38      §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39      suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40      frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41      that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42      field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44      Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45      parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46      NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47      the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48      never calls into it.
49      """
50      from __future__ import annotations
51
52      import argparse
53      import json
54      import math
55      import os
56      import re
57      import shutil
58      import sys
59      import tempfile
60      from dataclasses import dataclass
61      from pathlib import Path
62      from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64      # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-------------
65      from backend.eval import distill_local
66      from backend.eval.experiment import CONTROL, VideoCandidates, predict
67      from backend.eval.gt_loader import load_gt_intervals
68      from backend.eval.metrics import score
69      from backend.eval.regression import gt_hash
```

```
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    Interval = Tuple[float, float]

    #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
    #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
    CURRENT_SCHEMA = 1

    #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
    #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
    FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"

    #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
    #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
    #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
    #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
    #: re-states these in its provenance; the replay reads them from there.
    PINNED_VT = 0.02
    PINNED_MERGE_GAP = 2.0
    PINNED_MIN_WINDOW = 2.0
    #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
    #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
    #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
    #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
    #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
    PINNED_MAX_WINDOW = 0.0

    #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
    #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
    SHUTTLE_FEATURES: Tuple[str, ...] = (
        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
    )
    assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
        "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
    )

    #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
    #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
    FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")

    _FLOAT_ROUND_DP = 6  # all written floats rounded to this many decimals (determinism)
    _FLOAT_TOL = 1e-6    # abs tolerance when parse-comparing recomputed vs committed floats

    #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
    #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
    _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")

    #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
```

```
    #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
    #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
    #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
    _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")


    # ============================================================================================
    # Synthetic trajectory generation (deterministic ? NO random)
    # ============================================================================================


    @dataclass(frozen=True)
    class RallySpec:
        """One in-flight rally segment: the shuttle oscillates across the net midline.

        ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
        period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
        net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
        """

        start_frame: int
        n_frames: int
        period: float = 30.0
        amplitude: float = 400.0


    @dataclass(frozen=True)
    class GapSpec:
        """Dead-time between rallies ? what separates one window from the next.

        ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
        ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
        end an active run so the next rally forms a distinct window.
        """

        start_frame: int
        n_frames: int
        kind: str = "still"  # "still" | "occluded"


    @dataclass(frozen=True)
    class FixtureSpec:
        """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.

        ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
        "labels.csv". For a synthetic fixture they line up with the rallies by construction,
        which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
        field-quality one (§6).
        """

        slug: str
        description: str
        fps: float
        frame_width: float
        frame_height: float
        segments: Tuple[Any, ...]  # ordered (RallySpec | GapSpec)
        gts: Tuple[Interval, ...]
        net_midline: float = 640.0


    def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
```

```
                     ) -> List[Tuple[int, int, float, float]]:
        """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
        rows: List[Tuple[int, int, float, float]] = []
        y_mid = frame_height / 2.0
        for i in range(spec.n_frames):
            x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
            # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
            y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
            rows.append((spec.start_frame + i, 1, x, y))
        return rows


    def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
                    ) -> List[Tuple[int, int, float, float]]:
        """Dead-time rows: resting-visible (still) or absent (occluded)."""
        rows: List[Tuple[int, int, float, float]] = []
        y_mid = frame_height / 2.0
        for i in range(spec.n_frames):
            if spec.kind == "occluded":
                rows.append((spec.start_frame + i, 0, 0.0, 0.0))
            elif spec.kind == "still":
                rows.append((spec.start_frame + i, 1, net_midline, y_mid))
            else:  # pragma: no cover - guarded by builder
                raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'|'occluded')")
        return rows


    def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
        """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
        rows: List[Tuple[int, int, float, float]] = []
        for seg in spec.segments:
            if isinstance(seg, RallySpec):
                rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
            elif isinstance(seg, GapSpec):
                rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
            else:  # pragma: no cover - guarded by dataclass shape
                raise TypeError(f"unknown segment type {type(seg).__name__}")
        rows.sort(key=lambda r: r[0])
        return rows


    def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
        """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
floats."""
        lines = ["Frame,Visibility,X,Y"]
        for frame, vis, x, y in rows:
            lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
        path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")


    def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
        """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
        lines = ["start,end"]
        for s, e in gts:
            lines.append(f"{_fmt(s)},{_fmt(e)}")
        path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")


    # ============================================================================
    # Built-in synthetic scenarios (the 3 committed slugs)
```

```
    # ============================================================================

    def _fr(seconds: float, fps: float) -> int:
        return int(round(seconds * fps))


    def builtin_specs() -> List[FixtureSpec]:
        """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
        windowing behaviour (one window / dead-time split / occlusion split)."""
        fps = 30.0
        fw, fh = 1280.0, 720.0
        net = 640.0

        # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
        single = FixtureSpec(
            slug="clean_single_rally",
            description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
            fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
            segments=(RallySpec(start_frame=0, n_frames=120),),
            # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
            gts=((0.0, 4.0),),
        )

        # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
        #    low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
        r0 = RallySpec(start_frame=0, n_frames=120)
        gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
        r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
        multi = FixtureSpec(
            slug="multi_rally_with_deadtime",
            description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
            fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
            segments=(r0, gap_still, r1),
            gts=((0.0, 4.0), (7.0, 11.0)),
        )

        # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
        #    visibility break ? proves occlusion (not just low velocity) separates windows.
        o0 = RallySpec(start_frame=0, n_frames=120)
        gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
        o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
        occ = FixtureSpec(
            slug="occlusion_break",
            description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
            fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
            segments=(o0, gap_occ, o1),
            gts=((0.0, 4.0), (7.0, 11.0)),
        )

        return [single, multi, occ]


    def _spec_by_slug() -> Dict[str, FixtureSpec]:
        return {s.slug: s for s in builtin_specs()}


    # ============================================================================
```

```
    # The shared replay (CONTROL / pure-stdlib path)
    # ============================================================================
    def _fmt(x: float) -> str:
        """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
        r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
        if r == 0.0:
            r = 0.0
        return f"{r:.6f}"


    def _round(x: float) -> float:
        r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
        return 0.0 if r == 0.0 else r


    def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
        """The windowing proposal tuple ``(vt, mg, mwd)``, read from the fixture provenance
when present
        (the pinned params travel with the fixture), else the module-level pinned defaults.
NEVER
        ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
read here ?
        ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
path; the
        pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
PINNED_MAX_WINDOW)."""
        if prov and "windowing" in prov:
            w = prov["windowing"]
            return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
        return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)


    def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
        """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.

        Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
        ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:

            parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
            cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).

        Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
        control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
        merges overlaps; NO numpy is touched.
        """
        fdir = _resolve_dir(slug_or_dir)
        slug = fdir.name
        traj_csv = fdir / "trajectory.csv"
        labels_csv = fdir / "labels.csv"
        prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None

        fps, fw = _fps_fw(prov, slug)
        iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
        proposal = _pinned_proposal(prov)

        intervals, feature_rows = distill_local.build_candidates(
            str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
```

```
350
351         # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
352         # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
353         # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
354         features: Dict[str, List[float]] = {
355             name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
356         }
357         gts = load_gt_intervals(str(labels_csv), strict=True)
358         vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
359
360         preds = predict(CONTROL, vc)  # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
361         sc = score(preds, gts, iou)
362
363         candidates: List[Dict[str, Any]] = []
364         for j, (s, e) in enumerate(intervals):
365             shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
366             # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
367             shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
368             candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
369
370         return {
371             "slug": slug,
372             "schema_version": CURRENT_SCHEMA,
373             "iou": _round(iou),
374             "candidates": candidates,
375             "control_segments": [[_round(s), _round(e)] for s, e in preds],
376             "score": {
377                 "iou_threshold": _round(sc.iou_threshold),
378                 "num_pred": int(sc.num_pred),
379                 "num_gt": int(sc.num_gt),
380                 "true_positives": int(sc.true_positives),
381                 "false_positives": int(sc.false_positives),
382                 "false_negatives": int(sc.false_negatives),
383                 "precision": _round(sc.precision),
384                 "recall": _round(sc.recall),
385                 "f1": _round(sc.f1),
386                 "mean_iou": _round(sc.mean_iou),
387                 "mean_start_error": _round(sc.mean_start_error),
388                 "mean_end_error": _round(sc.mean_end_error),
389             },
390             "gt_hash": gt_hash(gts),
391         }
392
393
394     # ============================================================================
395     # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
396     # ============================================================================
397
398
399     def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
400                     ) -> Dict[str, Any]:
401         """The committed provenance block ? pins the windowing + privacy/licensing tags
(§4c, §5)."""
402         desc = ""
403         if isinstance(spec_or_dir, FixtureSpec):
404             desc = spec_or_dir.description
405         return {
406             "slug": slug,
```

```
            "schema_version": CURRENT_SCHEMA,
            "kind": "synthetic",
            "license": "synthetic",
            "minors_free": True,
            "description": desc,
            "fps": _round(fps),
            "frame_width": _round(fw),
            "iou": _round(iou),
            "windowing": {
                "vt": PINNED_VT,
                "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW,
                "max_win": PINNED_MAX_WINDOW,
            },
            "paths": {
                "trajectory": "trajectory.csv",
                "labels": "labels.csv",
                "expected": "expected.json",
            },
            "note": (
                "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
                "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
(§6)."
            ),
        }


    def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
                       iou: float = 0.5) -> Dict[str, Any]:
        """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
        ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).

        Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
        + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
        """
        fdir = fixtures_dir / slug
        traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
        if not traj_csv.is_file() or not labels_csv.is_file():
            raise FileNotFoundError(
                f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")

        # Provenance first (it pins the windowing the replay reads), then replay ? expected.
        spec = _spec_by_slug().get(slug)
        fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
        prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
        _write_json(fdir / "provenance.json", prov)

        expected = replay_fixture(fdir)
        _write_json(fdir / "expected.json", expected)
        return expected


    def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
        """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
        provenance.json, and (re)write the manifest. Returns the slugs frozen."""
        specs = builtin_specs()
        fixtures_dir.mkdir(parents=True, exist_ok=True)
        slugs: List[str] = []
        for spec in specs:
```

```
            fdir = fixtures_dir / spec.slug
            fdir.mkdir(parents=True, exist_ok=True)
            rows = make_synthetic_trajectory(spec)
            write_trajectory_csv(rows, fdir / "trajectory.csv")
            write_labels_csv(spec.gts, fdir / "labels.csv")
            freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
            slugs.append(spec.slug)
        _write_manifest(specs, fixtures_dir)
        return slugs


    def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
        """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
        NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
        mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
        if all_slugs:
            targets = [d.name for d in sorted(fixtures_dir.iterdir())
                       if d.is_dir() and (d / "trajectory.csv").is_file()]
        elif slug:
            targets = [slug]
        else:
            raise ValueError("refresh_snapshot needs --slug S or --all")
        for s in targets:
            freeze_fixture(s, fixtures_dir=fixtures_dir)
        return targets


    # ============================================================================
    # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
    # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
    # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
    # ============================================================================


    class RefusalError(ValueError):
        """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.

        A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
it) that
        names *why* the unsafe public-fixture path was refused. The refusal is the code-
enforced
        provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
minors-excluded
        / license flags are human-asserted, but the gate makes the unsafe path impossible to
reach
        *silently* ? a fixture cannot be written unless all three are affirmatively set.
        """


    def _new_surrogate_slug() -> str:
        """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).

        Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
random,
        NON-reversible id with no algebraic link to the source, so it can't be brute-forced
over a
        tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
off-git.
        """
        return f"fx_{os.urandom(8).hex()}"

```

```
520
521     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-
path token
522     #: that EXACTLY equals one of these is NOT a leak (the bytes would be written
regardless), so it is
523     #: excluded. Matched by exact membership, NEVER substring containment ? a substring rule
would let
524     #: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed
(red-team #4).
525     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
526         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
527         "provenance", "provenance.json",
528     )
529
530
531     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
532         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
attributable.
533
534         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
535         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
536         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
537         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
538         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
539         """
540         low = text.lower()
541         for key in FORBIDDEN_KEYS:
542             if key in low:
543                 raise RefusalError(
544                     f"{where}: forbidden key substring {key!r} found in written output ? "
545                     f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
546                     f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
547         m = _MD5_RE.search(text)
548         if m:
549             raise RefusalError(
550                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
551                 f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
552         for sp in source_paths:
553             if not sp:
554                 continue
555             p = Path(sp)
556             for token in (sp, p.name, p.stem):
557                 t = (token or "").strip().lower()
558                 # Skip ONLY a token that EXACTLY equals a schema-literal the writers always
emit
559                 # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
560                 # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
561                 # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
562                 # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
563                 if not t or t in _SCHEMA_FILENAME_LITERALS:
564                     continue
565                 if t in low:
566                     raise RefusalError(
567                         f"{where}: source-path token {token!r} found in written output ? the
source "
```

```
                        f"filename/path must never be persisted (§4c metadata strip).")


    def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
                               ) -> List[Tuple[int, int, float, float]]:
        """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.

        Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
        pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
        """
        rows: List[Tuple[int, int, float, float]] = []
        for p in points:
            rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
        rows.sort(key=lambda r: r[0])
        return rows


    def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
        """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant).

        ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller.
Each label
        endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference
of two exact
        6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
        ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-
reset score
        delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
``freeze_real_fixture``
        for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-
second
        offset (the grid, not a real metric divergence)."""
        return [(_round(s) - t0_sec, _round(e) - t0_sec) for s, e in gts]


    def freeze_real_fixture(
        label: str,
        trajectory_csv_path: Any,
        labels_csv_path: Any,
        *,
        license: str,
        minors_free: bool,
        owned: bool,
        fps: float,
        frame_width: float,
        fixtures_dir: Path = FIXTURES_DIR,
        slug: Optional[str] = None,
        time_origin_reset: bool = True,
        source_note: str = "",
    ) -> Dict[str, Any]:
        """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
        Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. (§4c / §5 / §7
row P1.)

        The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
        ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string,
        AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text
```

```
identifier like a
619         player/event name is refused, red-team #1). The committed bytes carry NO filename /
video_id /
620         match / player / capture-date / operator free-text ? only an opaque random surrogate
slug, the
621         shuttle-only trajectory (time-origin reset by default), the reset labels, and a
622         ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
written
623         ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped
in.
624
625         ``source_note`` is **accepted but NOT persisted** ? operator free-text is an
attribution
626         back-channel ``_scan_forbidden`` cannot fully police (an identifying note that
happens to
627         contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never
reaches the
628         committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
629         retained for CLI compatibility / future logging.
630
631         NOTE (§6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
632         only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
633         attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
634         owner-asserted Fork-A / minors / biometric-exclusion flags.
635
636         The fixture is written into a temp dir and promoted (``os.replace``) to the final
slug dir ONLY
637         after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture
dir
638         (red-team #3).
639
640         Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
641         """
642         # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail (§4c / §7 ?)
-------
643         reasons: List[str] = []
644         if minors_free is not True:
645             reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
646                            "of a minor is prohibited ? §3, guardrail 'exclude minors')")
647         if owned is not True:
648             reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
649                            "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
650         if not (isinstance(license, str) and license.strip()):
651             reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
652                            "'provenance hard-fail gate')")
653         # label is committed verbatim into provenance ? it MUST be a strict slug, never
operator free
654         # text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1
free-text leak).
655         if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
656             reasons.append(
657                 f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r}
(lowercase "
658                 f"alnum + '_'/'-', no spaces/punctuation) ? a free-text identifier
(player/event "
659                 f"name) is an attribution leak; got {label!r} (§4c metadata strip, red-team
#1)")
```

```
660         if reasons:
661             raise RefusalError(
662                 "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
663                 "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n  - "
664                 + "\n  - ".join(reasons))
665
666         traj_path = Path(str(trajectory_csv_path))
667         labels_path = Path(str(labels_csv_path))
668
669         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
670         out_slug = slug if slug is not None else _new_surrogate_slug()
671
672         # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
673         points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
674         if not points:
675             raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
676         gts = load_gt_intervals(str(labels_path), strict=True)
677
678         # --- (d) consistent time-origin reset (default ON; metric-invariant within
_FLOAT_TOL, §4c) --
679         # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
metrics.score
680         # is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure
time shift
681         # leaves the score essentially unchanged ? only the absolute timeline is severed. To
keep the
682         # two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-
team #2),
683         # t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each
endpoint first), so
684         # the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
685         #
686         # HONEST LIMIT (not a bug): the trajectory shifts by an INTEGER frame count while
labels shift
687         # in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),
688         # so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE
6dp ULP
689         # (1e-6) on a sub-second offset. That is the serialization grid, not a metric
divergence ? the
690         # score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise
(test #2).
691         if time_origin_reset:
692             t0_frame = min(int(p.frame) for p in points)
693             t0_sec = _round(t0_frame / float(fps))
694         else:
695             t0_frame, t0_sec = 0, 0.0
696         rows = _reset_trajectory_rows(points, t0_frame)
697         reset_gts = _reset_labels(gts, t0_sec)
698
699         # --- (d2) negative-label guard BEFORE any write (red-team #3a)
------------------------
700         # A reset label that starts < 0 means the label timeline predates the earliest
trajectory
701         # frame ? an inconsistent input that would crash the strict gt_loader on replay
(negative
702         # start). Refuse loudly up-front so the partial-write path is never even reached.
703         neg = [(s, e) for (s, e) in reset_gts if s < 0]
704         if neg:
```

```
705             raise RefusalError(
706                 f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset
label(s) have a "
707                 f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g.
{neg[0]!r}). The "
708                 f"earliest label must not precede the earliest trajectory frame (§4c time-
origin reset).")
709
710         iou = 0.5
711         # NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in
`prov` ?
712         # it never reaches the committed bytes (an identifying note is an attribution back-
channel
713         # _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below
stays.
714         prov = {
715             "slug": out_slug,
716             "schema_version": CURRENT_SCHEMA,
717             "kind": "cleared_real",
718             "label": str(label),
719             "license": license,
720             "minors_free": True,
721             "owned": True,
722             "fps": _round(fps),
723             "frame_width": _round(frame_width),
724             "iou": _round(iou),
725             "time_origin_reset": bool(time_origin_reset),
726             "windowing": {
727                 "vt": PINNED_VT,
728                 "mg": PINNED_MERGE_GAP,
729                 "mwd": PINNED_MIN_WINDOW,
730                 "max_win": PINNED_MAX_WINDOW,
731             },
732             "paths": {
733                 "trajectory": "trajectory.csv",
734                 "labels": "labels.csv",
735                 "expected": "expected.json",
736             },
737             "gt_hash": gt_hash(reset_gts),
738             "note": (
739                 "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
740                 "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
741                 "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
742                 "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2)."
743             ),
744         }
745
746         # --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b)
-----
747         # Everything is written under a sibling temp dir; the final slug dir is created by
an atomic
748         # os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore
leaves NO
749         # partial fixture dir behind. source_note (if any) is scanned defensively but never
written.
750         fdir = fixtures_dir / out_slug
751         fixtures_dir.mkdir(parents=True, exist_ok=True)
752         if fdir.exists():
753             raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber
(choose a new slug)")
754         tmp_parent = Path(tempfile.mkdtemp(prefix=f".{out_slug}.tmp.",
```

```
dir=str(fixtures_dir)))
755         tmp_dir = tmp_parent / out_slug
756         tmp_dir.mkdir(parents=True, exist_ok=True)
757         try:
758             write_trajectory_csv(rows, tmp_dir / "trajectory.csv")
759             write_labels_csv(reset_gts, tmp_dir / "labels.csv")
760             _write_json(tmp_dir / "provenance.json", prov)
761 
762             expected = replay_fixture(tmp_dir)   # REUSE M1 ? correct-by-construction
snapshot
763             _write_json(tmp_dir / "expected.json", expected)
764 
765             # --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text +
source_note) ---
766             source_paths = [str(traj_path), str(labels_path)]
767             _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
768                             where=f"{out_slug}/expected.json", source_paths=source_paths)
769             _scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
770                             where=f"{out_slug}/provenance.json", source_paths=source_paths)
771             # source_note is never persisted, but a leak in it is still a contract violation
? scan it.
772             if source_note:
773                 _scan_forbidden(str(source_note),
774                                 where=f"{out_slug}/source_note(not-persisted)",
source_paths=source_paths)
775 
776             # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed)
-----------
777             os.replace(str(tmp_dir), str(fdir))
778         finally:
779             # Clean up the temp parent whether we promoted or refused (a refusal leaves NO
partial dir).
780             shutil.rmtree(str(tmp_parent), ignore_errors=True)
781 
782         # --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
783         _append_manifest_entry(prov, fixtures_dir)
784         return expected
785 
786 
787     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
788         """A manifest row for a single fixture, derived from its provenance block."""
789         slug = prov["slug"]
790         return {
791             "slug": slug,
792             "fps": _round(float(prov["fps"])),
793             "frame_width": _round(float(prov["frame_width"])),
794             "windowing": dict(prov.get("windowing", {})),
795             "kind": prov.get("kind", "synthetic"),
796             "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
797             "paths": {
798                 "dir": slug,
799                 "trajectory": f"{slug}/trajectory.csv",
800                 "labels": f"{slug}/labels.csv",
801                 "expected": f"{slug}/expected.json",
802                 "provenance": f"{slug}/provenance.json",
803             },
804         }
805 
806 
807     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
808         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
809         entries = load_manifest(fixtures_dir)
810         entries = [e for e in entries if e.get("slug") != prov["slug"]]
```

```
811         entries.append(_manifest_entry_for(prov))
812         entries.sort(key=lambda e: str(e.get("slug")))
813         _write_json(fixtures_dir / "manifest.json", entries)
814 
815 
816     # ============================================================================
817     # Manifest + loaders
818     # ============================================================================
819 
820 
821     def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
822         entries = [{
823             "slug": spec.slug,
824             "fps": _round(spec.fps),
825             "frame_width": _round(spec.frame_width),
826             "windowing": {
827                 "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
828                 "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
829             },
830             "kind": "synthetic",
831             "schema_version": CURRENT_SCHEMA,
832             "paths": {
833                 "dir": spec.slug,
834                 "trajectory": f"{spec.slug}/trajectory.csv",
835                 "labels": f"{spec.slug}/labels.csv",
836                 "expected": f"{spec.slug}/expected.json",
837                 "provenance": f"{spec.slug}/provenance.json",
838             },
839         } for spec in specs]
840         _write_json(fixtures_dir / "manifest.json", entries)
841 
842 
843     def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
844         """Load the fixtures manifest (returns [] if absent)."""
845         path = fixtures_dir / "manifest.json"
846         if not path.is_file():
847             return []
848         data = _read_json(path)
849         if not isinstance(data, list):  # pragma: no cover - corrupt manifest
850             raise ValueError(f"{path}: manifest must be a JSON array")
851         return data
852 
853 
854     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
855         """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
856         whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
857         ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
858         bump in one is never masked by an in-range value in the other (red-team #7)."""
859         fdir = fixtures_dir / slug
860         prov = _read_json(fdir / "provenance.json")
861         expected = _read_json(fdir / "expected.json")
862         for which, blob in (("expected", expected), ("provenance", prov)):
863             sv = int(blob.get("schema_version", 0))
864             if sv > CURRENT_SCHEMA:
865                 raise ValueError(
866                     f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
867                     f"upgrade the harness")
868         return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
869 
870 
871     # ============================================================================
```

```
872     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
873     # ================================================================================
874
875
876     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
877         """Return a list of human-readable mismatch messages (empty == match).
878
879         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
880         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
881         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
882         safe (the repo runs with core.autocrlf=true).
883
884         Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
885         per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
886         a fixed-field loop alone would ignore an added field (red-team #5)."""
887         out: List[str] = []
888
889         def exact(path: str, a: Any, b: Any) -> None:
890             if a != b:
891                 out.append(f"{path}: {a!r} != {b!r} (exact)")
892
893         def approx(path: str, a: Any, b: Any) -> None:
894             try:
895                 if abs(float(a) - float(b)) > _FLOAT_TOL:
896                     out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
897             except (TypeError, ValueError):
898                 out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
899
900         # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
901         if set(recomputed) != set(committed):
902             out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")
903
904         exact("slug", recomputed.get("slug"), committed.get("slug"))
905         exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
906         exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
907         approx("iou", recomputed.get("iou"), committed.get("iou"))
908
909         rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
910         if len(rc) != len(cc):
911             out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
912         else:
913             for i, (r, c) in enumerate(zip(rc, cc)):
914                 approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
915                 approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
916                 rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
917                 if set(rs) != set(cs):
918                     out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
919                 else:
920                     for k in rs:
921                         if k == "net_crossings":
922                             exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
923                         else:
924                             approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
925
926         rseg, cseg = recomputed.get("control_segments", []),
```

```
committed.get("control_segments", [])
        if len(rseg) != len(cseg):
            out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
        else:
            for i, (r, c) in enumerate(zip(rseg, cseg)):
                approx(f"control_segments[{i}][0]", r[0], c[0])
                approx(f"control_segments[{i}][1]", r[1], c[1])

        rsc, csc = recomputed.get("score", {}), committed.get("score", {})
        # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
        if set(rsc) != set(csc):
            out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
        int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
        float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
                        "mean_start_error", "mean_end_error")
        for k in int_fields:
            exact(f"score.{k}", rsc.get(k), csc.get(k))
        for k in float_fields:
            approx(f"score.{k}", rsc.get(k), csc.get(k))

        return out


    # ============================================================================
    # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
    # ============================================================================


    def _write_json(path: Path, obj: Any) -> None:
        text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
        path.write_text(text + "\n", encoding="utf-8", newline="\n")


    def _read_json(path: Path) -> Any:
        with open(path, encoding="utf-8") as f:
            return json.load(f)


    def _resolve_dir(slug_or_dir: Any) -> Path:
        if isinstance(slug_or_dir, Path):
            return slug_or_dir
        p = Path(str(slug_or_dir))
        if p.is_dir():
            return p
        return FIXTURES_DIR / str(slug_or_dir)


    def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
        if prov and "fps" in prov and "frame_width" in prov:
            return float(prov["fps"]), float(prov["frame_width"])
        spec = _spec_by_slug().get(slug)
        if spec is not None:
            return spec.fps, spec.frame_width
        raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")


    def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
        prov_path = fdir / "provenance.json"
        prov = _read_json(prov_path) if prov_path.is_file() else None
        return _fps_fw(prov, slug)
```

```
    # ============================================================================================
    # CLI
    # ============================================================================================


    def _cmd_check(fixtures_dir: Path) -> int:
        manifest = load_manifest(fixtures_dir)
        if not manifest:
            print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
            return 1
        failures = 0
        for entry in manifest:
            slug = entry["slug"]
            committed = _read_json(fixtures_dir / slug / "expected.json")
            recomputed = replay_fixture(fixtures_dir / slug)
            mismatches = compare_expected(recomputed, committed)
            if mismatches:
                failures += 1
                print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
                      f"(characterization = behaviour-locking, NOT correctness ? §6):")
                for m in mismatches:
                    print(f"    - {m}")
            else:
                sc = recomputed["score"]
                print(f"[ok]   {slug}: {len(recomputed['candidates'])} candidates, "
                      f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
        print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
              f"(synthetic-tier = plumbing correctness, not field quality ? §6)")
        return 0 if failures == 0 else 1


    def main(argv: Optional[Sequence[str]] = None) -> int:
        ap = argparse.ArgumentParser(
            description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
        g = ap.add_mutually_exclusive_group(required=True)
        g.add_argument("--freeze-synthetic", action="store_true",
                       help="generate synthetic trajectory+labels for all built-in
scenarios, then "
                            "freeze expected.json+provenance.json + write manifest.json")
        g.add_argument("--refresh-snapshot", action="store_true",
                       help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
        g.add_argument("--check", action="store_true",
                       help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
        g.add_argument("--freeze-real", action="store_true",
                       help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
                            "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
        ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
                                       "--freeze-real (default: a random fx_<hex> id)")
        ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
        ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
        # --freeze-real (P1 / M2 refusal gate) options:
        ap.add_argument("--trajectory", help="(--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
        ap.add_argument("--labels", help="(--freeze-real) source golden labels CSV
(start,end)")
```

```
1040        ap.add_argument("--license", dest="license_", default=None,
1041                        help="(--freeze-real) the source license tag (required, non-blank)")
1042        ap.add_argument("--owned", action="store_true",
1043                        help="(--freeze-real) assert owner-recorded Fork-A source
(required)")
1044        ap.add_argument("--minors-free", dest="minors_free", action="store_true",
1045                        help="(--freeze-real) assert the source contains NO minors
(required)")
1046        ap.add_argument("--fps", type=float, help="(--freeze-real) source fps")
1047        ap.add_argument("--frame-width", dest="frame_width", type=float, help="(--freeze-
real) source frame width (px)")
1048        ap.add_argument("--label", default="cleared_real",
1049                        help="(--freeze-real) non-identifying scenario label ? STRICT slug
only "
1050                             "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
1051        ap.add_argument("--source-note", dest="source_note", default="",
1052                        help="(--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
1053                             "attribution back-channel, so it never reaches the committed
provenance "
1054                             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
1055        ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
1056                        help="(--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
1057        ap.set_defaults(time_origin_reset=True)
1058        ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
1059                        help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
1060        args = ap.parse_args(list(argv) if argv is not None else None)
1061
1062        fixtures_dir: Path = args.fixtures_dir
1063
1064        if args.freeze_synthetic:
1065            slugs = freeze_synthetic(fixtures_dir)
1066            print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {',
'.join(slugs)}")
1067            print(f"  -> {fixtures_dir}")
1068            return 0
1069
1070        if args.refresh_snapshot:
1071            if not args.slug and not args.all:
1072                ap.error("--refresh-snapshot requires --slug S or --all")
1073            # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
real
1074            # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
1075            if not (args.reason and args.reason.strip()):
1076                ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
stamp a real "
1077                         "regression; record the deliberate reason and review the visible
diff (§6).")
1078            targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
1079            print(f"[golden-fixtures] refreshed snapshot for: {', '.join(targets)}"
1080                  f"  (reason: {args.reason})")
1081            return 0
1082
1083        if args.freeze_real:
1084            missing = [n for n, v in (("--trajectory", args.trajectory),
1085                                      ("--labels", args.labels),
1086                                      ("--fps", args.fps),
1087                                      ("--frame-width", args.frame_width)) if v is None]
1088            if missing:
```

```
1089                ap.error("--freeze-real requires " + ", ".join(missing))
1090            try:
1091                expected = freeze_real_fixture(
1092                    args.label, args.trajectory, args.labels,
1093                    license=args.license_ if args.license_ is not None else "",
1094                    minors_free=args.minors_free, owned=args.owned,
1095                    fps=args.fps, frame_width=args.frame_width,
1096                    fixtures_dir=fixtures_dir, slug=args.slug,
1097                    time_origin_reset=args.time_origin_reset, source_note=args.source_note)
1098            except RefusalError as e:
1099                # The code-enforced provenance gate (§4c / §7 ?): refuse loudly, exit non-
zero.
1100                print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
1101                return 2
1102            print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
"
1103                  f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
1104            print("  §6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
1105                  "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
1106            return 0
1107
1108        # --check
1109        return _cmd_check(fixtures_dir)
1110
1111
1112    if __name__ == "__main__":
1113        raise SystemExit(main())
1114
```

42. assistant (2026-06-18T01:34:46.152Z)

This is comprehensive. Let me now scrutinize a couple of remaining suspicious spots and verify behavior empirically. Let me re-examine the `bootstrap_ci` CI index math and the `paired_delta_ci` for the case where `ci_excludes_zero` could be miscomputed, plus the `_percentile` and `_spread` rounding. I'll run a quick test of the statistical functions since they're pure stdlib.

43. user (2026-06-18T01:34:51.319Z)

```
all-equal: (0.5, 0.5)
mixed: (0.0, 1.0)
n=6: (0.21666666666666667, 0.48333333333333334)
```

44. assistant (2026-06-18T01:34:57.213Z)

Bootstrap looks fine. Now let me scrutinize the one area I flagged as most promising: the R4 `_trim_inactive_tail` and the interaction between `merge_gap` frame computation and window boundaries. Let me also carefully re-examine `_points_in` in rally_gate which uses `start * fps <= p.frame <= end * fps`. There's a subtle issue: `gate_windows` is called from `windows_for` in rally_seg_eval with `ivs` derived from `windows_to_intervals`. Those window start/end are `frame/fps + chunk_start`. With chunk_start=0, the boundaries are exact. But float rounding `start*fps` could exclude the boundary frame. Let me test concretely.

45. user (2026-06-18T01:34:59.962Z)

```
frame=100 s=1.6683350016683351 s*fps=100.0 includes_frame=True
frame=250 s=4.170837504170838 s*fps=250.00000000000003 includes_frame=False
frame=1799 s=30.01334668001335 s*fps=1799.0 includes_frame=True
frame=3000 s=50.050050050050054 s*fps=3000.0 includes_frame=True
```

46. assistant (2026-06-18T01:35:09.008Z)

Interesting ? for `frame=250`, `s*fps = 250.00000000000003 > 250`, so the boundary frame would be EXCLUDED by `start*fps <= p.frame`. This is a floating-point edge effect in `_points_in`. But is it a real defect? This is a window-boundary inclusivity issue: a single boundary frame may be excluded from the net-crossing count. Given it affects at most the first/last frame of a window and the gate counts many crossings, the practical impact is negligible ? this is borderline noise, not a maintainer-recognized defect. I'll keep it in mind but it's low-confidence.

Let me look at something more impactful. Let me re-examine the `kfold_indices` round-robin in calibration.py. `folds = [list(range(i, n, k)) for i in range(k)]`. This is a round-robin assignment. With `k = max(2, min(k, n))`. The fold sizes are balanced. Each test fold and train. This looks correct.

Let me now look closely at a potential real issue in `over_seg_guard`'s `_aligned` ? and the `paired_delta_ci` when `a_values`/`b_values` have different lengths (it uses `min`, dropping the tail silently). Let me also examine the `mean_average_precision` and the AP precision-envelope loop. Let me check the `_percentile` function edge ? when `p=100`, `k = (len-1)*100/100 = len-1`, `lo=len-1`, `hi=min(len, len-1)=len-1`. Fine.

Let me look at the remaining unread eval files: batch.py, per_user_regression.py, calibrate_local.py, distill_local.py (build_candidates used by fixtures), and rally_seq_proto.py.

47. user (2026-06-18T01:35:09.288Z)

```
1       """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3       Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4       further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5       using a few resolution/fps-INVARIANT local features, so the model transfers across
6       videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8       Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10        filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11        active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12        -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Optional, Tuple
28
29
30      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31      from backend.pipeline.detectors import rally_gate
32      from backend.eval.calibrate_wasb import load_golden_gts
33      from backend.eval.training import label_candidates
34      from backend.eval.classifier import LogisticRegression
35      from backend.eval.metrics import score
36      from backend.eval.gemini_refine import merge_overlaps
37      from backend.eval import windowing as _windowing
38
39      Interval = Tuple[float, float]
40
```

```
    FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
    # Recall-oriented proposal: deliberately permissive so real rallies are among the
    # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
    # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
    # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
    # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
    # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
    PROPOSAL = _windowing.PROPOSAL.as_tuple()
    BASELINE_LOCAL = _windowing.SERVED.as_tuple()   # the served (production) windowing, no
learning


    def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
        """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = proposal
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        intervals = [(w["start"], w["end"]) for w in wins]
        gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-
crossings
        X: List[List[float]] = []
        for w, g in zip(wins, gated):
            dur = max(1e-6, w["end"] - w["start"])
            win_frames = max(1.0, dur * fps)
            X.append([
                dur,                                            # rally length (sec) ?
fps-invariant
                float(w["peak_velocity"]),                      # already normalized by
frame_width
                float(w["mean_velocity"]),
                float(w["active_frame_count"]) / win_frames,    # active-shuttle ratio ?
fps-invariant
                float(g.crossings),                             # net crossings (count) ?
the rally signal
            ])
        return intervals, X


    def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = BASELINE_LOCAL
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        return [(w["start"], w["end"]) for w in wins]


    def load_video(spec: Dict, iou: float = 0.5) -> Dict:
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        intervals, X = build_candidates(spec["trajectory"], fps, fw)
        gts = load_golden_gts(spec["labels"])
        y = label_candidates(intervals, gts, iou)
        return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
                "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
```

```
92
93
94      def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
95                          threshold: float = 0.5) -> List[Interval]:
96          if not intervals:
97              return []
98          proba = model.predict_proba(X)
99          pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
100         return merge_overlaps(pos, gap_tol=1.0)
101
102
103     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
104             model_out: Optional[str] = None) -> dict:
105         videos = [load_video(s, iou) for s in specs]
106         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
107               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
108         for v in videos:
109             ceil = score(v["intervals"], v["gts"], iou).recall
110             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
111                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
112
113         result: dict = {"videos": [v["name"] for v in videos]}
114         if len(videos) >= 2:
115             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
116             clf, base = [], []
117             for i, held in enumerate(videos):
118                 others = [v for j, v in enumerate(videos) if j != i]
119                 X = [row for v in others for row in v["X"]]
120                 y = [t for v in others for t in v["y"]]
121                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
122                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
123                 sc = score(preds, held["gts"], iou)
124                 bsc = score(held["baseline"], held["gts"], iou)
125                 clf.append(sc)
126                 base.append(bsc)
127                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
128                       f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
129             def mean(xs, a):
130                 return sum(getattr(s, a) for s in xs) / len(xs)
131             print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f}
R={mean(clf,'recall'):.3f} "
132                   f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
133                   f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
134             result["mean_learned_f1"] = mean(clf, "f1")
135             result["mean_baseline_f1"] = mean(base, "f1")
136
137         # Final model on ALL videos (for production / running on a fresh held-out video).
138         Xall = [row for v in videos for row in v["X"]]
139         yall = [t for v in videos for t in v["y"]]
140         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141         assert final.w is not None  # set by fit()
142         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144         if model_out:
145             final.save(model_out)
146             print(f"final model (all videos) -> {model_out}")
147             result["model_out"] = model_out
148         return result
149
150
151     def main() -> None:
```

```
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166
```

48. assistant (2026-06-18T01:35:16.114Z)

Now let me carefully scrutinize the `paired_sign_test` two-sided p-value computation in eval_stats.py ? this is a common source of statistical bugs. The exact two-sided binomial test at p=0.5 should be `2 * P(X <= k)` where `k = min(wins, losses)`, but capped at 1.0. Let me verify the math is correct for a critical edge case: when `wins == losses`, `k = wins`, and `tail = P(X <= wins)`. Since `wins = losses = n_eff/2`, `P(X <= n_eff/2)` already exceeds 0.5, so `2*tail > 1`, capped at 1.0. Correct (no evidence either way ? p=1.0).

But there's a subtle issue. The exact two-sided sign test p-value formula `2 * P(X <= k)` is the standard formula, but it double-counts the central probability when `wins==losses`. Capping at 1.0 handles that. This is the standard implementation. Fine.

Let me now look at the `per_user_regression.py`, `batch.py`, and `calibrate_local.py` for completeness, plus the `served_gate_a.py`.

49. user (2026-06-18T01:35:17.531Z)

```
1       """Per-user NO-REGRESSION guard ? the missing per-user analogue of `regression.py`.
2
3       The shared golden set has a disciplined no-regression baseline (`regression.py` +
4       ``regression_baseline.json``): a candidate is gated so a global change can never
silently
5       make the *shared* corpus worse. Personalization has no such rail ? a deliberately-tuned
6       per-user model "has no shared baseline to regress against", so a per-user retrain or a
7       baseline upgrade could quietly make ONE user worse than the model they're already
running.
8       This module builds that missing per-user regression baseline, mirroring the shared
discipline.
9
10      The decision it makes is deliberately narrow and CONSERVATIVE ? it is a *guarantee
against
11      regression*, not a promotion decision (that is `per_user_gate.should_promote_for_user`):
12
13        * **CONTROL** = the user's *frozen incumbent* ? the variant/config currently active
FOR THAT
14          USER. For a brand-new user with nothing personalized yet, the incumbent simply IS
the
15          shipped baseline. This is the thing we must never regress below.
16        * **CANDIDATE** = the proposed replacement (a per-user retrain, or a global baseline
upgrade
17          rolled out to this user).
18
19      Both are evaluated on the user's own held-out videos (`experiment.evaluate_variant`,
LOVO-honest
20      for a learned recipe ? no new scoring math), paired by video, and summarised with the
```

```
shipped
21      honest statistics (`eval_stats.paired_delta_ci` ? bootstrap ?F1 CI + exact sign test).
Then:
22
23        * **regression** ? the candidate is *significantly worse*: the paired ?F1 (candidate ?
control)
24          95% CI excludes 0 on the WRONG side (``ci_high < 0``). ? REJECT, keep the incumbent.
25        * **clear improvement** ? the candidate is *significantly better*: the CI excludes 0
on the
26          right side AND the sign test agrees (candidate wins on strictly more videos than it
loses).
27          ? ACCEPT the swap.
28        * **tie / insufficient evidence** ? the CI spans 0 (the small-N near-null regime):
there is no
29          evidence the swap helps, so we never take the risk. ? KEEP THE INCUMBENT.
30
31      So we only ACCEPT a swap we can *defend* and only ever toward a measured improvement;
everything
32      ambiguous keeps the incumbent. That is strictly stronger than "don't regress" (it also
refuses
33      risk-neutral churn), which is exactly the safety the personalization feature was
missing.
34
35      A small per-user baseline record is keyed by a ``user_id::stage`` string (``user_id``
nullable ?
36      ``None`` = the local single-user deployment; design-for-north-star, implement-for-
current) and
37      saved/loaded as JSON, a sibling to `regression.py`'s ``regression_baseline.json``
approach.
38
39      Pure-CPU, deterministic given ``seed`` (the only randomness is the CI bootstrap); no GPU
/ torch /
40      cv2 / file I/O beyond the explicit baseline save/load. See `docs/EXPERIMENT_HARNESS.md`
+
41      `docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
42      """
43      from __future__ import annotations
44
45      import json
46      import logging
47      import os
48      from dataclasses import asdict, dataclass, field
49      from typing import Any, Dict, List, Optional, Sequence
50
51      from backend.eval import eval_stats as _stats
52      from backend.eval import experiment
53      from backend.eval.experiment import Variant, VideoCandidates
54
55      logger = logging.getLogger(__name__)
56
57      __all__ = [
58          "DEFAULT_PER_USER_BASELINE_PATH",
59          "PerUserRegressionResult",
60          "PerUserBaseline",
61          "baseline_key",
62          "load_baselines",
63          "save_baseline",
64          "evaluate_no_regression",
65      ]
66
67      # Tracked location (NOT under the gitignored output/), sibling to
68      # regression.DEFAULT_BASELINE_PATH so a fresh checkout / CI can load a committed per-
user
69      # baseline. NULL user_id (local single-user) keys as the literal "local".
70      DEFAULT_PER_USER_BASELINE_PATH = os.path.join("eval_baselines",
```

```
"per_user_baseline.json")
71
72
73      # ------------------------------------------------------------------------ result
74
75      @dataclass
76      class PerUserRegressionResult:
77          """The per-user no-regression decision plus the evidence behind it.
78
79          - ``accept`` ? swap the candidate in for this user (only ever ``True`` on a
*defended*
80            improvement: the ?F1 CI clears 0 on the right side AND the sign test agrees).
81          - ``keep_incumbent`` ? the user keeps their frozen incumbent (control). Always the
exact
82            complement of ``accept`` ? a swap is taken or it is not.
83          - ``regressed`` ? the candidate is *significantly worse* than the incumbent (CI
excludes 0
84            on the wrong side). A True here always implies ``accept=False``.
85          - ``reason`` ? short human-readable explanation, for logging / audit.
86          - the remaining fields surface the paired evidence (per-variant mean held-out F1,
mean ?F1,
87            CI bounds, and the sign test) so a caller can audit exactly why the swap was/
wasn't taken.
88          """
89
90          accept: bool
91          keep_incumbent: bool
92          reason: str
93          n: int
94          control_f1: float
95          candidate_f1: float
96          mean_delta: float
97          ci_low: float
98          ci_high: float
99          ci_excludes_zero: bool
100         regressed: bool
101         sign_test: Dict[str, float] = field(default_factory=dict)
102
103         def as_dict(self) -> Dict[str, Any]:
104             """Plain-dict view (for JSON logging / telemetry)."""
105             return asdict(self)
106
107
108     # ------------------------------------------------------------------ per-user baseline
store
109
110     @dataclass
111     class PerUserBaseline:
112         """A snapshotted per-user incumbent record ? the per-user analogue of a regression
baseline.
113
114         Keyed by ``user_id::stage``. ``user_id`` is nullable (``None`` ? the local single-
user
115         deployment, keyed as the literal ``"local"``). ``control_spec_hash`` pins WHICH
incumbent
116         variant the recorded numbers were measured for, so a later check against a different
117         incumbent is detectable rather than silently trusted (mirrors regression.py's
118         iou/detector/gt fingerprint guards).
119         """
120
121         user_id: Optional[str]
122         stage: str
123         control_f1: float
124         control_spec_hash: str
125         n: int
```

```
        iou: float

        def as_dict(self) -> Dict[str, Any]:
            return asdict(self)


    def baseline_key(user_id: Optional[str], stage: str) -> str:
        """Stable ``user_id::stage`` key. A NULL ``user_id`` (local single-user) keys as
``local``."""
        return f"{user_id if user_id is not None else 'local'}::{stage}"


    def load_baselines(path: str = DEFAULT_PER_USER_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-user baseline file (returns ``{}`` if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path, encoding="utf-8") as f:
            return json.load(f)


    def save_baseline(baseline: PerUserBaseline,
                      path: str = DEFAULT_PER_USER_BASELINE_PATH) -> str:
        """Snapshot a user's incumbent numbers as the new per-user baseline, keyed
``user_id::stage``.

        Round-trips through `load_baselines`; returns the key written. Mirrors
        `regression.save_baseline` (atomic-ish read-modify-write of the one JSON file).
        """
        baselines = load_baselines(path)
        key = baseline_key(baseline.user_id, baseline.stage)
        baselines[key] = baseline.as_dict()
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved per-user baseline %s (control_f1=%.3f n=%d)",
                    key, baseline.control_f1, baseline.n)
        return key


    # ---------------------------------------------------------------------- the guard

    def _mean(xs: Sequence[float]) -> float:
        return sum(xs) / len(xs) if xs else 0.0


    def evaluate_no_regression(
        videos: Sequence[VideoCandidates],
        control: Variant,
        candidate: Variant,
        *,
        user_id: Optional[str] = None,
        stage: str = "final",
        iou: float = 0.5,
        honest: bool = True,
        p_threshold: float = 0.05,
        confidence: float = 0.95,
        n_boot: int = 2000,
        seed: int = 0,
        eps: float = 0.0,
        baseline_path: Optional[str] = None,
    ) -> PerUserRegressionResult:
        """Guard a per-user swap: ACCEPT only if ``candidate`` does NOT regress vs the
``control``.

        ``control`` is the user's FROZEN incumbent (their currently-active variant ? for a
```

```
        brand-new user this is just the shipped baseline). ``candidate`` is the proposed
        replacement (a per-user retrain or a baseline upgrade). Both are scored on the
user's own
        ``videos`` via `experiment.evaluate_variant` (LOVO-honest for a learned recipe ? no
new
        scoring math), the per-video held-out F1s are paired by video, and the paired ?F1
        (candidate ? control) is summarised with `eval_stats.paired_delta_ci` (bootstrap CI
+
        exact sign test). The decision (faithful to the module docstring):

          * **regression** (``ci_high < 0`` ? candidate significantly worse): REJECT, keep
incumbent.
          * **clear improvement** (CI excludes 0 on the right side AND the sign test agrees
?
            candidate wins on strictly more videos than it loses): ACCEPT the swap.
          * **tie / insufficient evidence** (CI spans 0): KEEP THE INCUMBENT (conservative;
never
            accept a risky swap).

        ``user_id``/``stage`` key the baseline record (``user_id=None`` ? local single-
user). When
        ``baseline_path`` is given the incumbent's measured numbers are snapshotted there
(the swap
        is then evaluated as usual). Deterministic given ``seed``. Raises `ExperimentError`
(from
        `evaluate_variant`) on unlabeled videos or a learned recipe with too few videos for
LOVO.
        """
        eval_control = experiment.evaluate_variant(videos, control, iou, honest)
        eval_candidate = experiment.evaluate_variant(videos, candidate, iou, honest)

        # Per-video held-out F1, video-aligned (evaluate_variant iterates ``videos`` in
order, so
        # both lists are pairable by position ? exactly how honest_summary pairs B ? A).
        control_f1s: List[float] = [p["f1"] for p in eval_control["per_video"]]
        candidate_f1s: List[float] = [p["f1"] for p in eval_candidate["per_video"]]
        n = min(len(control_f1s), len(candidate_f1s))

        control_mean = _mean(control_f1s)
        candidate_mean = _mean(candidate_f1s)

        if n == 0:
            # No paired videos at all ? no evidence either way ? keep the incumbent.
            sign_test: Dict[str, float] = _stats.paired_sign_test([], eps)
            result = PerUserRegressionResult(
                accept=False, keep_incumbent=True,
                reason="no held-out videos to evaluate; keeping incumbent",
                n=0, control_f1=control_mean, candidate_f1=candidate_mean,
                mean_delta=0.0, ci_low=0.0, ci_high=0.0, ci_excludes_zero=False,
                regressed=False, sign_test=sign_test,
            )
        else:
            delta = _stats.paired_delta_ci(
                control_f1s, candidate_f1s,
                confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
            )
            mean_delta = float(delta["mean_delta"])
            ci_low = float(delta["ci_low"])
            ci_high = float(delta["ci_high"])
            ci_excludes_zero = bool(delta["ci_excludes_zero"])
            sign_test = delta["sign_test"]
            sign_p = float(sign_test.get("p_value", 1.0))
            sign_wins = float(sign_test.get("wins", 0.0))
            sign_losses = float(sign_test.get("losses", 0.0))

```

```
242             # A regression = the candidate is significantly WORSE: the ?F1 CI excludes 0
entirely
243             # on the wrong side (its whole high end is below 0). Never accept such a swap.
244             regressed = ci_excludes_zero and ci_high < 0.0
245             # A defended improvement = the CI clears 0 on the RIGHT side AND the sign test
agrees
246             # (more candidate-wins than losses, significant) ? the same promotion-grade bar
the
247             # global eval uses, applied per user. Everything else is a tie / insufficient
evidence.
248             sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
249             improved = ci_excludes_zero and ci_low > 0.0 and sign_agrees
250
251             if regressed:
252                 accept = False
253                 reason = (
254                     f"REGRESSION: candidate ?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] "
255                     f"is entirely below 0 (n={n}); keeping incumbent"
256                 )
257             elif improved:
258                 accept = True
259                 reason = (
260                     f"accepted: candidate ?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] excludes
0 "
261                     f"(improvement), sign test {int(sign_wins)}-{int(sign_losses)} "
262                     f"(p={sign_p:.4f} <= {p_threshold}); swapping in candidate"
263                 )
264             else:
265                 accept = False
266                 reason = (
267                     f"tie / insufficient evidence: candidate ?F1 95% CI "
268                     f"[{ci_low:.4f}, {ci_high:.4f}] spans 0 (n={n}); keeping incumbent "
269                     "(no defended improvement, never a risky swap)"
270                 )
271
272             result = PerUserRegressionResult(
273                 accept=accept, keep_incumbent=not accept, reason=reason,
274                 n=n, control_f1=control_mean, candidate_f1=candidate_mean,
275                 mean_delta=mean_delta, ci_low=ci_low, ci_high=ci_high,
276                 ci_excludes_zero=ci_excludes_zero, regressed=regressed, sign_test=sign_test,
277             )
278
279         # Snapshot the incumbent's measured numbers as this user's baseline when asked ? the
280         # per-user analogue of regression.save_baseline. Does not change the decision above.
281         if baseline_path is not None:
282             save_baseline(
283                 PerUserBaseline(
284                     user_id=user_id, stage=stage,
285                     control_f1=control_mean, control_spec_hash=control.spec_hash(),
286                     n=n, iou=iou,
287                 ),
288                 path=baseline_path,
289             )
290
291         return result
292
```

50. user (2026-06-18T01:35:17.800Z)

```
1       """Batch evaluation across a collector export manifest.
2
3       Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4       sports-data-collector's `curate export`, (optionally) process each video through
5       a segmenter so detections land in the indexer DB, then score every video's
6       detected rallies against its ground-truth CSV and aggregate the results.
```

```

    This module holds the *pure* pieces (path resolution, stage selection, metric
    aggregation) so they can be unit-tested without touching video files, the GPU,
    or any API. The orchestration/side-effects live in `scripts/run_phase3_eval.py`.
    """

    import os
    from typing import Dict, List, Optional


    # Segmenters that only ever populate the `final` play_segments table (no raw
    # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
    _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}


    def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
        """Resolve a manifest `local_path` to an absolute path.

        The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
        to its own repo root and with Windows separators. Normalise separators, strip
        a leading ``./``, and resolve relative paths against `videos_root` (the
        collector root). Returns None for a missing/empty path.
        """
        if not local_path:
            return None
        p = local_path.replace("\\", "/")
        if p.startswith("./"):
            p = p[2:]
        # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
        # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
        # This keeps collector manifest paths working in tests and mixed environments.
        if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
            return os.path.normpath(p)
        return os.path.normpath(os.path.join(videos_root, p))


    def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
        """Pick which prediction stages to score.

        ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
        but only ``final`` for segmenters that don't emit candidates. An explicit
        request ('candidates' | 'final' | 'both') overrides the heuristic.
        """
        if requested == "both":
            return ["candidates", "final"]
        if requested in ("candidates", "final"):
            return [requested]
        # auto
        if segmenter in _FINAL_ONLY_SEGMENTERS:
            return ["final"]
        return ["candidates", "final"]


    def _safe_div(num: float, den: float) -> float:
        return num / den if den else 0.0


    def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
        """Aggregate per-video stage scores into corpus-level metrics.

        `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
        ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
        compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
        a TP-weighted mean IoU, plus the macro mean of per-video F1.
        """
```

```
        out: Dict[str, dict] = {}
        for stage in stages:
            tp = fp = fn = 0
            iou_weighted = 0.0
            f1s: List[float] = []
            n_videos = 0
            for rep in per_video:
                sr = rep.get("stages", {}).get(stage)
                if not sr:
                    continue
                s = sr["score"]
                tp += s["true_positives"]
                fp += s["false_positives"]
                fn += s["false_negatives"]
                iou_weighted += s["mean_iou"] * s["true_positives"]
                f1s.append(s["f1"])
                n_videos += 1
            precision = _safe_div(tp, tp + fp)
            recall = _safe_div(tp, tp + fn)
            out[stage] = {
                "videos": n_videos,
                "tp": tp, "fp": fp, "fn": fn,
                "micro_precision": precision,
                "micro_recall": recall,
                "micro_f1": _safe_div(2 * precision * recall, precision + recall),
                "macro_f1": _safe_div(sum(f1s), len(f1s)),
                "mean_iou": _safe_div(iou_weighted, tp),
            }
        return out


    def format_aggregate(agg: Dict[str, dict]) -> str:
        """Human-readable corpus-level summary table."""
        lines = [
            "=" * 78,
            " CORPUS AGGREGATE",
            "=" * 78,
            f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
            f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
            "-" * 78,
        ]
        for stage, a in agg.items():
            lines.append(
                f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
                f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
                f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
            )
        lines.append("=" * 78)
        return "\n".join(lines)

```

51. assistant (2026-06-18T01:35:22.482Z)

I've now covered the bulk of the eval + pipeline windowing numeric/statistical surface. Let me do one final focused pass on the `_trim_inactive_tail` sliding-window logic, which is the most algorithmically intricate windowing code and a prime off-by-one candidate. Let me trace it with a concrete example to verify correctness.

52. user (2026-06-18T01:35:29.604Z)

```
last frame kept: 38 len 39
all fast last: 39 len 40
all slow last: 39 len 40
```

53. assistant (2026-06-18T01:35:41.032Z)

Wait ? for the mixed group (30 fast frames 0-29, then 30 slow frames 30-59), the trim kept frame 38, len 39. That means it kept 9 slow frames past the rally end. Let me examine this more carefully ? is this correct behavior? The trailing window is `trail_frames=15`. At frame `i=38`, the window covers frames `(38-15, 38] = (23, 38]`, i.e. frames 24..38. Of those, frames 24-29 are fast (6 fast) and 30-38 are slow (9 slow), window size 15. Density = 6/15 = 0.4 >= 0.34, so it's "dense". At frame 39: window (24,39] = 25..39, fast=25..29 (5), size 15, density 5/15=0.333 < 0.34 ? not dense. So last_dense=38.

This is the documented behavior (trailing-window density), working as designed ? the window keeps frames until the trailing density drops. Not a bug, just the inherent lag of a trailing-window detector. Correct per spec.

Let me verify there isn't a subtle issue in the sliding window where `frames` may have gaps (non-contiguous frame indices after gap-splitting). The window condition uses actual frame values, which handles gaps. Good.

Now let me double-check one thing in `merge_split_report` that I want to be confident about: the `complex` component case. A `(>1, >1)` tangled component contributes nothing to `gts_merged` or `gts_split`, so those GTs are uncounted in both rates. That's a deliberate design choice (documented), not a bug.

Let me verify the `over_seg_report`/`merge_count` consistency with `over_seg_guard`. In `over_seg_guard`, when `use_rates=False`, the cost uses `merge_count`/`split_count`. The `_cost` reads `row["merge_count"]` and `row["split_count"]`. In `rally_seg_eval.score_intervals`, the row has `merge_count`/`split_count` from `osr` (over_seg_report) AND `merge_rate`/`split_rate` from `ms`. Both present. Consistent.

Let me do a final check on the `average_precision` monotone envelope and integration. I want to confirm the recall steps are handled when multiple predictions share the same recall (no new GT matched ? recall flat, skipped correctly).

54. user (2026-06-18T01:35:44.948Z)

```
perfect AP@0.5: 1.0
FP-first AP@0.5: 0.6666666666666666
mAP: 1.0
```

55. assistant (2026-06-18T01:35:57.159Z)

AP math is correct (perfect=1.0; FP-first gives the expected 0.667 via the precision envelope).

I've now thoroughly cross-cut the eval + pipeline windowing surface by bug class: division-by-zero (all guarded with `if ... else 0.0`), n=0/1 means (bootstrap/sign-test handle these), NaN/inf (sigmoid clipped, pct_improvement guards `before<=0`), fps/frame?time rounding, interval boundary inclusivity, IoU/tolerance match edges, bootstrap/sign-test/CI math, and cap/trim boundary conditions.

The one floating-point edge I found in `rally_gate._points_in` (`start * fps <= p.frame` can exclude a boundary frame due to FP error, e.g. frame 250 at fps 59.94) is real but its practical impact is negligible ? it can drop at most the single boundary frame of a window from a net-crossing count that requires multiple crossings, and it never changes a keep/drop decision in any realistic scenario. A maintainer would treat this as noise, not a defect. Given the precision-over-recall mandate and that this doesn't reach a concrete consequential trigger, I will not report it as a finding.

After reading the code carefully across all these files, I found no concrete, trigger-reachable defect that a maintainer would call a real bug. The statistical and windowing code is unusually careful ? every aggregate guards its empty/singleton cases, the percentile/CI index math is bounds-clamped, the recursive split/chop functions provably terminate, and the tolerance/IoU matchers correctly require positive overlap. The empty list is the honest answer here.

56. user (2026-06-18T01:35:58.000Z)

Structured output provided successfully

57. assistant (2026-06-18T01:36:14.415Z)

I completed an adversarial cross-cut of the eval + pipeline windowing surface by the numeric/statistical/off-by-one bug class and found no reportable defect.

What I reviewed

Read in full and traced concretely:
- `backend/eval/tolerance_metrics.py` ? strict 1:1 tolerance match (greedy + Hungarian), `_percentile`, over-seg guard rate math, F1 zero-guards
- `backend/eval/metrics.py` ? `interval_iou` degeneracy/zero-overlap guard, greedy matcher, P/R/F1/mean-error denominators
- `backend/eval/eval_stats.py` ? `bootstrap_ci` (n=0/1 + percentile index clamping), `paired_sign_test` exact two-sided binomial, `paired_delta_ci`, LOGO folds
- `backend/pipeline/detectors/tracknet_runner.py` ? `trajectory_to_action_windows`, the R4 `_trim_inactive_tail` sliding window, `_split_overlong_groups` (hard-cap midpoint), `_blob_fallback_chop` ? verified termination and boundary inclusivity
- `backend/pipeline/detectors/rally_gate.py` ? `_spread`/`_median` percentile indices, `count_net_crossings`, `_points_in` frame/time round-trip
- `backend/eval/` ? segmentation_metrics (AP envelope + mAP), calibration (kfold/LOVO), classifier (logreg std guard), score_calibration (Platt/isotonic/PAVA/ECE binning), splits, gt_loader, annotations (timestamp parsing), promotion, per_user_gate, per_user_eval, per_user_regression, experiment, harness, regression, golden_fixtures, batch, distill_local, calibrate_wasb, gemini_refine, eval_partial_labels

Excluded per instructions: ablation.py, fusion_compare.py, fusion_audio.py, scripts/nightly_regression.py.

Findings: none

Every aggregate guards its empty/singleton case (`... if den else 0.0`), the bootstrap/percentile index math is bounds-clamped, the sign-test caps the two-sided tail at 1.0, the recursive split/chop functions provably make progress (`split_i ? range(1, len(g))` keeps both halves non-empty and strictly smaller), and the IoU/tolerance matchers correctly require strictly-positive overlap. I empirically confirmed bootstrap CIs, AP envelope, and the R4 trailing-window trim behave as documented.

One borderline observation I deliberately did **not** report: in `rally_gate._points_in`, the boundary test `start * fps <= p.frame` can exclude a single boundary frame due to float error (e.g. frame 250 at fps 59.94 gives `start*fps = 250.0000000000003`). It is real but inconsequential ? it can drop at most one boundary frame from a multi-crossing count and never flips a keep/drop decision ? so a maintainer would treat it as noise, not a defect. Per the precision-over-recall mandate, the empty list is the honest answer.

The read-only worktree (`origin/master`, HEAD `4bd9d2d`) was not modified.